Large Language Model

류영표 강사
ryp1662@gmail.com



Youngpyo Ryu. All rights reserved.
Unauthorized reproduction or distribution is prohibited.

류 영 표

AI & Applied Mathematics Researcher · Lecturer

ryp1662@gmail.com



학력

- 동국대학교 일반대학원 응용수학 석사 수료 (2016-2019)
- 동국대학교 수학과 졸업 (2010-2016)

심사 및 전문가 자문 활동

- 인공지능 학습용 데이터 구축 사업 품질검증 전문가 자문위원 (2023)
- 2023 신뢰할 수 있는 인공지능 개발 안내서(안)_고도화 전문위원
- 2023 자율주행/의료/공공사회 분야 신뢰할 수 있는 인공지능 개발 안내서(안) 자문위원

연구 및 프로젝트

- 부산대학교 병원: 개인 맞춤형 당뇨병 예방·관리 인공지능 시스템 외주 프로젝트 (2022)
- 연세대학교 병원: 영상의학과 FLASK 고도화 연구 (2023, 2024)
- 산업수학 스터디 그룹 (국가수리과학연구소): 피부암/유방암 분류, 유전자 정보 분석
- 폐플라스틱 이미지 객체 검출 경진대회 3위 (2021)
- 동국대학교 수학과 연구원(2025.10 ~ / 양자 AI, XAI 연구 진행 중)

논문

- Quantum computing for the optimization of CT image reconstruction, IEEE ICTC 2022

㈜ 헬로윌드랩스 전담교육팀 과장

강의 및 교육 활동

대학 강의

- 고려대학교, 인천대학교, 동양미래대학교, 목포대학교
- 이젠 아카데미: 파이썬/ADSP 과정

기업 교육

- 현대자동차 연구원
- 한화, 하나금융, 한전 KDN, IBK 기업은행, 아가방앤컴퍼니, IM증권

공공/국비 과정

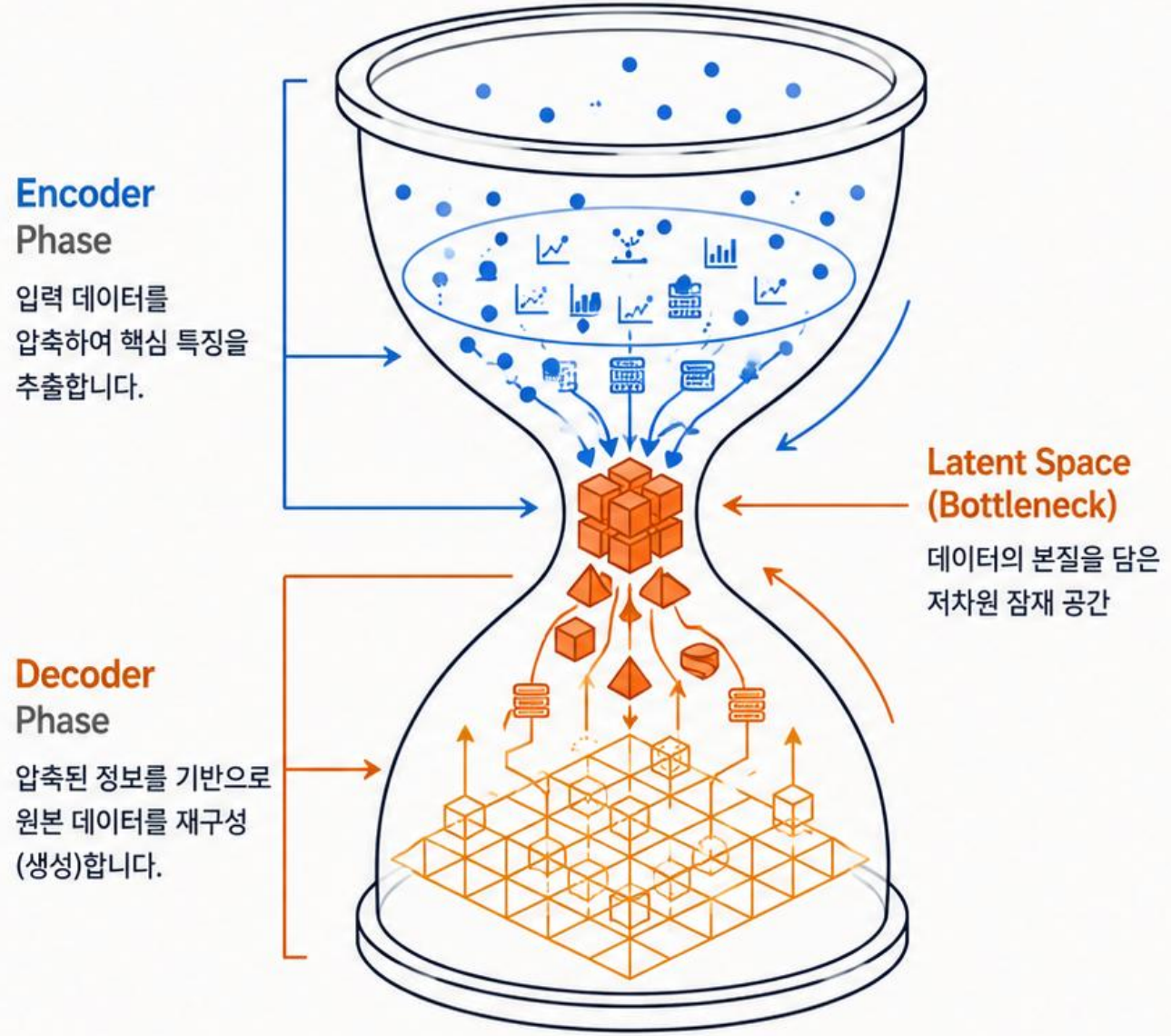
- [Est Camp] AI 퀀트 캠프 1기
- 보건·바이오 AI 활용 전문가 양성과정 (1-3기)
- 공공데이터 청년 인턴 멘토
- 충청 ICT 취창업 역량강화 프로그램 (SQLD)

아카데미 & 부트캠프

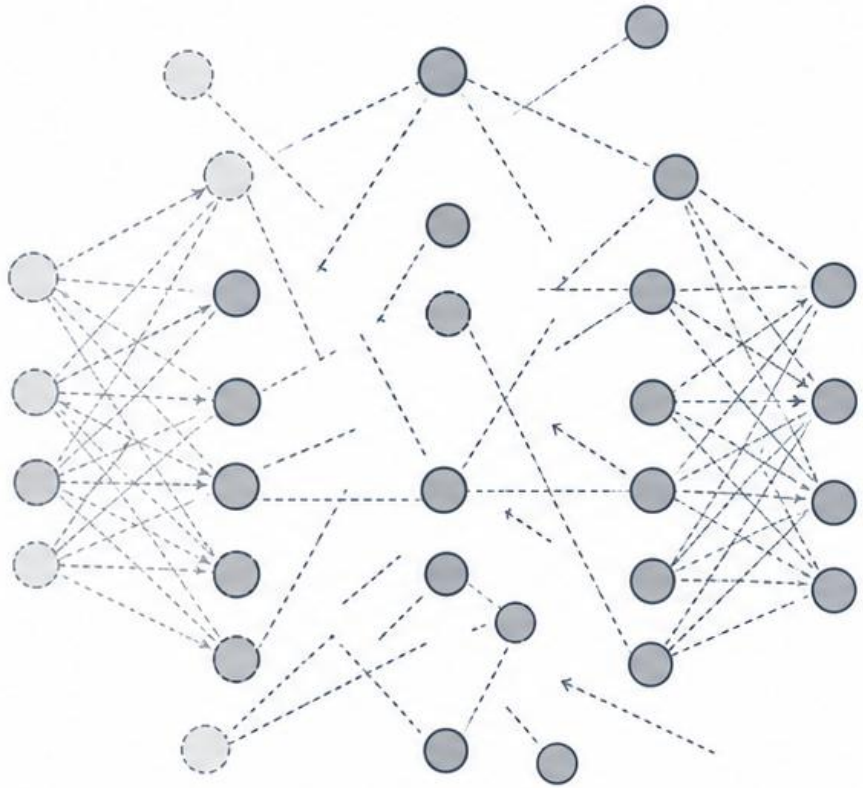
- 모두의연구소 Aiffel 퍼실리테이터 (1기)
- Upstage AI Boostcamp 멘토 (1-6기)
- 패스트캠퍼스/멀티캠퍼스 조교 및 멘토
- 공개SW개발자대회 멘토

Autoencoder (AE): 압축과 생성의 아키텍처

데이터의 본질을 추출하고 새로운 차원을 창조하는 딥러닝의 핵심 메커니즘



인체 해부학과 알고리즘의 완벽한 대칭성

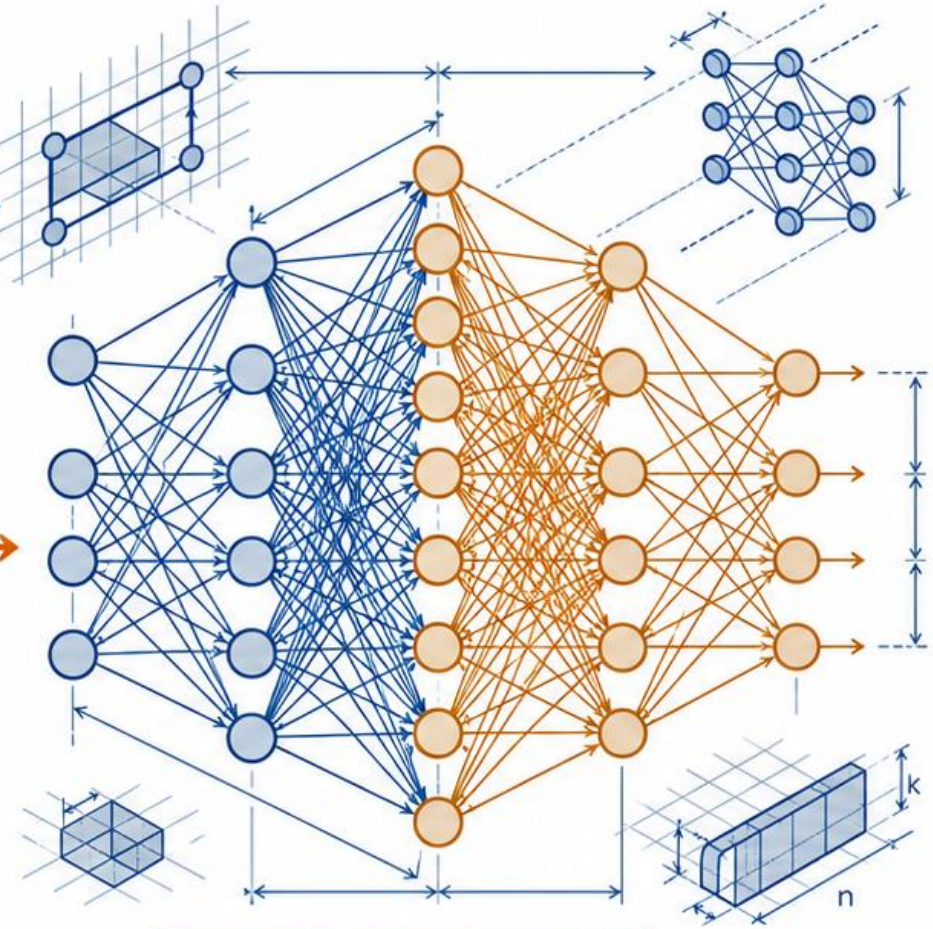


문제 (Before 2006)

AI 겨울 (AI Winter)

깊은 신경망(Deep Neural Networks)의 부적절한 가중치 초기값 문제로 인한 학습 실패. 심층 학습의 한계 적면.

2006:
층별 사전훈련
(제프리 힌튼)

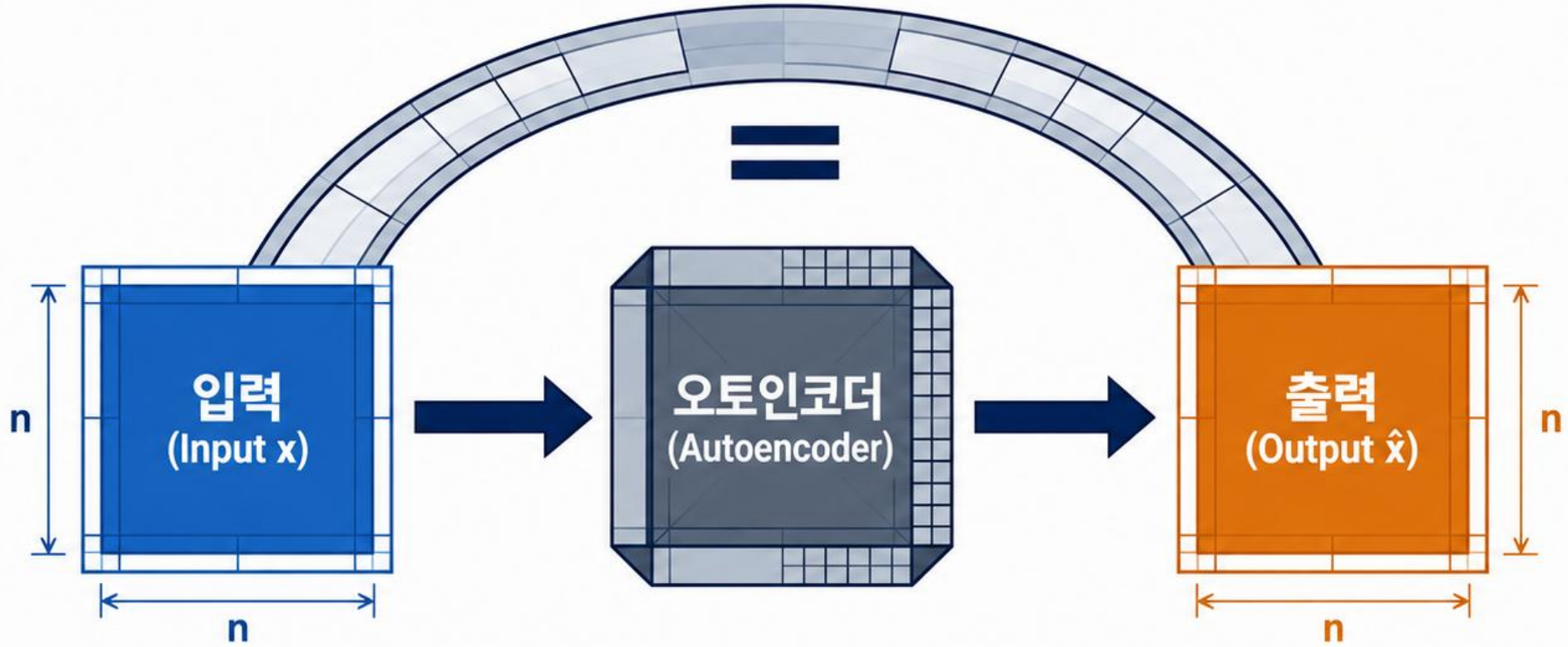


해결 및 돌파구 (After 2006)

딥러닝의 르네상스

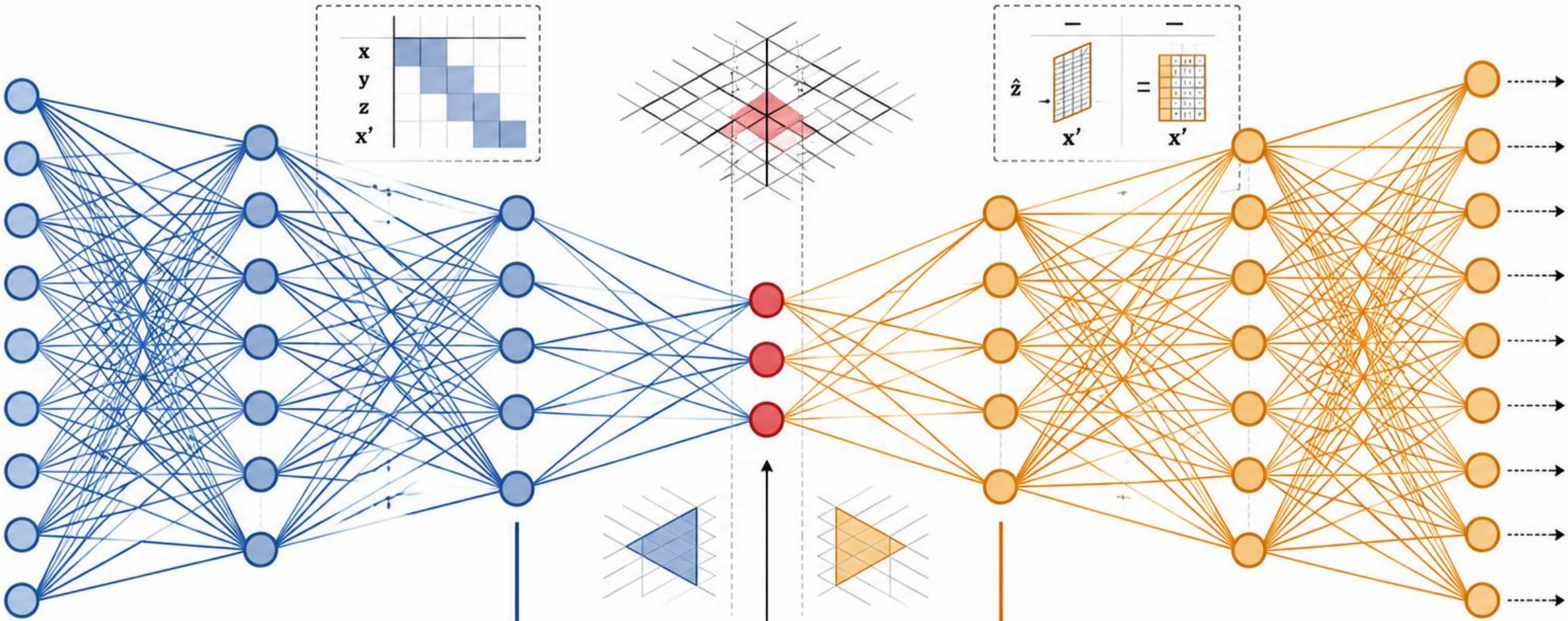
오토인코더를 활용하여 다층 신경망의 '완벽한 초기 가중치'를 찾아내어 AI 겨울을 종식.

오토인코더(Autoencoder) 한눈에 이해하기



비지도 학습 (Unsupervised)	왜 '오토(Auto)'인가?	3대 주요 목적
정답 레이블 없이 데이터 자체를 타겟으로 학습.	최종 출력이 최초 입력을 스스로(Self) 완벽하게 재현하도록 설계됨.	• 데이터 압축 (Compression) • 노이즈 제거 (Denoising) • 이상 탐지 (Anomaly Detection)

인코더–잠재 공간–디코더: 딥러닝의 압축과 복원



인코더 (Encoder)

입력 데이터 x를 압축하여
핵심 특징(Feature)만 추출.

$$z = f_\theta(\mathbf{x})$$

잠재 공간 (Latent Space)

압축된 정보의 병목. 불필요한 정보는
버려지고 본질만 남는 연속적 표현 공간.

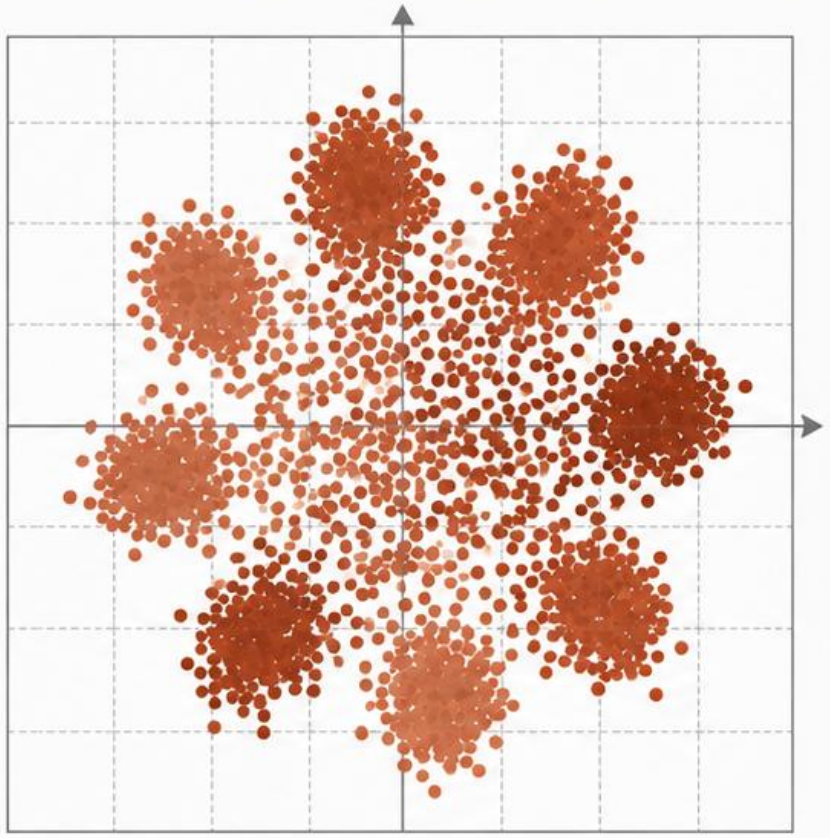
$$\dim(\mathbf{z}) < \dim(\mathbf{x})$$

디코더 (Decoder)

잠재 변수 z를 바탕으로
원본 데이터 x'를 복원.

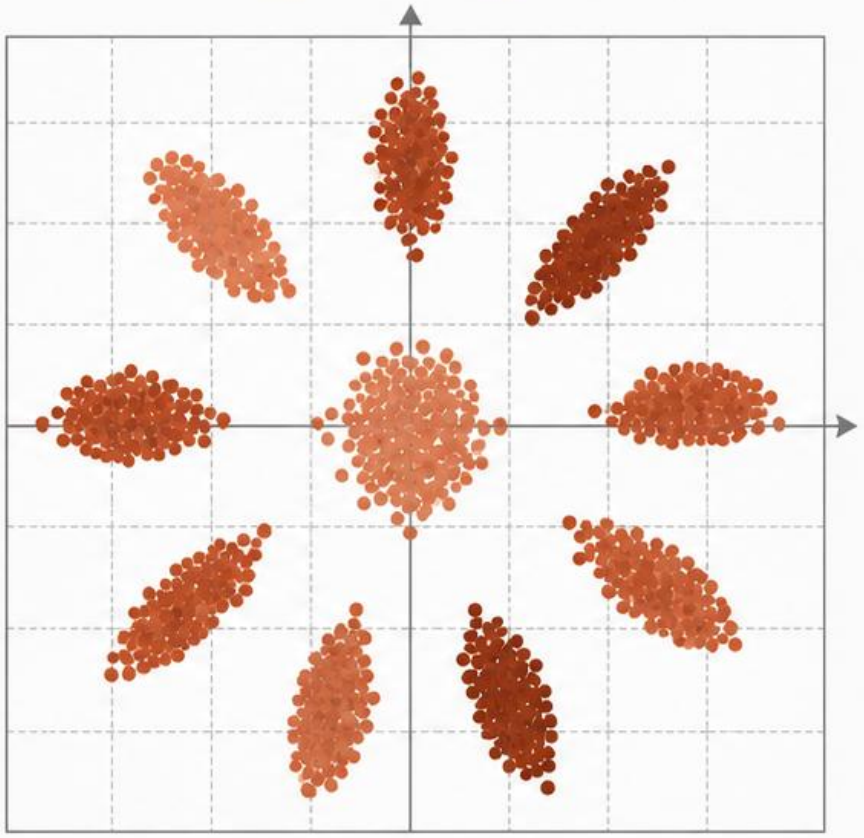
$$\hat{\mathbf{x}} = g_\phi(\mathbf{z})$$

정보 병목의 마법: MNIST 잠재 공간



일반 오토인코더 (Trained AE)

숫자별로 군집을 이루나, 경계면에서 서로 섞이는 한계 발생.



분류기 기반 오토인코더 (Classifier-based AE)

각 숫자의 고유한 특징을 명확하게 구별하여 깔끔하게 분리된 잠재 공간 형성.

**병목 현상(Bottleneck)은 네트워크가 픽셀을 단순 암기하지 못하게 막고,
데이터의 '본질적 구조'를 학습하도록 강제한다.**

정보 병목의 마법: MNIST 잠재 공간

데이터 유형 (Data Type)?

연속형 데이터
(이미지, 신호 등)

→ **평균 제곱 오차 (MSE)**

$$\mathcal{L}_{\text{MSE}} = \frac{1}{n}\sum(x_i - \hat{x}_i)^2$$

이진 데이터
(0/1 정규화)

→ **이진 교차 엔트로피 (BCE)**

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{n}\sum[x_i \log(\hat{x}_i) + (1 - x_i)\log(1 - \hat{x}_i)]$$

이상치가 많은 데이터

→ **평균 절대 오차 (MAE)**

$$\mathcal{L}_{\text{MAE}} = \frac{1}{n}\sum|x_i - \hat{x}_i|$$

학습 엔진 (Optimization Engine)

- **목적:** 재구성 오차(Reconstruction Error) 최소화
- **방법:** 역전파(Backpropagation)와 경사 하강법(Gradient Descent) 적용

NotebookLM

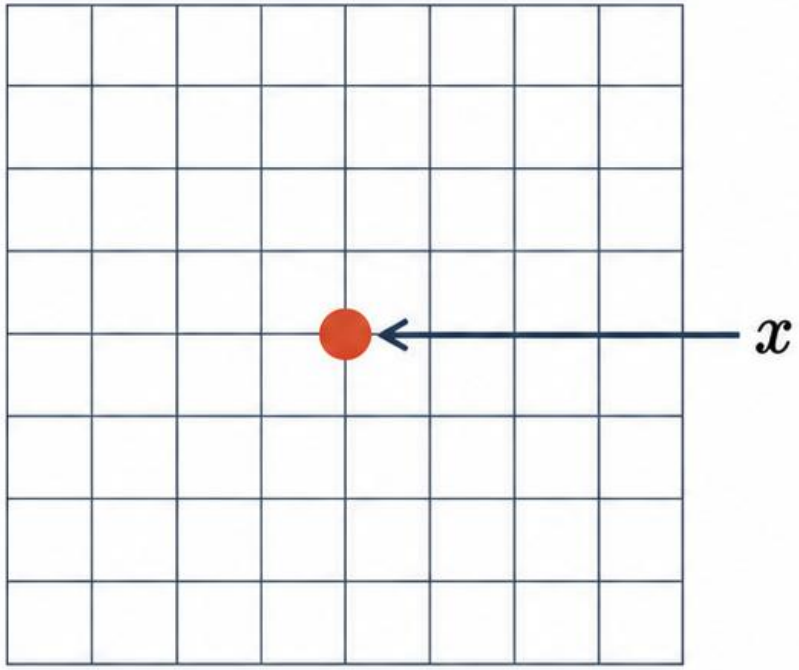
오토인코더 패밀리 (The Autoencoder Variants)

	특징 (Features)	제약 조건 / 손실 함수 (Constraints)	주요 목적 (Core Purpose)
 **언더컴플리트 (Undercomplete)**	$\dim(z) < \dim(x)$ 구조 유지	차원 축소 자체를 제약으로 활용	가장 기본적인 핵심 특징 압축 및 학습
 **희소 (Sparse AE)**	활성화 뉴런을 최소화하도록 강제	$\mathcal{L}_{\text{recon}} + \lambda \sum \lvert \rho - \hat{\rho}_j \rvert$	해석 가능한 특징 학습, 과적합 방지
 **디노이징 (DAE)**	노이즈(ϵ)가 추가된 입력 $\tilde{x}$ 복원	$\mathcal{L}\left(x, g_\phi\left(f_\theta(\tilde{x})\right)\right)$	데이터의 본질적 구조 파악, 강건성 향상
 **수축 (CAE)**	입력의 미세한 변화에 둔감한 잠재 표현	$\mathcal{L}_{\text{recon}} + \lambda \left\lVert \frac{\partial f_\theta(x)}{\partial x} \right\rVert_F^2$	견고하고 안정적인 매니폴드(Manifold) 학습

핵심 요약: 오토인코더는 다양한 제약과 목적을 통해 데이터를 효과적으로 표현하고, 그 본질을 학습하도록 설계됩니다.

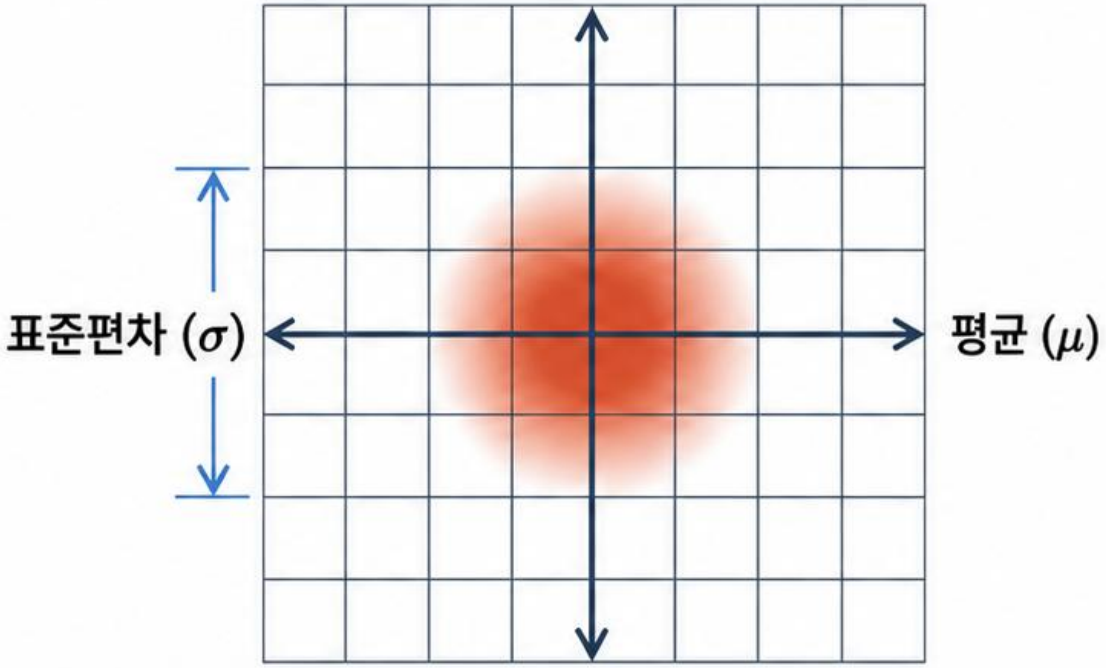
패러다임의 전환: 변분 오토인코더 (VAE)

일반 오토인코더 (Standard AE)



데이터를 특정 '점(Point)'으로 매핑.
결정론적 압축.

변분 오토인코더 (VAE)



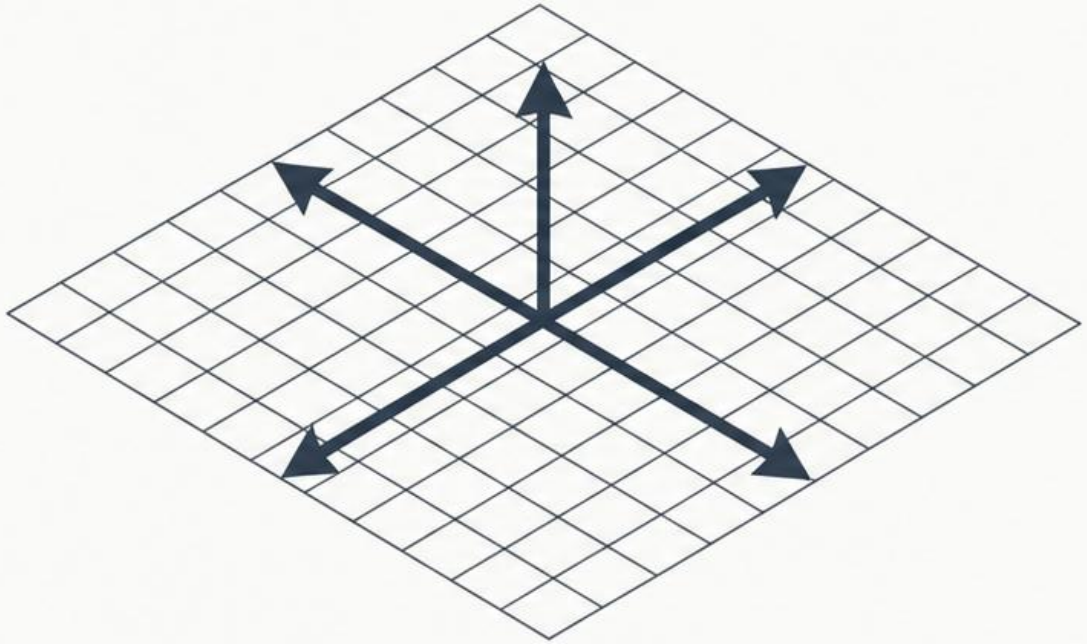
데이터를 '확률 분포(Distribution)'로 매핑.
새로운 데이터 생성이 가능한 생성형 AI의 시작.

ELBO 손실 함수: $\mathcal{L}_{VAE}$ = (재구성 항) + (KL 발산 페널티)

$$-\mathbb{E}_{q(z|x)}[\log p(x|z)] + D_{KL}(q(z|x) \parallel p(z))$$

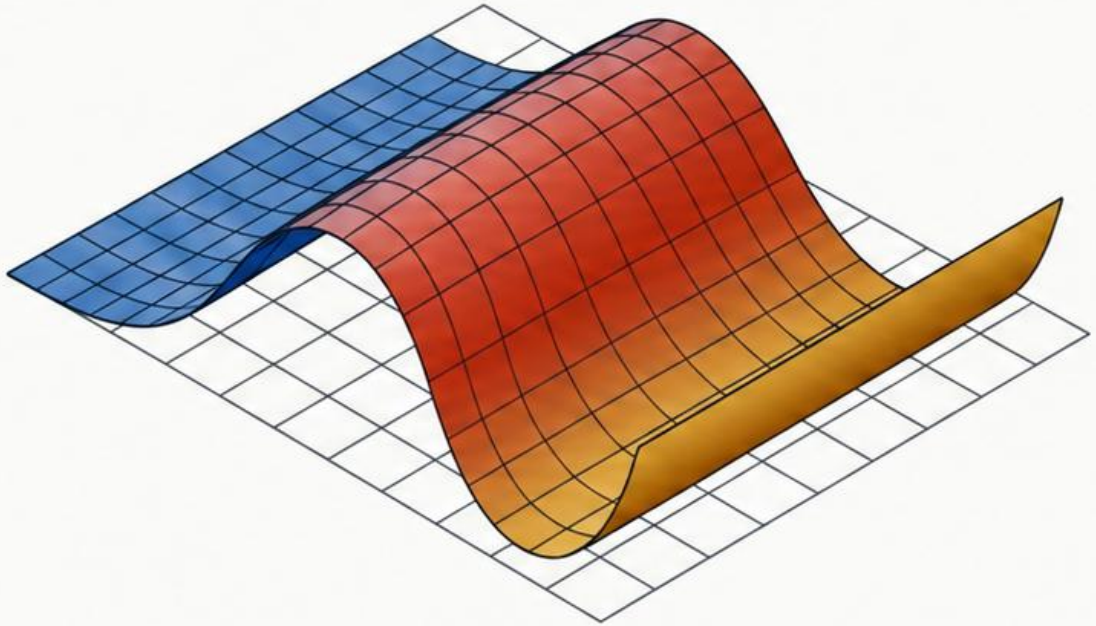
응용 I: 차원 축소 (PCA vs Autoencoder)

PCA (주성분 분석)



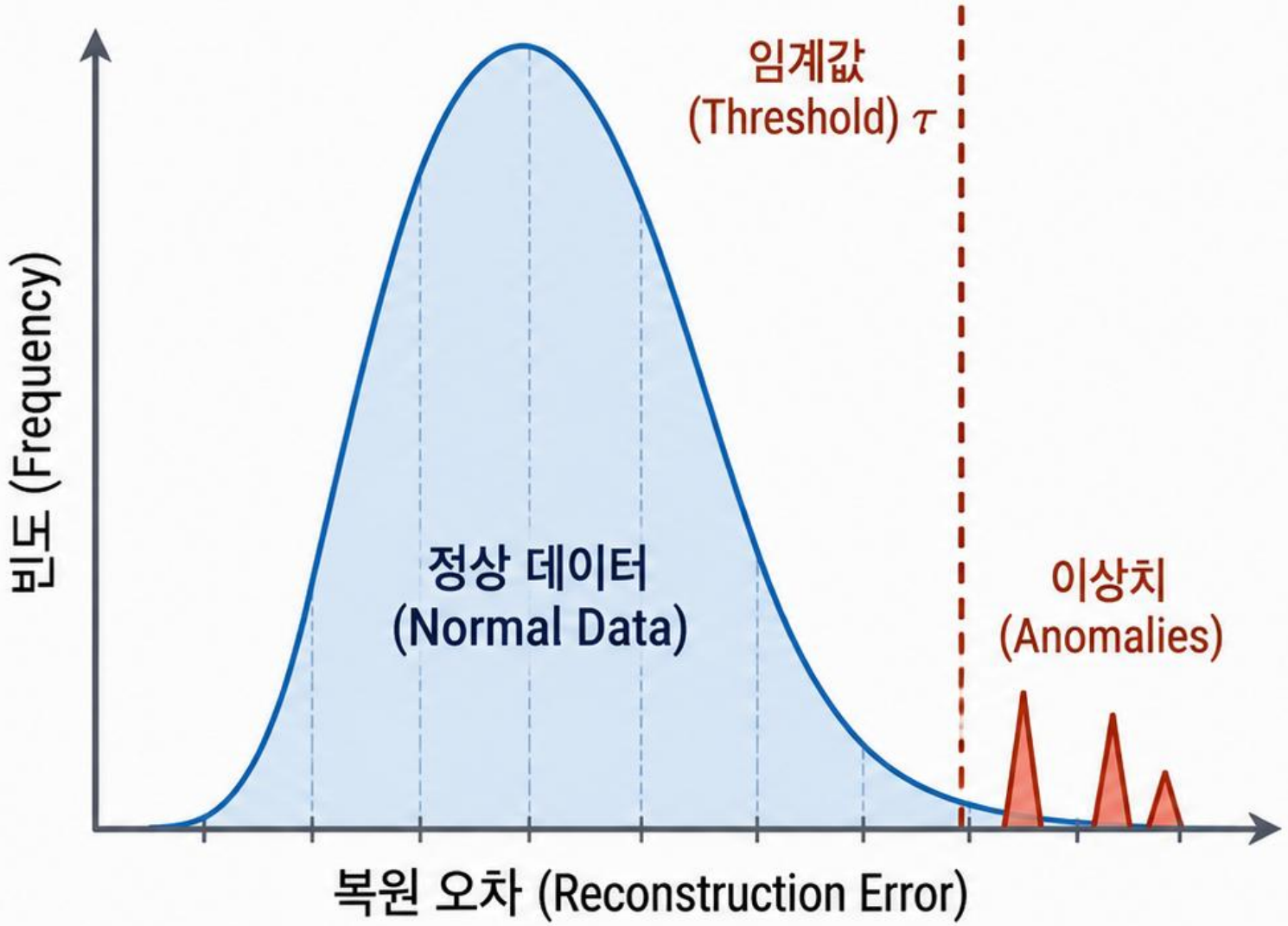
선형 변환 (Linear). 직교 기저를
기반으로 하여 계산이 효율적이나,
데이터의 복잡한 비선형 구조를
온전히 담아내지 못함.

오토인코더 (Autoencoder)



비선형 변환 (Non-linear).
유연한 신경망 구조를 통해
데이터의 복잡한 굽음꼰(Manifold)과
내재된 패턴을 완벽히 학습.

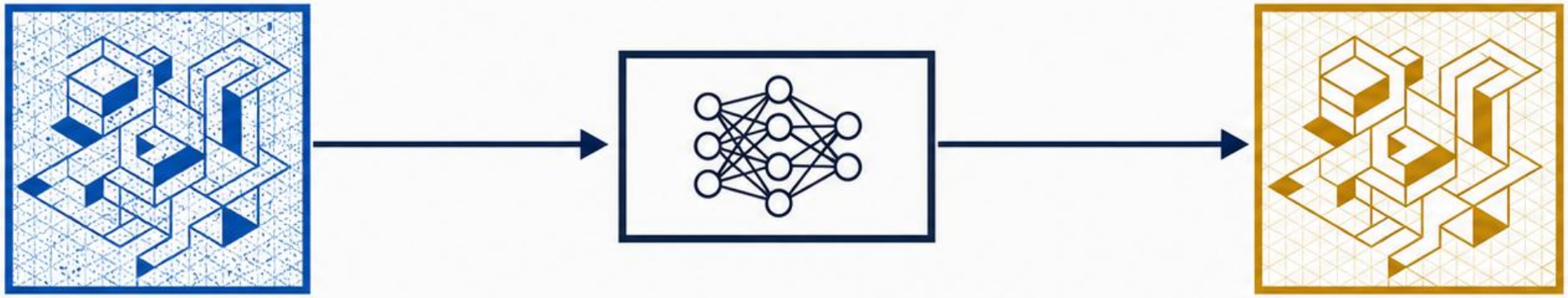
응용 II: 이상 탐지 (Anomaly Detection)



논리: 본 적 없는 데이터는 복원할 수 없다.

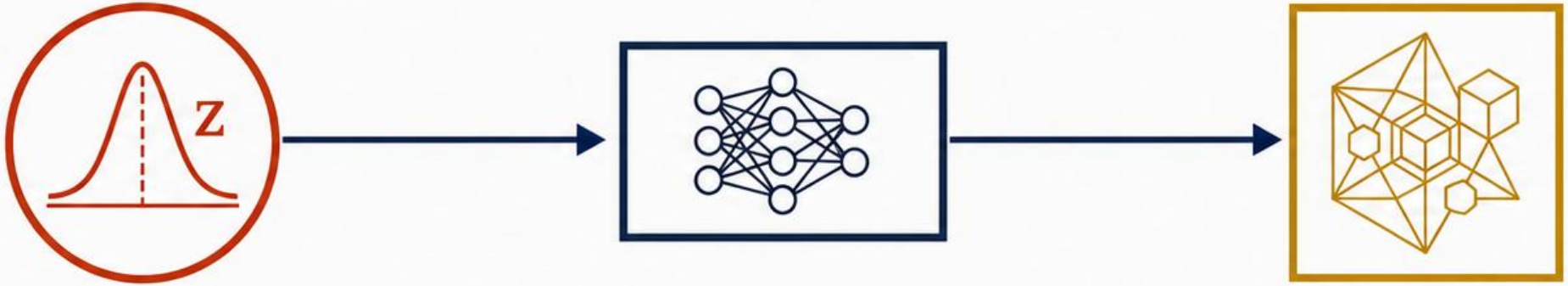
- **정상 데이터:** 모델이 패턴을 이미 학습하여 복원 오차가 매우 낮음.
- **이상치:** $A(\mathbf{x}) = \|\mathbf{x} - \hat{\mathbf{x}}\|^2 > \tau$ (오차가 임계값 초과).
- **장점:** 레이블이 없는 '정상 데이터'만으로도 비지도 학습 가능.
- **활용:** 뇌 MRI 이상 분할, 금융 사기 탐지, 보안.

응용 III: 노이즈 제거 (Denoising Autoencoder)



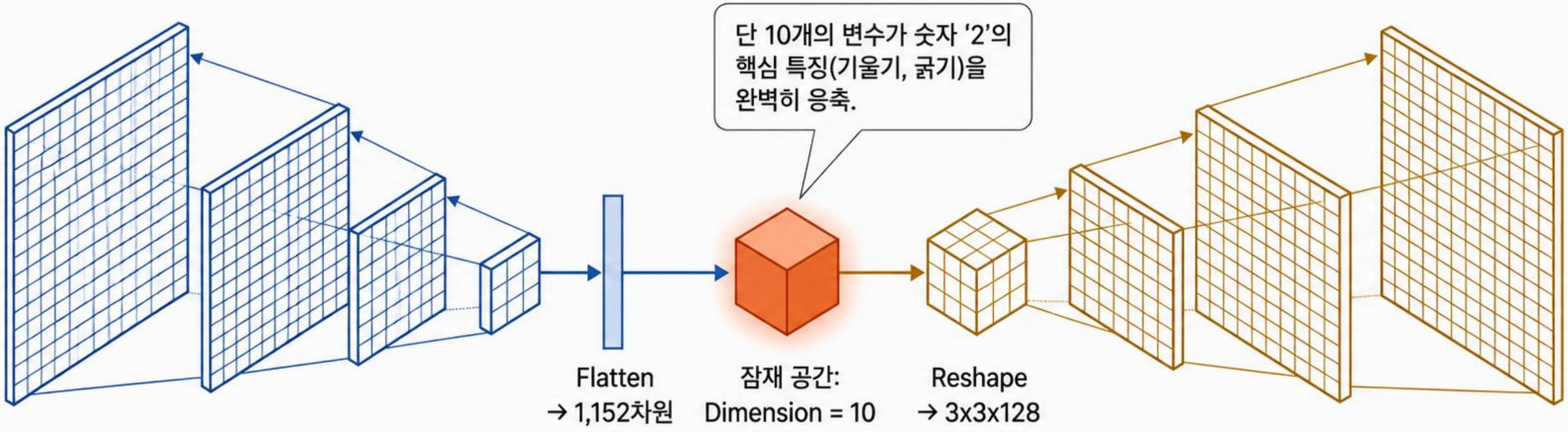
고의로 노이즈가 추가된 입력($\tilde{x}$)을 원본으로 복원하도록 학습.
의료 영상(CT, MRI) 개선 및 오래된 사진 복원에 탁월.

응용 IV: 생성 모델 (Generative Model)

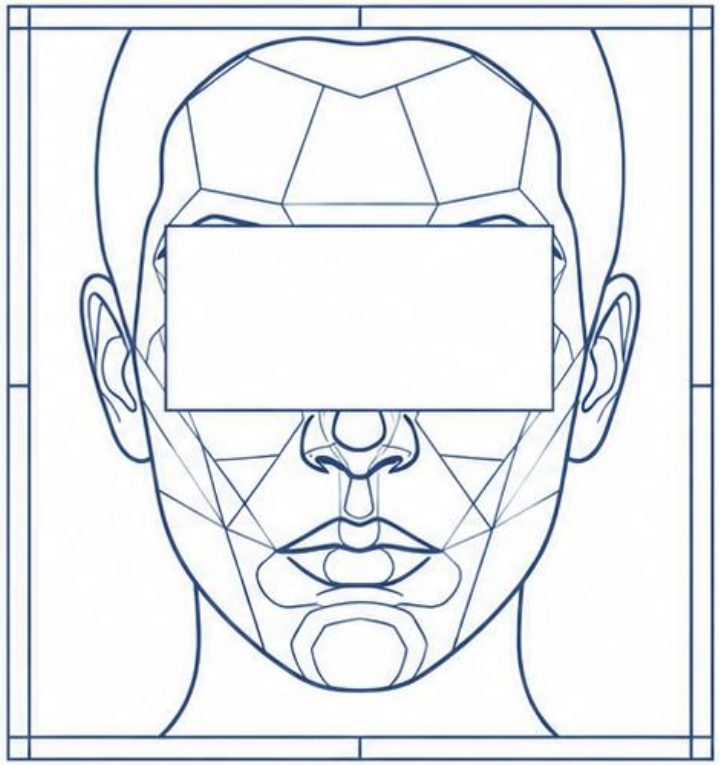


연속적인 잠재 확률 분포에서 무작위 샘플(z)을 추출하여,
학습하지 않은 전혀 새로운 데이터(얼굴, 텍스트, 분자 구조)를 창조.

아키텍처의 실제: MNIST 숫자 탐지

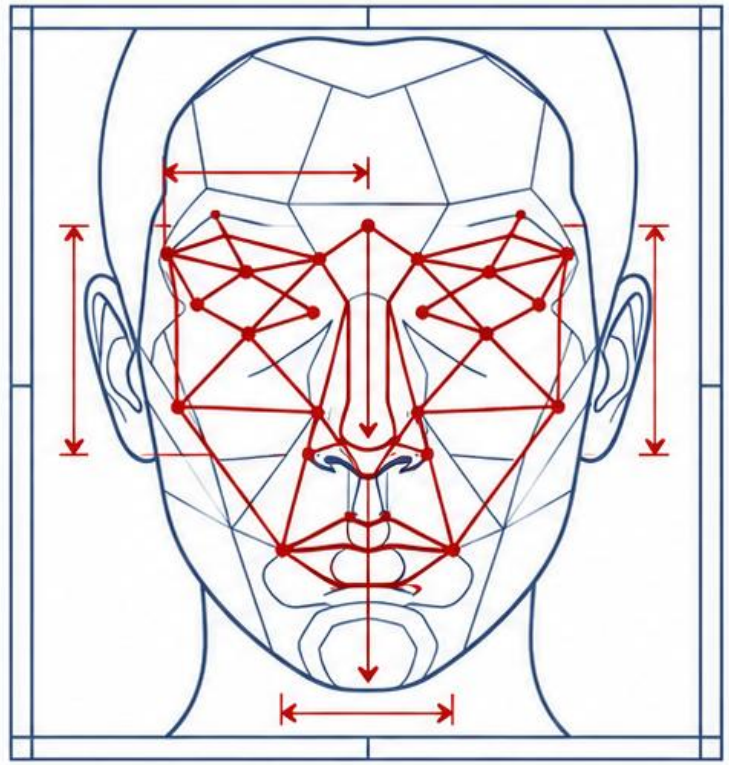


이미지 복원 (Contextual In-painting)



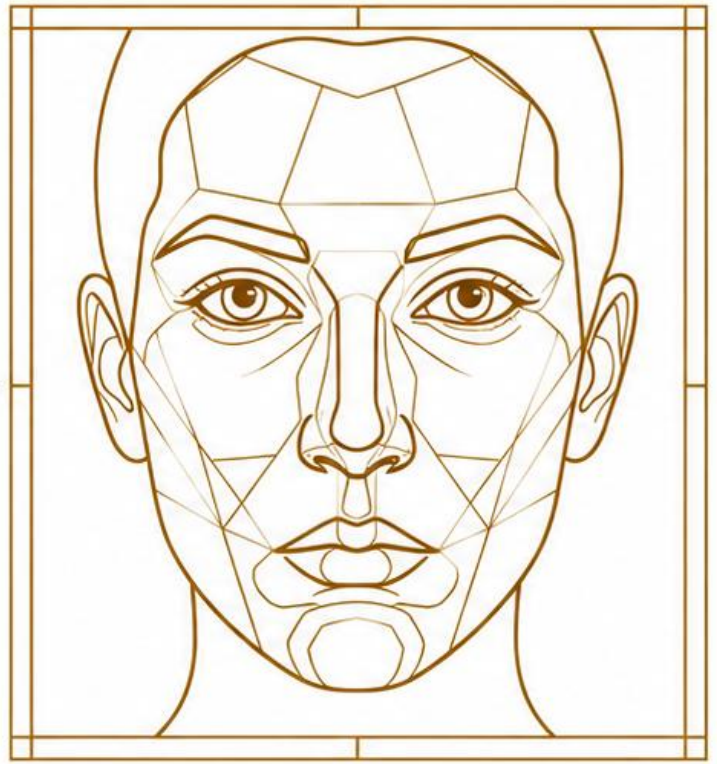
1. 특징 파악 (Encoding)

손상되지 않은 영역을 분석하여
열굴 각도와 주변 맥락 파악.



2. 잠재적 지식 (Latent Features)

수만 장의 대이터를 통해 습득한 눈과 코의
위치, 형태에 대한 사전 지식(Prior) 가동.



3. 문맥 기반 생성 (Decoding)

파악된 맥락과 학습된 분포를 결합하여 빈
공간을 자연스럽고 정교한 픽셀로 완벽히 복원.

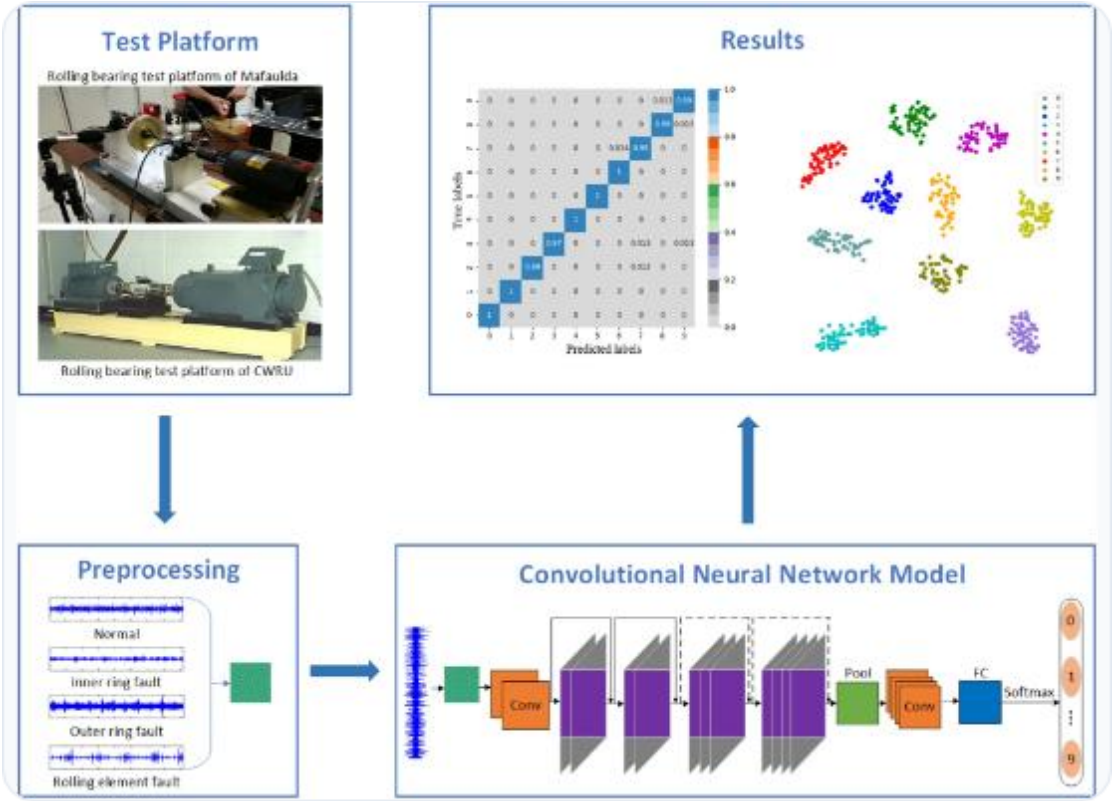
왜 Seq2Seq가 필요한가?

고정된 출력을 넘어서

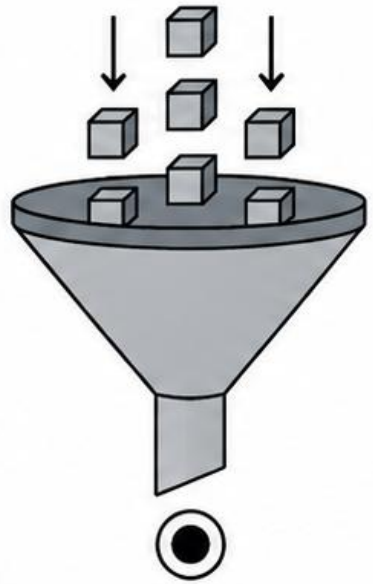
일반적인 분류 문제는 감정(긍정/부정)과 같이 출력이 짧고 고정적입니다. 그러나 번역, 요약, 질의응답은 정답이 **연속된 토큰들의 시퀀스**입니다.

- **번역:** 문장 → 문장
- **요약:** 긴 글 → 짧은 요약
- **대화:** 사용자 발화 → 응답

단순한 클래스 분류가 아닌, 한 토큰씩 순차적으로 생성하는 구조가 필요합니다.

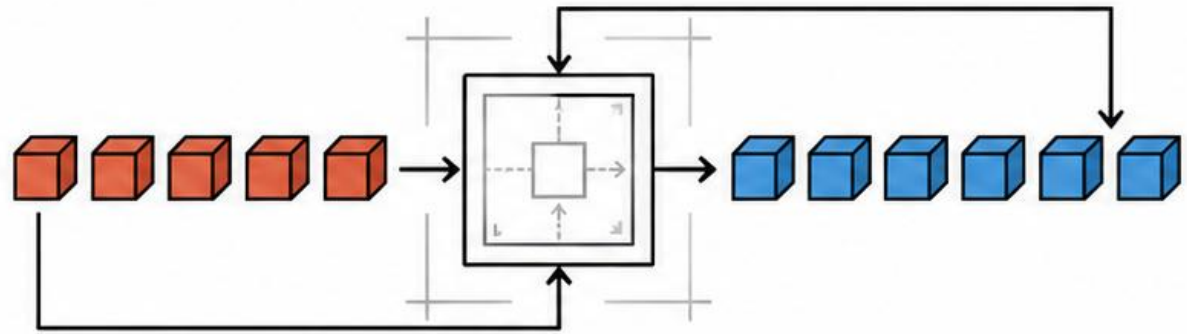


단일 정답을 찾는 분류를 넘어, 연속된 흐름을 생성하는 세계로



기존 분류 모델

- 다수의 입력 → 1개의 고정 라벨 (예: 감정 분석)
- 하나의 정답 클래스를 고르는 정적인 구조

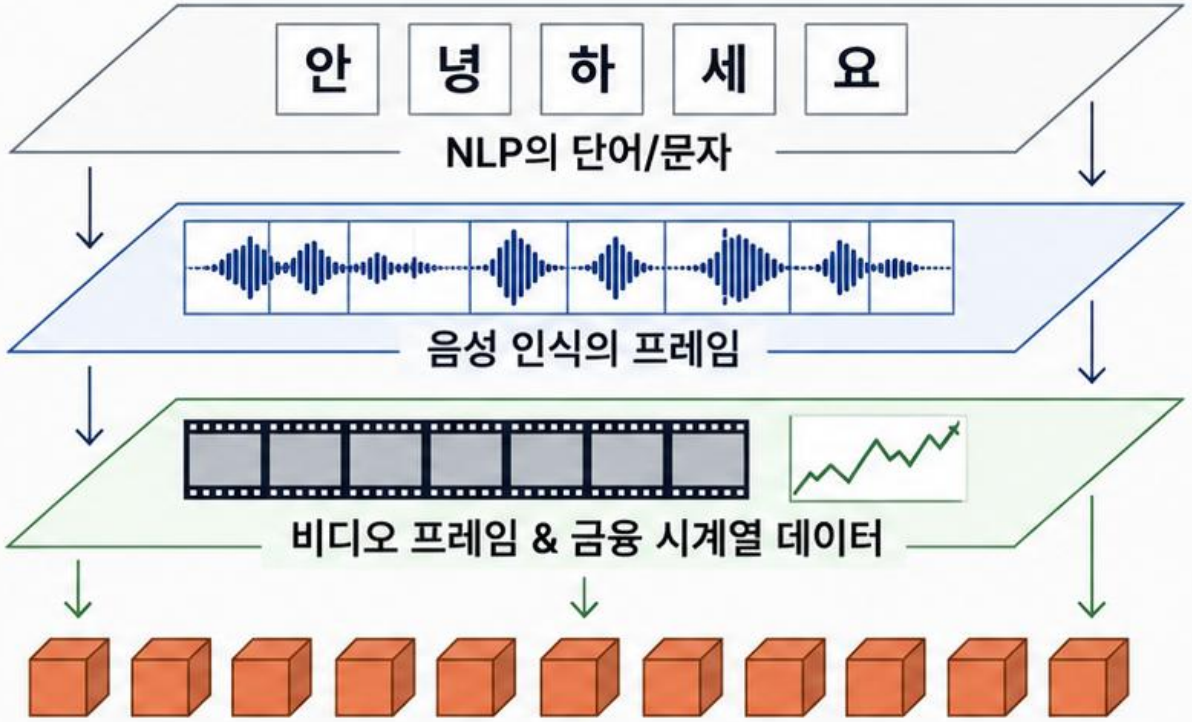


Seq2Seq 모델

- 가변 길이의 시퀀스 → 가변 길이의 시퀀스 (예: 변역, 요약)
- 한 번에 정답을 찾는 것이 아니라, 한 토큰씩 순차적으로 생성하는 완전히 다른 메커니즘이 필요합니다.

시퀀스(Sequence)는 텍스트에 국한되지 않는 보편적인 데이터의 형태입니다

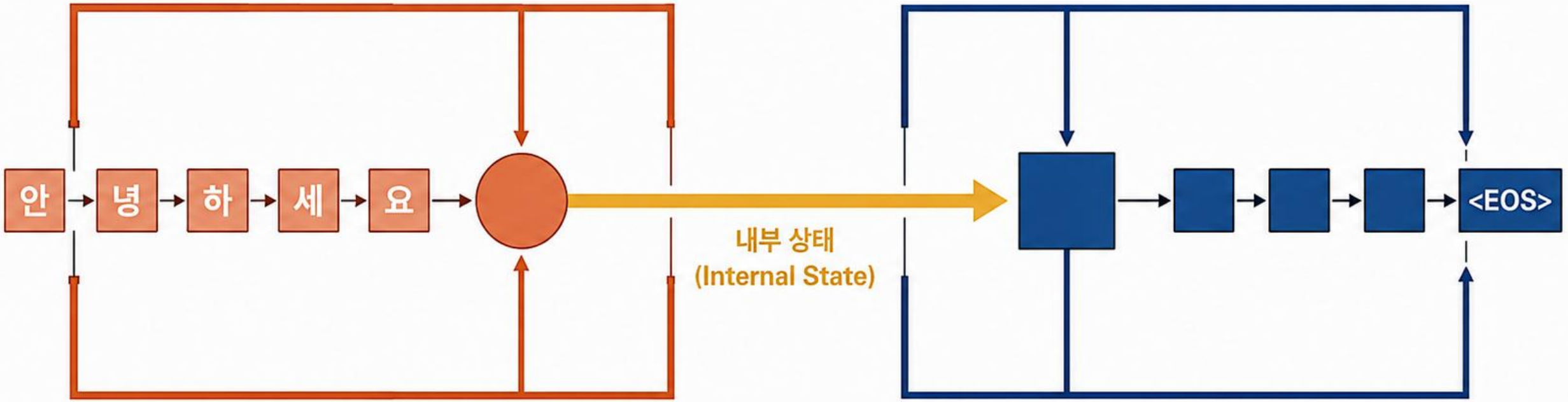
시퀀스란 순서가 핵심적인 의미를 갖는 모든 데이터의 나열을 뜻합니다.



NLP의 단어/문자뿐만 아니라, 음성 인식의 프레임, 비디오 프레임, 금융 시계열 데이터까지 동일한 토큰의 흐름으로 치환할 수 있습니다. 이는 **Seq2Seq**가 도메인을 넘나드는 **범용적 아키텍처**임을 의미합니다.

이해와 생성의 완벽한 분업: 인코더와 디코더의 해부학

“인코더는 이해하고, 디코더는 말한다.”



이해의 영역 (Encoder)

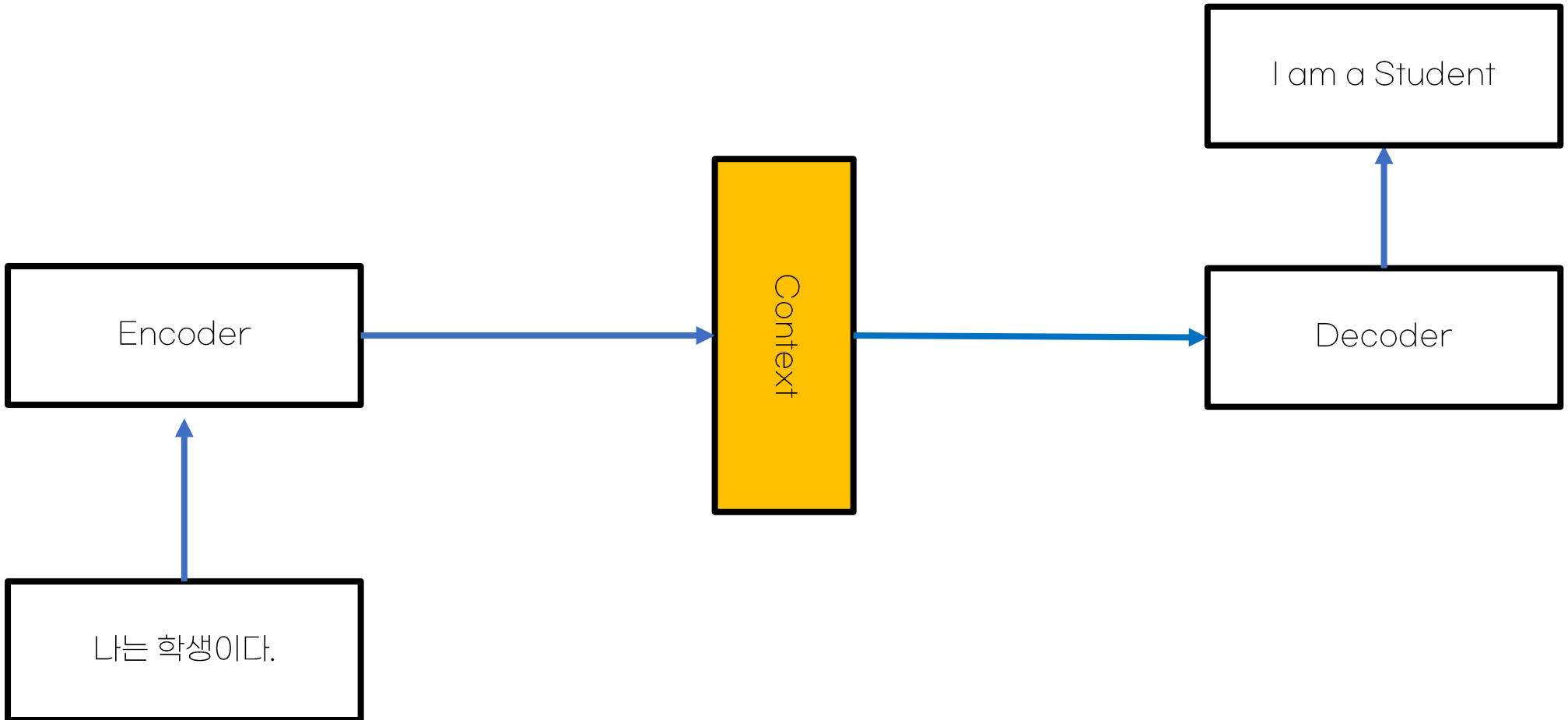
입력 시퀀스를 읽고, 그 의미를 네트워크 내부의 압축된 표현(내부 상태)으로 변환합니다.

생성의 영역 (Decoder)

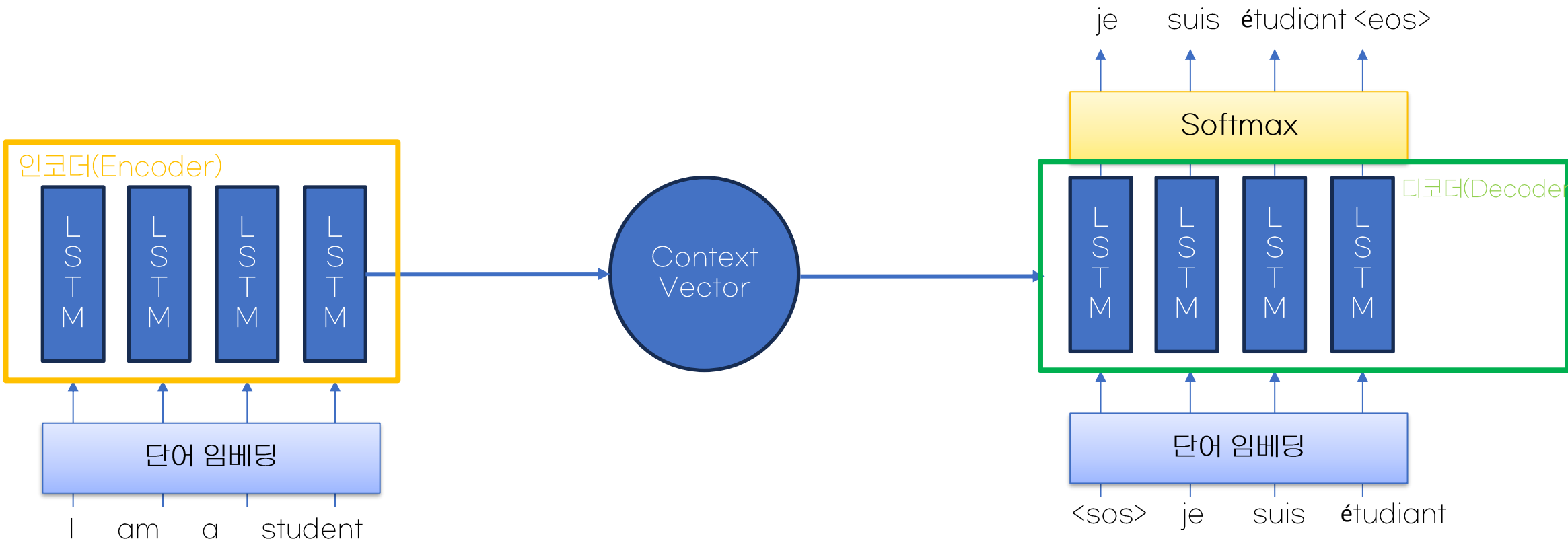
인코더가 넘겨준 표현을 바탕으로 첫 단어부터 종료 토큰(<EOS>)이 나올 때까지 출력 시퀀스를 순차적으로 생성합니다.

Seq2Seq

- Seq2Seq(sequence-to-sequence)모델은 한 시퀀스를 다른 시퀀스로 변환하는 작업을 수행하는 딥러닝 모델로, 주로 자연어 처리(NLP) 분야에서 활용됨.
- 인코더와 디코더라는 모듈을 가지고 있음. Seq2seq를 Encoder-Decoder모델이라고도 함.

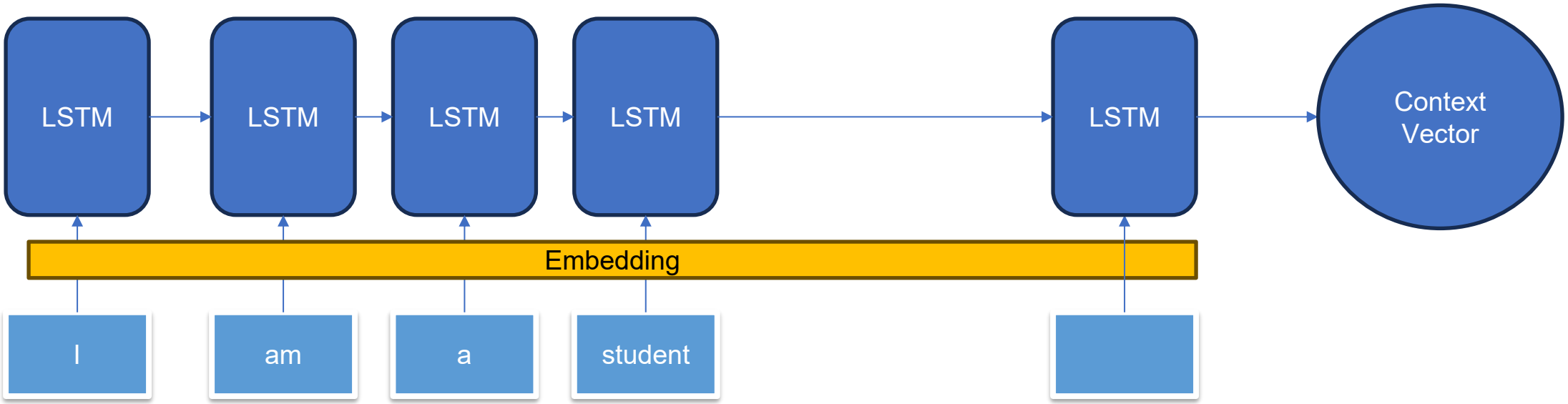


Seq2Seq



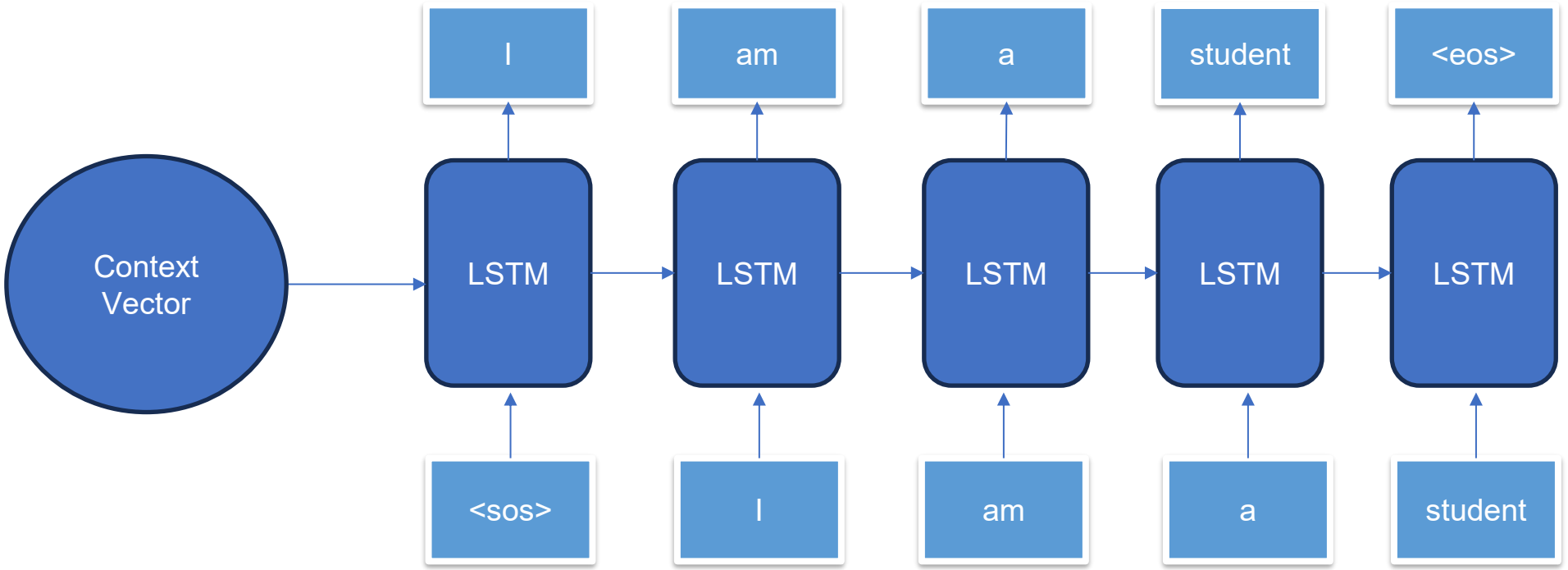
- 인코더나 디코더는 RNN으로 구성되어 있음.
- 입력 문장을 받는 RNN 셀을 인코더라고 하고, 출력 문장을 출력하는 RNN 셀을 디코더라고 함.

Seq2Seq_Encoder



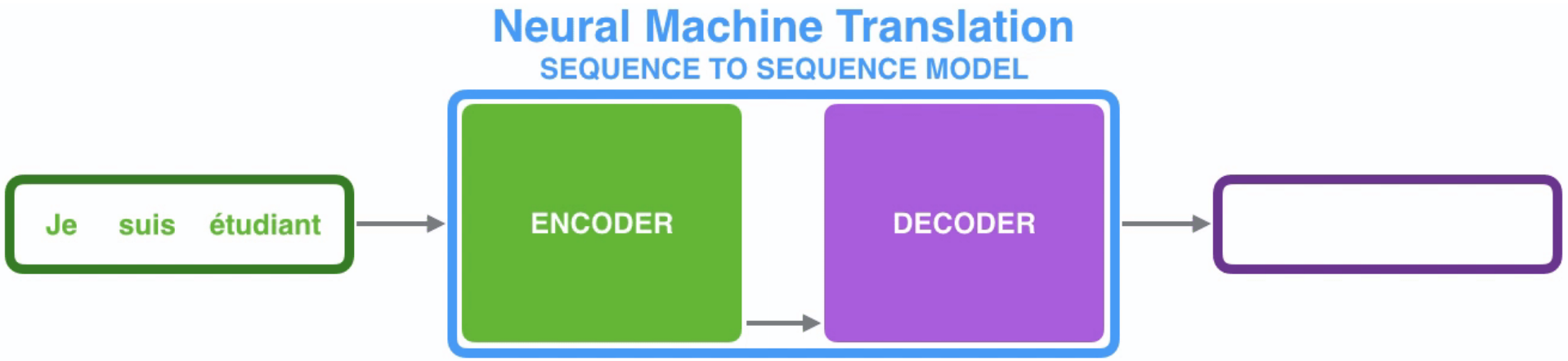
- 인코더가 인코딩 한다는 것은 임의의 길이의 문장을 고정 길이 벡터로 변환하는 작업
- 입력 문장을 단어 토큰화를 통해서 단어 단위로 쪼개지고 단어 토큰 각각 RNN 셀의 각 시점의 입력이 됨.
  - LSTM 기준으로 hidden state와 cell state, GRU의 경우에는 hidden state만 추출
- 인코더 RNN 셀은 모든 단어를 입력 받은 뒤, 인코더 RNN 셀의 마지막 시점의 은닉 상태를 디코더 RNN셀로 넘겨줌
  -> 이를 Context Vector라고 함.

Seq2Seq_Decoder



- 디코더는 인코더의 출력인 고정 길의의 벡터를 기반으로 원하는 출력 시퀀스를 생성하는 역할을 수행.
- 디코더의 예측은 일반적으로 소프트맥스 활성화 함수를 통해 확률 분포로 변환되며, 가장 확률이 높은 단어가 선택
- 디코더는 인코더에서 추출한 초기 상태를 받아 LSTM으로 순차적으로 결과를 출력해주는 파트.

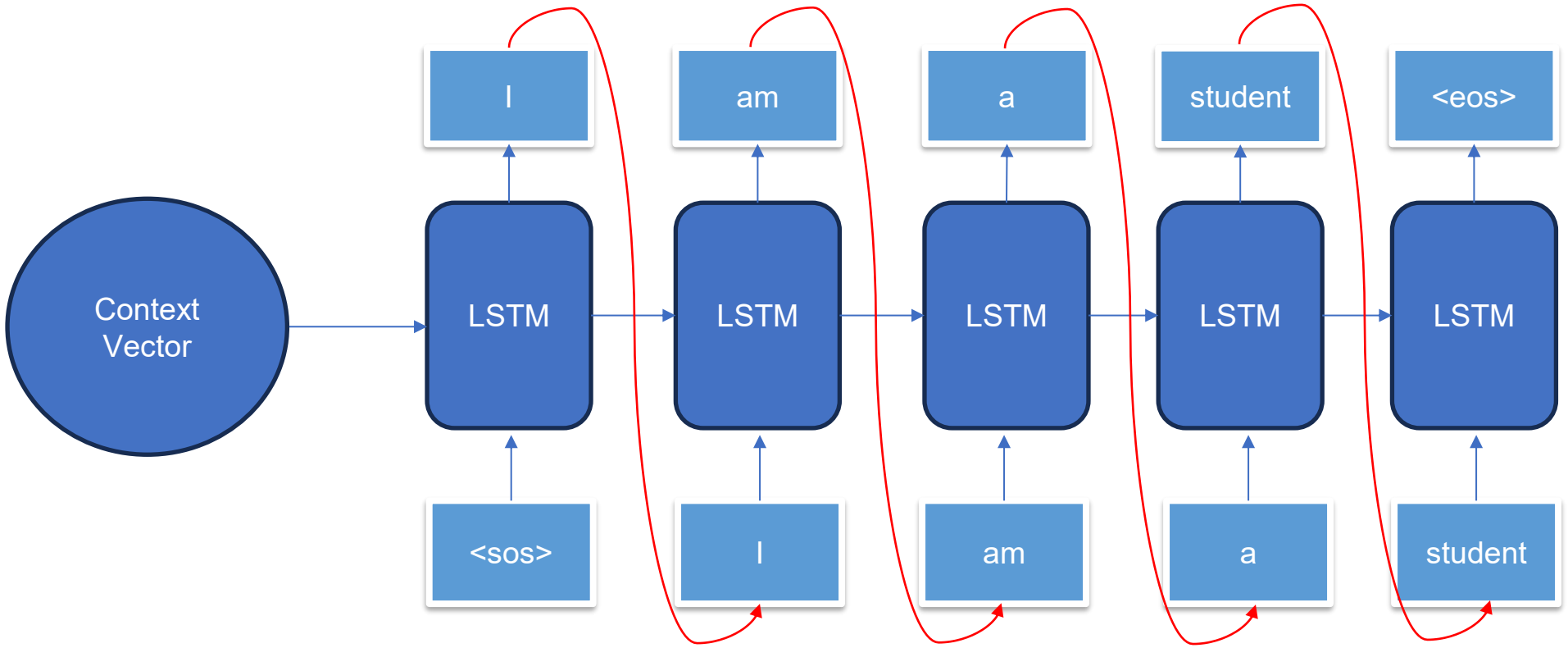
Seq2Seq - Machine Translation



- Seq2seq를 기계번역에 적용할 시 위와 같이 프랑스어에 해당하는 입력 시퀀스들이 순차적으로 Encoder로 입력되어 마지막 스퀀스까지 weight을 업데이트하고, 그렇게 업데이트 된 마지막 입력 시퀀스
- 즉, Context vector를 Decoer의 입력으로 넘겨주어 영어로 출력하게 됨.

출처 https://velog.io/@jaehyeong/Basic-NLP1-Sequence-to-Sequence-with-Attention

Seq2Seq_Teacher Forcing

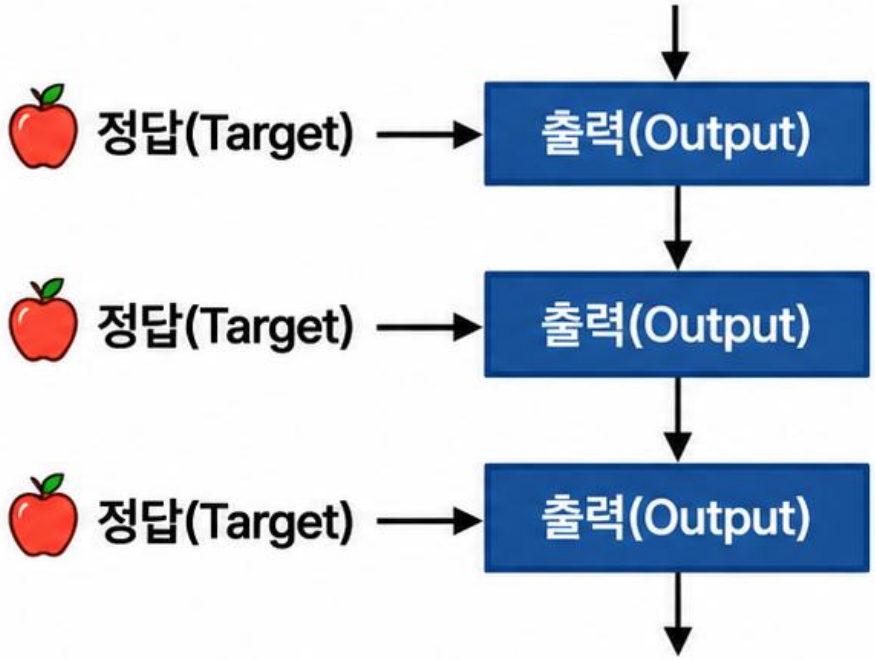


- 훈련 과정에서는 이전 시점의 디코더 셀의 출력을 현재 시점의 디코더 셀의 입력으로 넣어주지 않고, 이전 시점의 실제값을 현재 시점의 디코더 셀의 입력값으로 하는 방법을 사용
  - 이전 시점의 디코더 셀의 예측이 틀렸는데, 이를 현재 시점의 디코더 셀의 입력으로 사용하면 현재 시점의 디코더 셀의 예측도 잘못될 가능성이 높고, 이는 연쇄 작용으로 디코더 전체의 예측을 어렵기 하기 때문

- Teacher Forcing : RNN의 모든 시점에 대해서 이전 시점의 예측값 대신 실제값을 입력으로 주는 방법

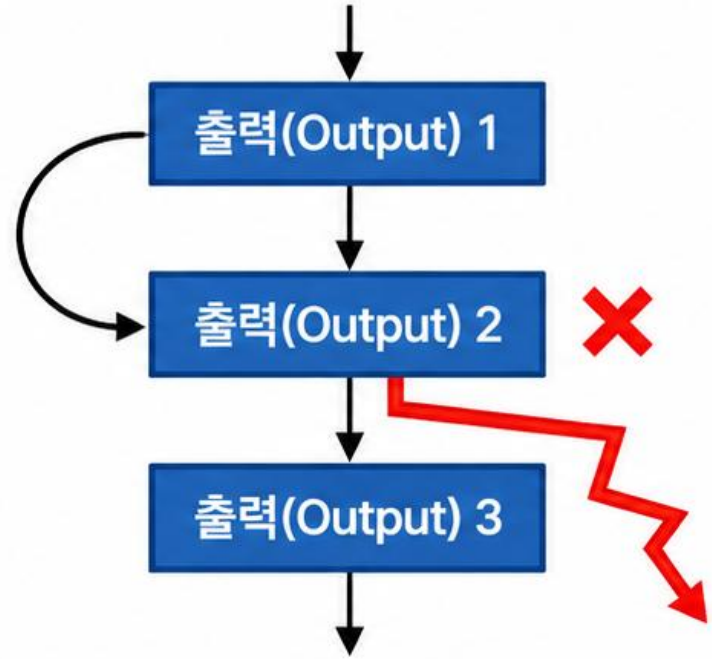
학습과 추론의 결정적 차이: 교사 강요(Teacher Forcing)와 노출 편향

학습 시 (Training)



학습 시 (Training): 모델이 직전 스텝에서 오답을 냈더라도, 다음 입력으로는 무조건 '실제 정답'을 넣어줍니다. 학습이 빠르고 매우 안정적입니다.

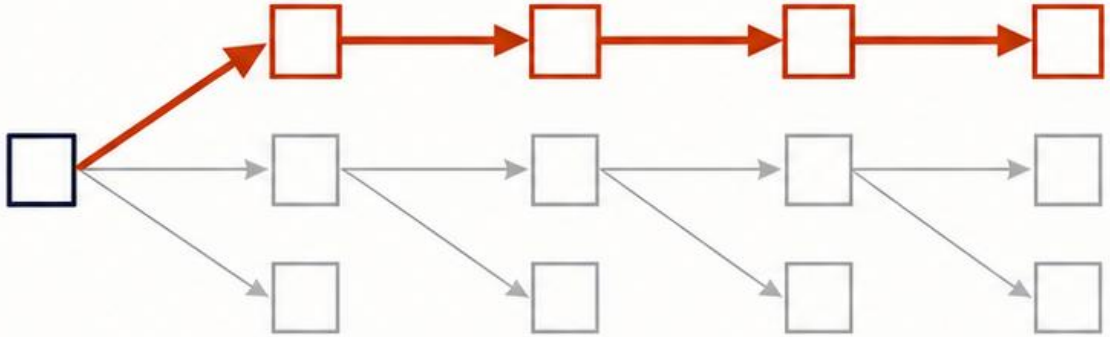
추론 시 (Inference)



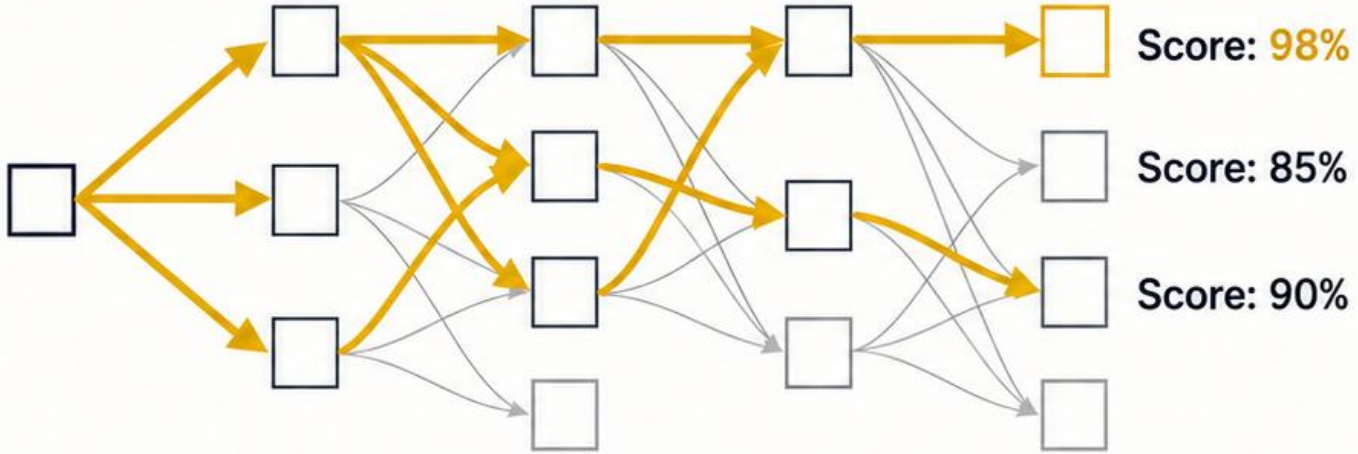
추론 시 (Inference): 정답지가 없으므로 '자신이 방금 생성한 출력'을 다음 입력으로 재사용해야 합니다.

노출 편향 (Exposure Bias) 학습 환경과 실제 생성 환경의 근본적 불일치. 추론 중 한 번 오답을 생성하면, 그 오답이 다음 입력으로 들어가 오차가 오차가 겹잡을 수 없이 누적되는 현상.

최적의 문장을 탐색하는 디코딩(Decoding) 전략



Greedy Decoding: 매 순간 확률이 가장 높은 단어 하나만 선택하며 직진합니다. 연산이 매우 빠르지만, 문장 전체의 관점에서 최적의 결과물을 놓칠 위험이 높습니다.



Beam Search: 가능성이 높은 후보 경로(Beam) 여러 개를 동시에 유지하며 끝까지 탐색합니다. 계산량은 크게 늘어나지만, 번역과 텍스트 생성의 최종 품질이 압도적으로 상승합니다.

입력과 과거의 출력을 바탕으로 다음을 예측하는 하나의 수학적 원리

다음 출력 단어(y_t)가 무엇일지 맞히는 확률 게임.

이전까지 생성한 모든 출력($y_{<t}$)을 참고하여,

$$P(y|x) = \prod P(y_t|y_{<t}, x)$$

입력(x)을 참고하고,

수학적으로 Seq2Seq는 철저한 **조건부 확률**의 모델링입니다.
이 단순하고도 우아한 원리가 오늘날 우리가 경험하는 **모든 생성형 AI 혁신의 근간**이 되었습니다.

완벽하지 않은 설계도: 현재의 한계와 극복 과제



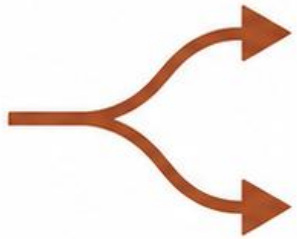
긴 문장의 정보 손실

초기 모델의 한계점. 현재는 Attention 메커니즘을 통해 대부분 극복되었습니다.



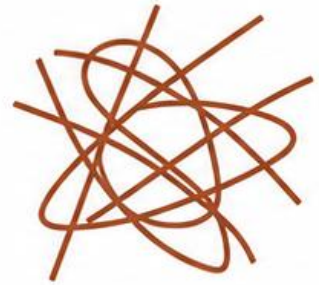
느린 추론(Inference) 속도

디코더는 필연적으로 한 토큰씩 순차적으로 생성해야 하므로, 병렬 생성이 불가능하고 속도에 근본적 한계가 존재합니다.



학습/추론 환경의 불일치

Teacher forcing으로 인한 노출 편향(Exposure bias) 문제로 오차가 누적될 위험이 있습니다.

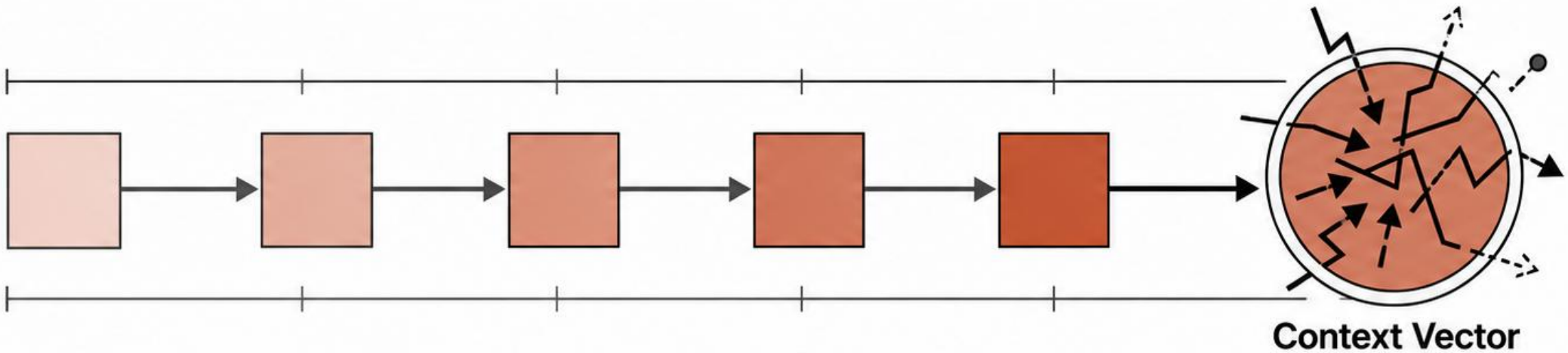


출력 품질 제어의 난해함

원치 않는 중복 생성(할루시네이션), 너무 길거나 짧은 응답 등 생성 과정을 완벽히 통제하기 어렵습니다.

초기 RNN 기반 구조가 직면했던 치명적인 정보 병목(Bottleneck)

초기 Seq2Seq 모델은 RNN/LSTM을 사용하여 모든 입력 데이터를 순차적으로 읽었습니다.

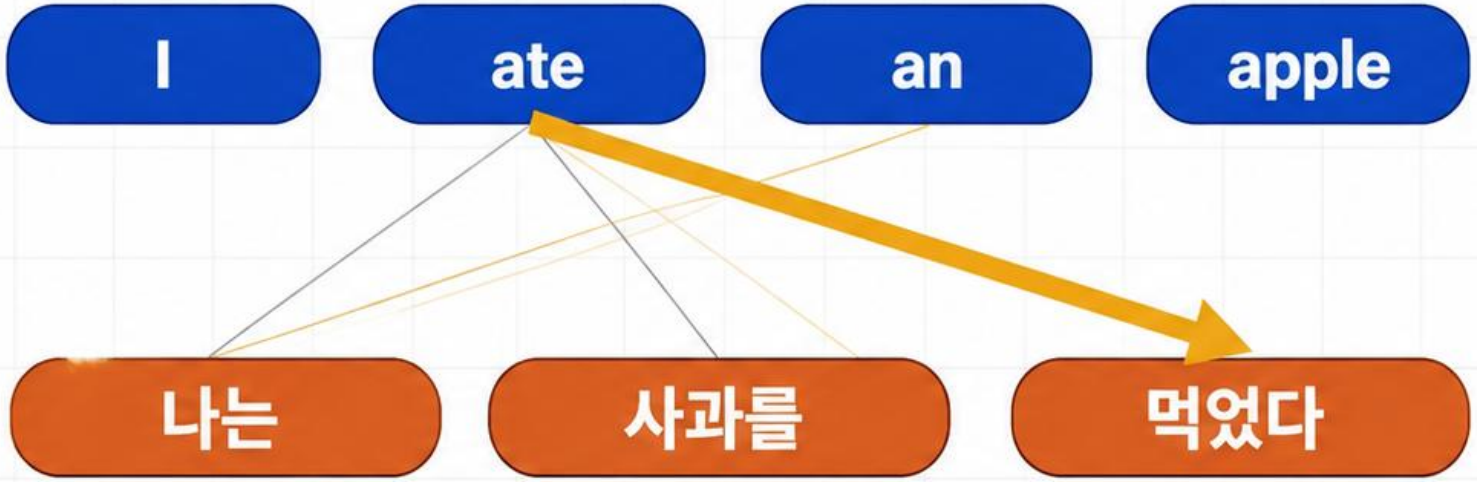


Context Vector의 한계: 인코더는 아무리 긴 문장이라도
결국 단 '하나의 고정 길이 벡터'로 전체 의미를 압축해야만 했습니다.

결과: 문장이 길어질수록 앞부분의 정보, 복잡한 문법적 관계,
그리고 핵심단들이 유실되며 번역/요약의 품질이 급격히 저하됩니다.

문맥을 되돌아보는 집중의 마법: 어텐션(Attention)의 등장

Attention = 출력의 각 시점에서 입력의 어느 부분에 가중치를 둘지 결정하는 메커니즘

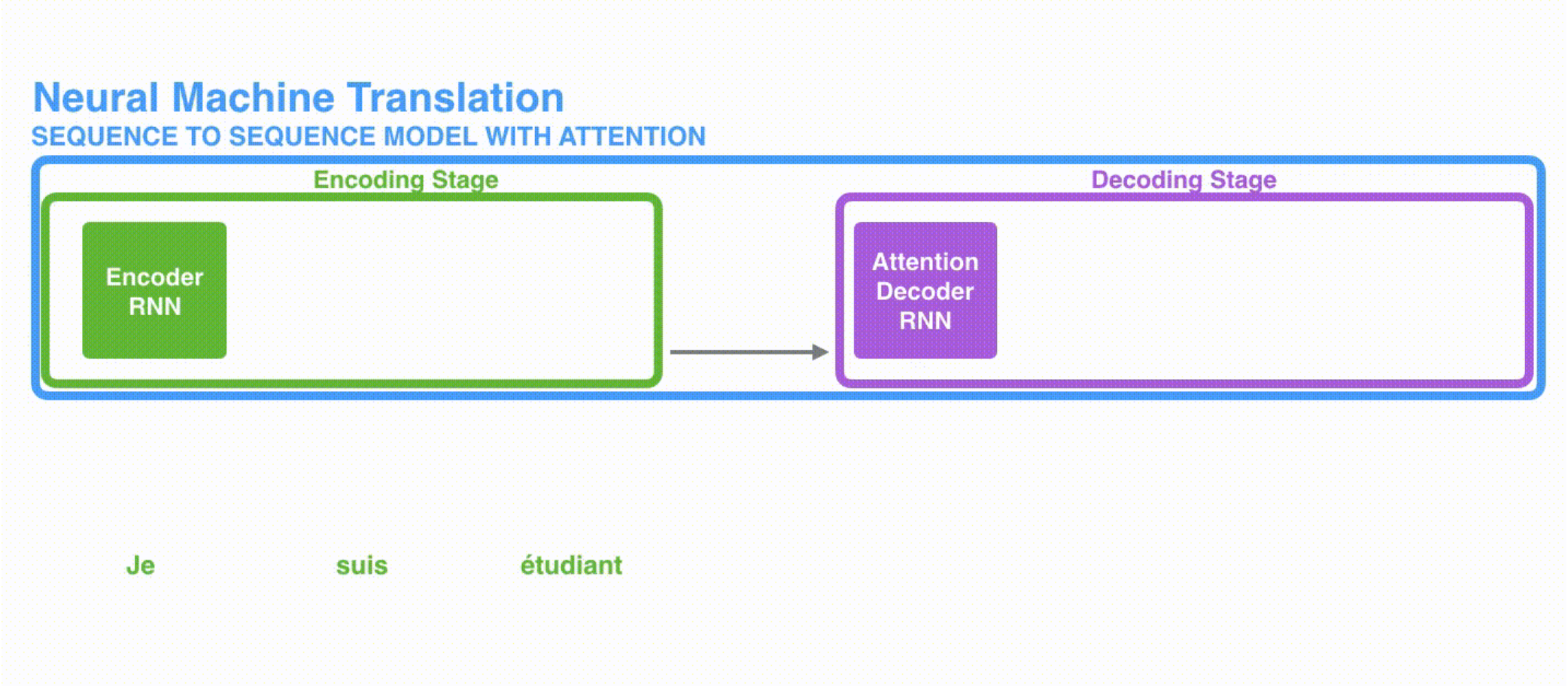


솔루션:	**핵심 이점:**
디코더가 압축된 요약본 **(Context Vector)** 하나만 보지 않습니다. 출력 단어를 만들 때마다 입력 문장 전체를 다시 스캔하여 중요한 부분에 집중합니다.	정보 압축으로 인한 병목 현상을 우회하고, 출력 단어와 입력 단어 사이의 유연한 정렬(**Alignment**)을 모델 스스로 학습합니다.

Seq2Seq with Attention

- Seq2seq의 한계를 극복하기 위해 제안된 개념.
- 기본 아이디어)
  - 디코더에서 출력 단어를 예측하는 매 시점(time step)마다, 인코더에서의 전체 입력 문장을 다시 한 번 참고함
  - 전체 입력 문장을 전부 다 동일한 비율로 참고하는 것이 아니라, 해당 시점에서 예측해야 할 단어와 연관이 있는 입력 단어 부분을 좀 더 집중해서 보는 것이 핵심
  - Decoder에서 출력 토큰 예측 시 매 시점(time step)마다 입력 시퀀스의 토큰을 참조하여 **연관성이 높은 토큰에** 가중치를 높여 학습한다는 것.

Attention

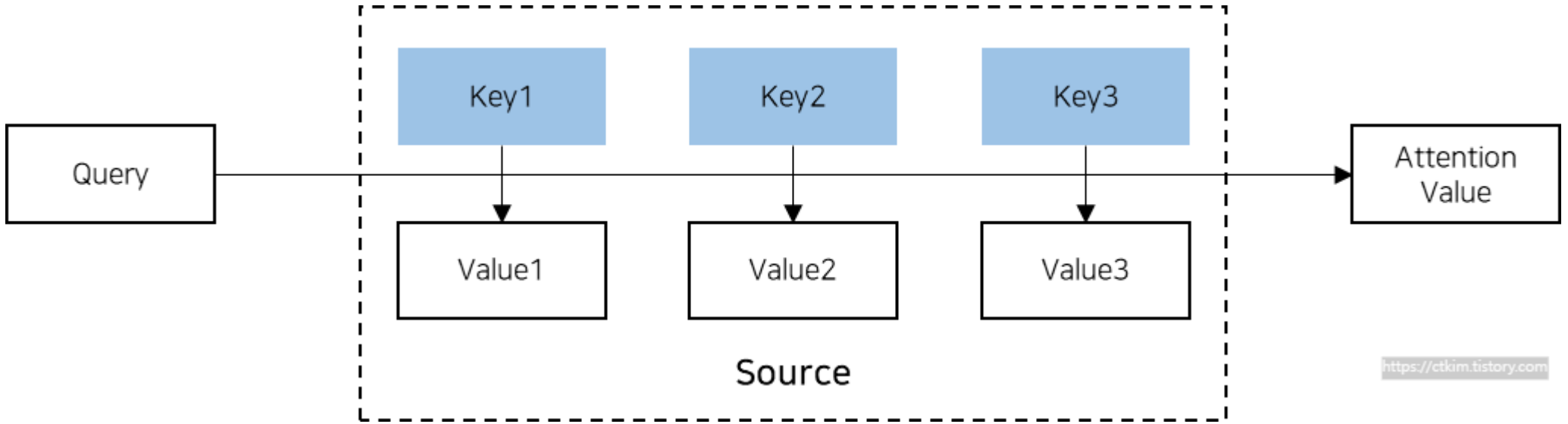


출처 : https://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/

Attention Function

- Attention Function은 입력 시퀀스의 각 단어들에 대한 가중치를 계산하는 함수.
- 이 함수는 각 단어의 중요도를 측정하여 출력 결과에 반영함.
- Query, Key, Value로 구성되어 있음.
- Query는 현재 출력 단어를 나타내는 벡터(t시점의 디코더 셀에서의 은닉 상태)이고,

  Key와 value는 입력 시퀀스의 각 단어에 대응하는 벡터(모든 시점의 인코더 셀의 은닉 상태)

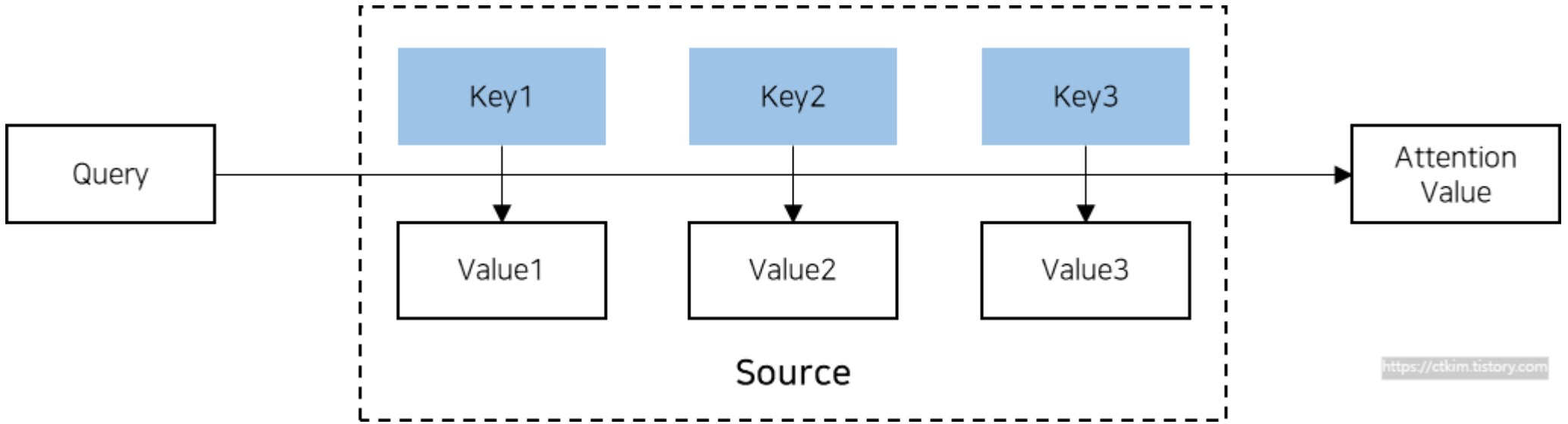


출처 : https://ctkim.tistory.com/entry/Attention-Mechanism

Attention Function

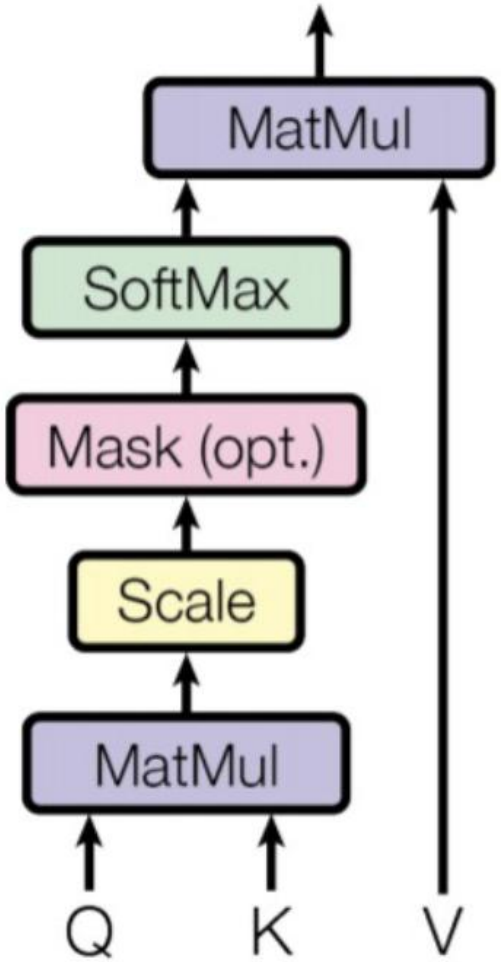
- Attention Function은 먼저 Query와 key 벡터 간의 유사도를 측정.
- 이 유사도는 내적, 외적 등 다양한 방법으로 계산될 수 있음. 이후 유사도를 정규화하여 각 단어 가중치를 계산
- 이 가중치는 value벡터와 곱해져 최종 출력 벡터가 생성됨.
- 다양한 방법으로도 구현될 수 있음. Dot-Product Attention, Multi-Head Attention등이 있음.

Dot-Product Attention은 내적 연산을 사용하여 유사도를 계산하고, Multi-head Attention은 여러 개의 Query, key, value 벡터를 사용하여 각각의 가중치를 계산하여 결합



출처 : https://ctkim.tistory.com/entry/Attention-Mechanism

Scaled Dot-Product Attention

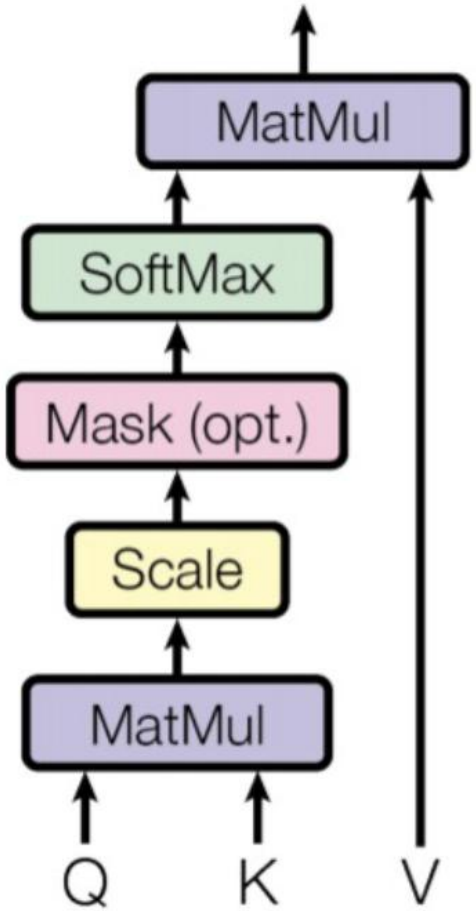


Attention(Q,K,V) = Attention Value

- Q = Query : t 시점의 디코더 셀에서의 은닉 상태
- K = Keys : 모든 시점의 인코더 셀의 은닉 상태들
  - Query에 대한 attention 기여도를 계산할 대상
- V = Values : 모든 시점의 인코더 셀의 은닉 상태들
  - Attention 크기 계산하기 위한 값.

출처 : https://arxiv.org/pdf/1706.03762.pdf

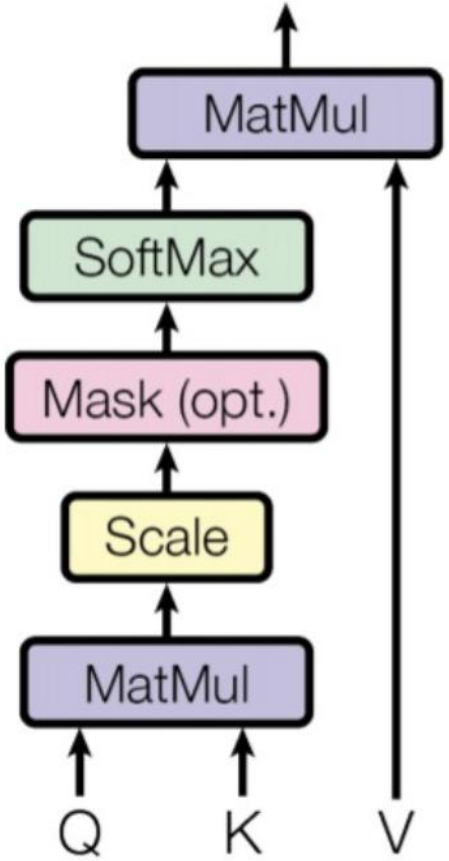
Scaled Dot-Product Attention



- Attention에서 Encoder의 모든 hidden state중에서 어디서 더 집중할 것인가를 Query, Key, Value를 통해서 구함.
  - Attention 함수는 주어진 Query에 대해서 모든 Key와의 유사도를 각각 구함.
  - 해당 유사도를 키와 맵핑되어 있는 각각의 값(Value)에 반영함.
  - 유사도가 반영된 값(Value)을 모두 더해서 리턴

출처 : https://arxiv.org/pdf/1706.03762.pdf

Scaled Dot-Product Attention



- Q, K, V : 각각 Query, Key, Value

$$Attension(Q,K,V) = Softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

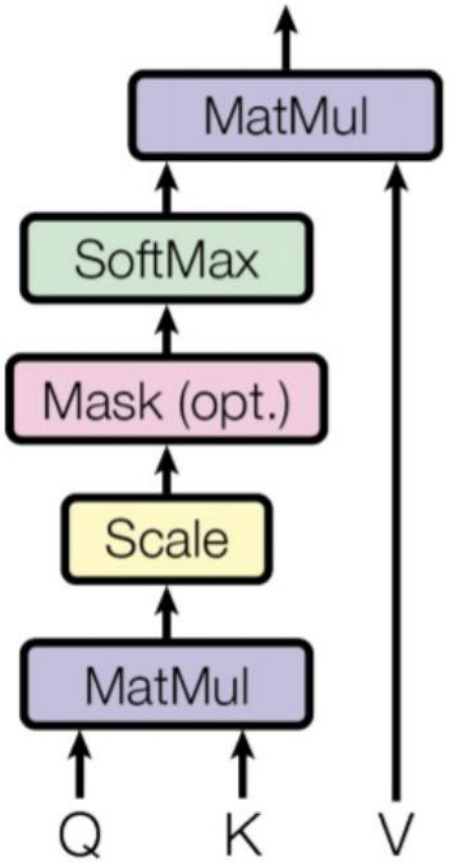
- d_k : key 벡터의 차원 수를 의미.
- $\frac{QK^T}{\sqrt{d_k}}$: Query벡터와 Key 벡터 간의 유사도를 계산하는 것.

-> 이를 통해 어탠션 매커니즘이 입력 시퀀스 내에서 특정 토큰에 집중할 수 있도록 도와줌.

- $\sqrt{d_k}$로 나눠주는 이유, key 벡터의 차원 수가 증가할수록 내적 값이 커짐.

출처 : https://arxiv.org/pdf/1706.03762.pdf

Scaled Dot-Product Attention



- Q, K, V : 각각 Query, Key, Value

$$Attension(Q,K,V) = Softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$\sqrt{d_k}$로 나눠주는 이유, key 벡터의 차원 수가 증가할수록 내적 값이 커짐.

이를 방지하기 위해 Softmax함수를 통해 정규화하여 어텐션 분포 조절, 어텐션 가중치를 균등하게 분배할수 있도록 도와줌.

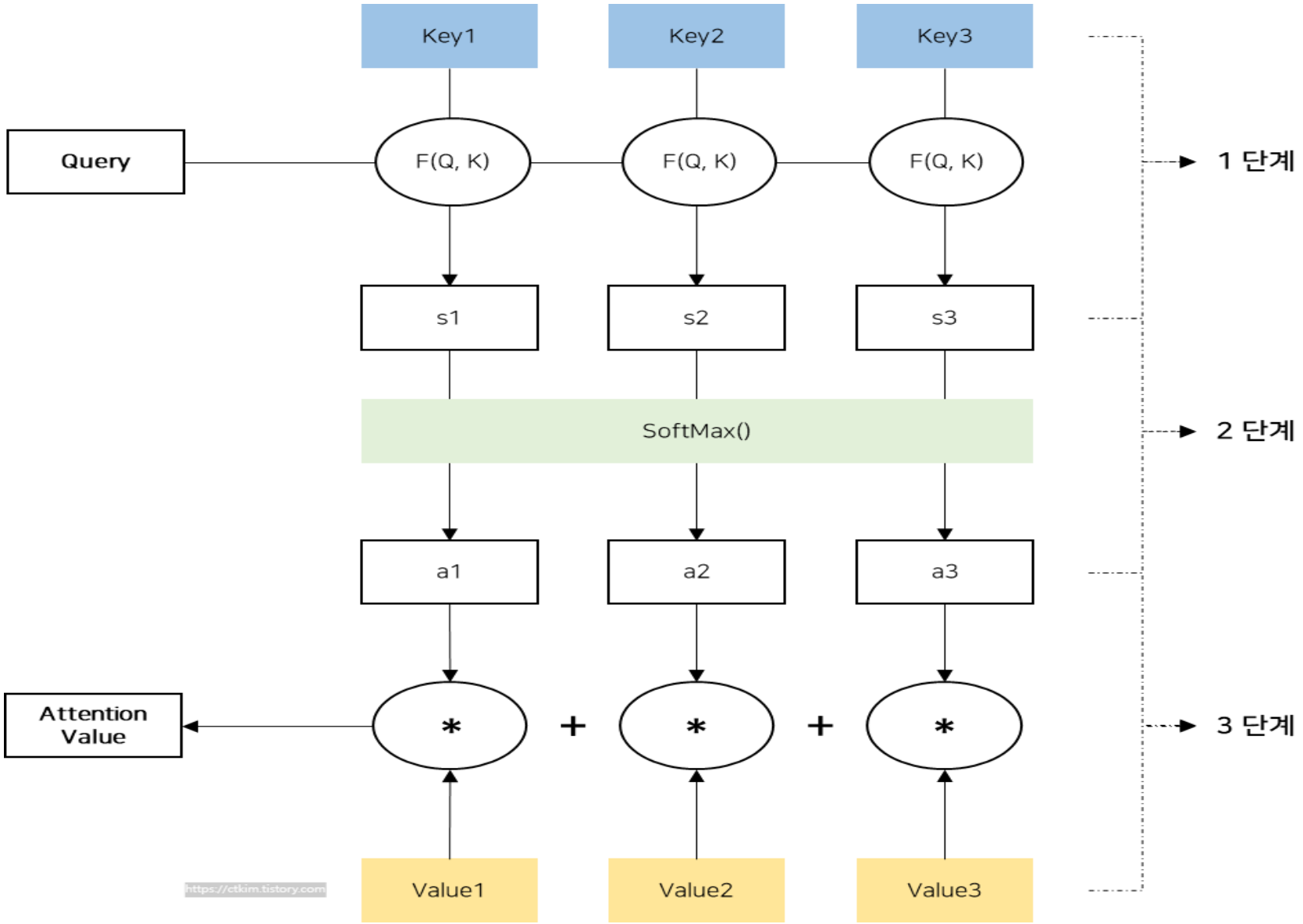
V를 곱해 최종 값을 구함.

-> V를 곱하는 이유는 어텐션 가중치를 이용하여 입력 시퀀스에서 해당 토큰의 정보를 추출하기 위해서.

출처 : https://arxiv.org/pdf/1706.03762.pdf

Scaled Dot-Product Attention

- 단계별로 살펴보자

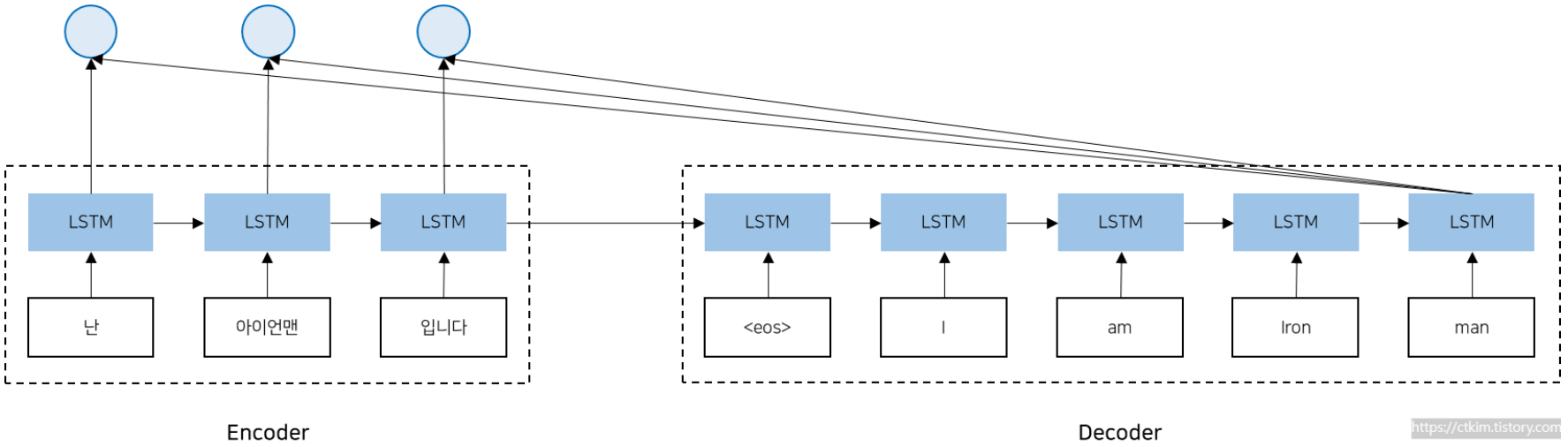


출처 : https://ctkim.tistory.com/entry/Attention-Mechanism

1단계: Attention Score 계산

- Attention Score는 Query와 Key 사이의 유사도를 나타내는 값.
- 여기서 Query는 decoder의 t 시점의 은닉 상태, key는 encoder의 모든 시점의 은닉 상태를 의미.
- Query와 key의 내적을 통해 얻은 값은 유사도를 나타내며, 이 값을 key 벡터의 차원의 수 $\sqrt{d_k}$으로 나눠 스케일링.
- 결과적으로 각 인코더 시점의 은닉 상태와 디코더의 t 시점의 은닉 상태간의 어텐션 스코어가 계산됨.

$$Score(Q, K) = \left(\frac{QK^T}{\sqrt{d_k}}\right)$$

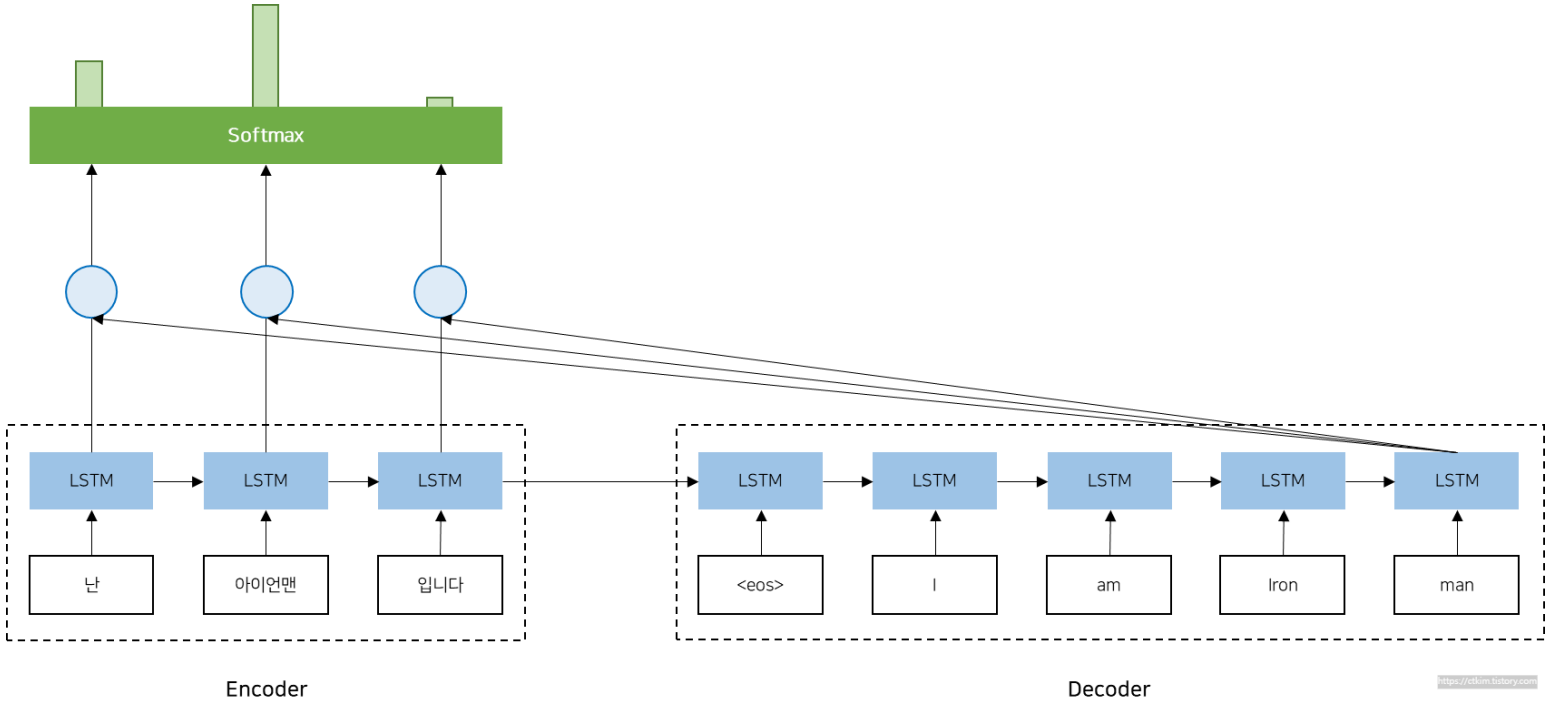


출처 : https://ctkim.tistory.com/entry/Attention-Mechanism

2단계: Attention Distribution 계산

- 어텐션 스코어를 softmax 함수에 적용하면, 각 인코더 시점의 은닉 상태에 대한 가중치(확률 분포)를 얻을 수 있음.
- 이 가중치는 디코더의 t시점에 인코더의 각 시점의 정보가 얼마나 중요한지를 나타냄.
- Softmax 함수를 통해 얻은 모든 인코더 시점에 대해 합이 1이 되는 확률 분포를 가짐.

$$Attention\ Distribution = softmax(Score(Q, K))$$

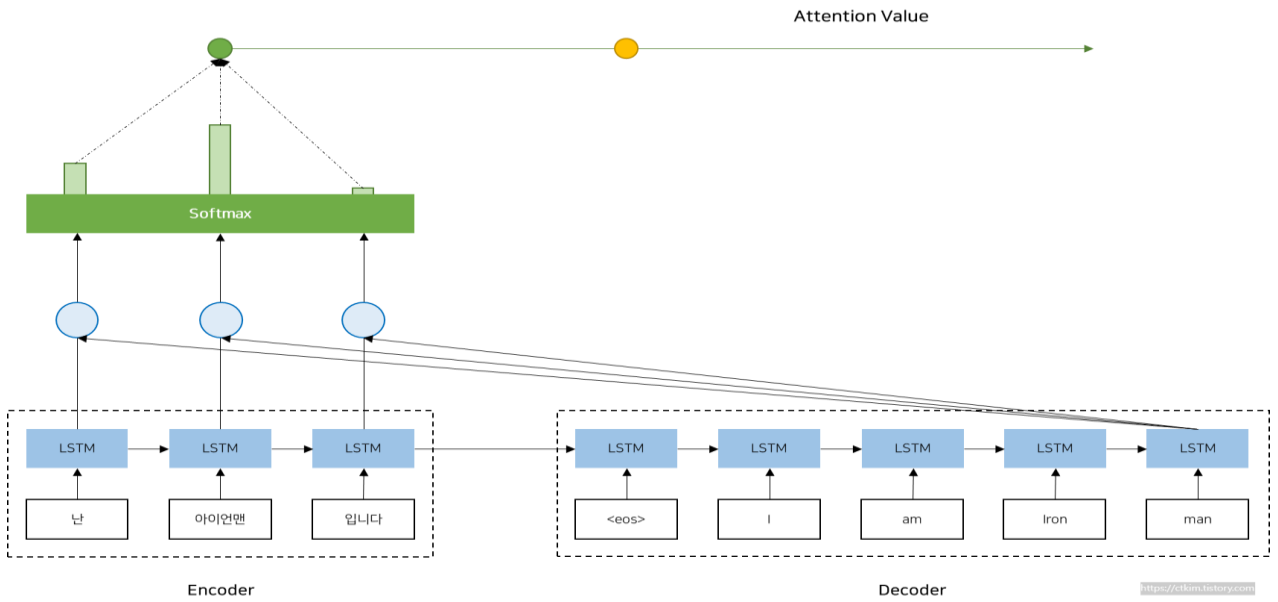


출처 : https://ctkim.tistory.com/entry/Attention-Mechanism

3단계: Attention Value계산

- 어텐션 가중치와 인코더의 값(Value) 벡터를 곱한 후, 모든 인코더 시점에 대해 합산하여 Attention Value을 구함.
- 여기서 Value 벡터는 인코더의 각 시점의 은닉 상태를 의미함.
- 어텐션 값은 인코더의 정보를 디코더에 전달하는 역할을 하며, 가중치를 고려해 각 인코더 시점의 정보를 조합한 결과물.

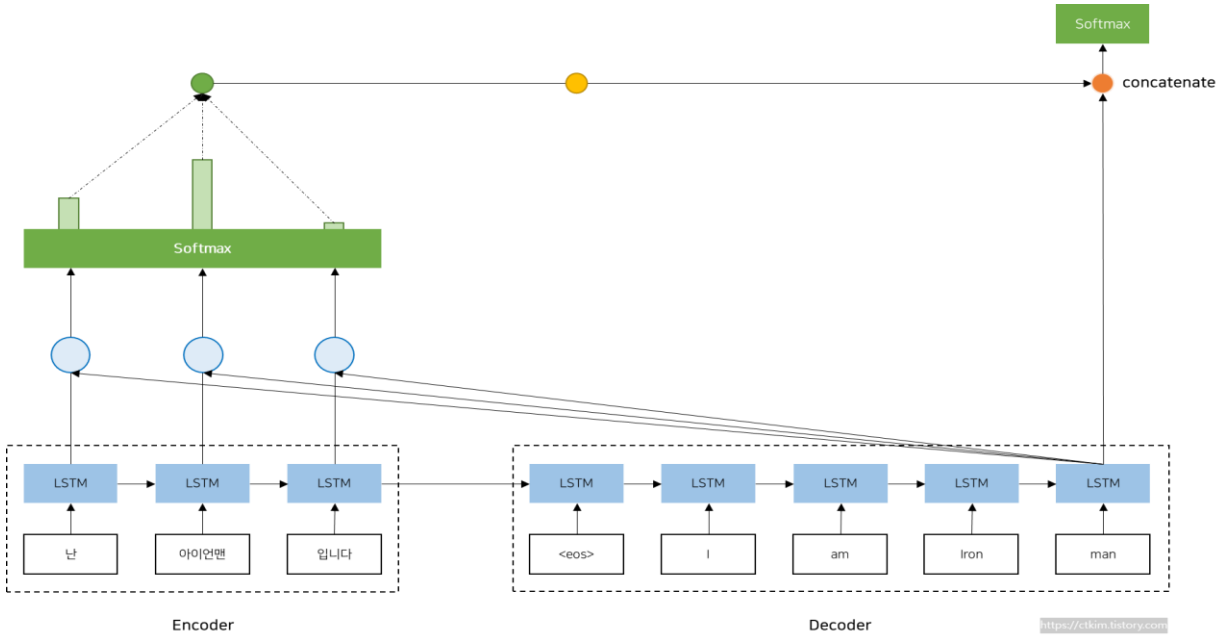
$$AttentionValue = Attention\ Distribution \times V$$



출처 : https://ctkim.tistory.com/entry/Attention-Mechanism

4단계: Attention Value과 디코더의 은닉 상태 concatenate

- 어탠션 값과 디코더의 t시점 은닉상태를 연결(concatenate)하여, 새로운 입력으로 사용.
- 어텐션 매커니즘을 통해 인코더와 디코더가 상호 작용하며, 입력 문장의 정보를 공유하게 됨.
- 두 벡터를 Concatenate : $[h_t : c_t]$
- 연결된 벡터를 적절한 크기의 출력 벡터로 변환. $o_t = f([h_t : c_t])$
- 출력 벡터를 소프트 맥스 함수에 적용하여 최종확률 분포를 얻음 : $p_t = softmax(o_t)$



출처 : https://ctkim.tistory.com/entry/Attention-Mechanism

Scaled Dot-Product Attention의 특징

1. 계산이 빠름. 내적 연산은 고속으로 처리할 수 있기 때문에, 다른 Attention 방법보다 계산 속도가 빠름.

2. 정규화를 위해 스케일링이 사용됨. 내적 연산을 통해 계산된 유사도를 정규화 하는데, 내적 연산 결과의 차원 수를 나누어 스케일링을 수행함. 이를 통해 가중치의 분산을 조절하여 학습을 안정화 시키고, Attention mechanism의 성능을 향상시킴

3. 병렬 처리가 가능함. Scaled Dot-Product Attention은 입력 시퀀스의 각 단어 간에 독립적으로 계산이 가능하므로, 병렬 처리에 용이함. 이를 통해 더 빠른 학습 속도를 달성할 수 있음.

4. 다양한 입력 시퀀스 길이를 처리할 수 있음. Scaled Dot-Product Attention은 입력 시퀀스의 길이에 대해 제한이 없기 때문에, 다양한 길이의 입력 시퀀스를 처리할 수 있음.

다양한 종류의 Attention

1. Dot-Product Attention

   : Query와 Key의 내적을 사용하여 어텐션 스코어를 계산

$$Score(Q, K) = QK^T$$

2. Scaled Dot-Product Attention

   : Dot-Product Attention의 변형으로, 내적 값을 키 벡터의 차원 수의 제곱근 $\sqrt{d_k}$으로 나누어 스케일링함.

   : 이는 각 차원에 대한 영향력이 과도하게 커지는 것을 방지하고, 학습의 안정성을 향상시킴.

$$Score(Q, K) = \frac{QK^T}{\sqrt{d_k}}$$

다양한 종류의 Attention

3. Additive Attention

: Query와 Key를 결합한 후, 학습 가능한 가중치 행렬을 적용하여 어텐션 스코어를 계산하는 방식

: 이후 활성화 함수를 적용한 결과를 가지고 어텐션 스코어를 구함.

$$Score(Q, K) = v^T * \tanh(W_1 * Q + W_2 * K)$$

4. Multi-Head Attention

: 여러 개의 독립적인 어텐션을 병렬로 수행한 후, 결과를 결합하는 방식.

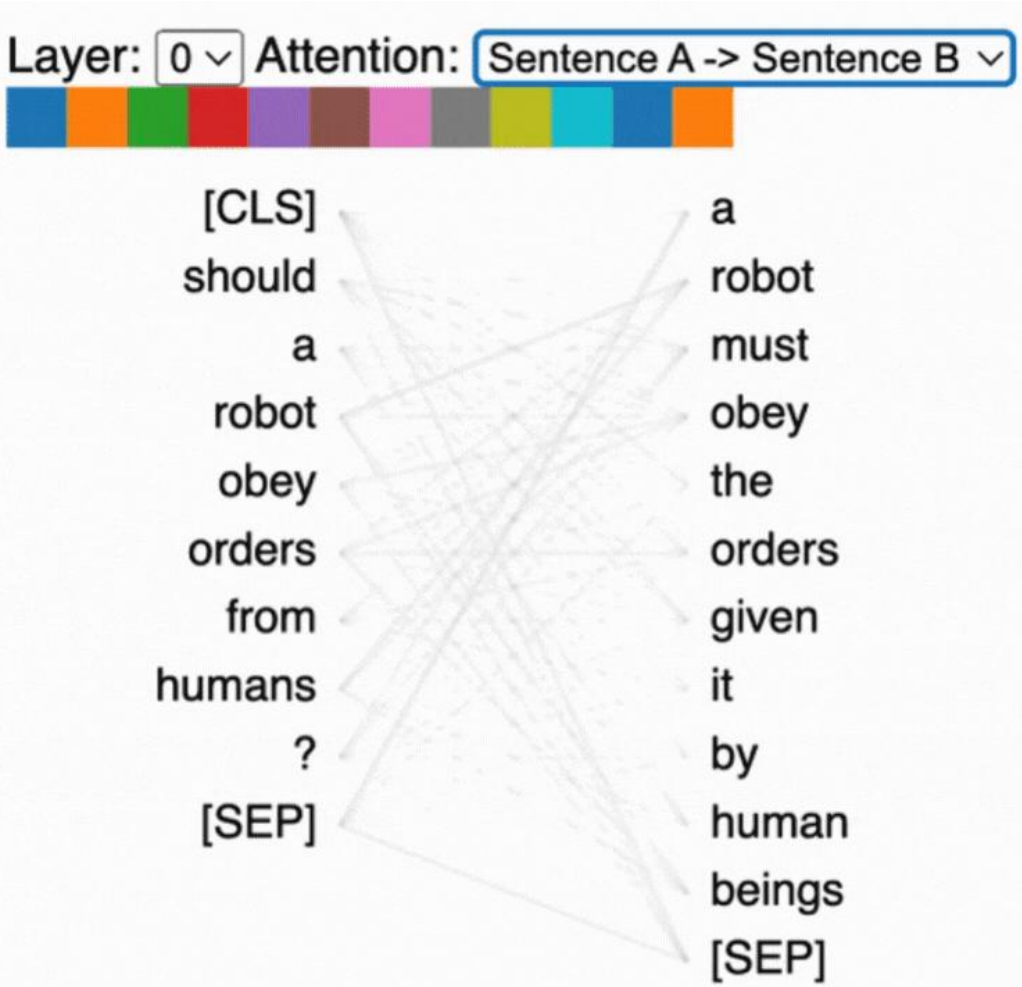
: 각 어텐션 헤드는 서로 다른 가중치를 사용하여 입력에 대한 다양한 관점이 있습니다.

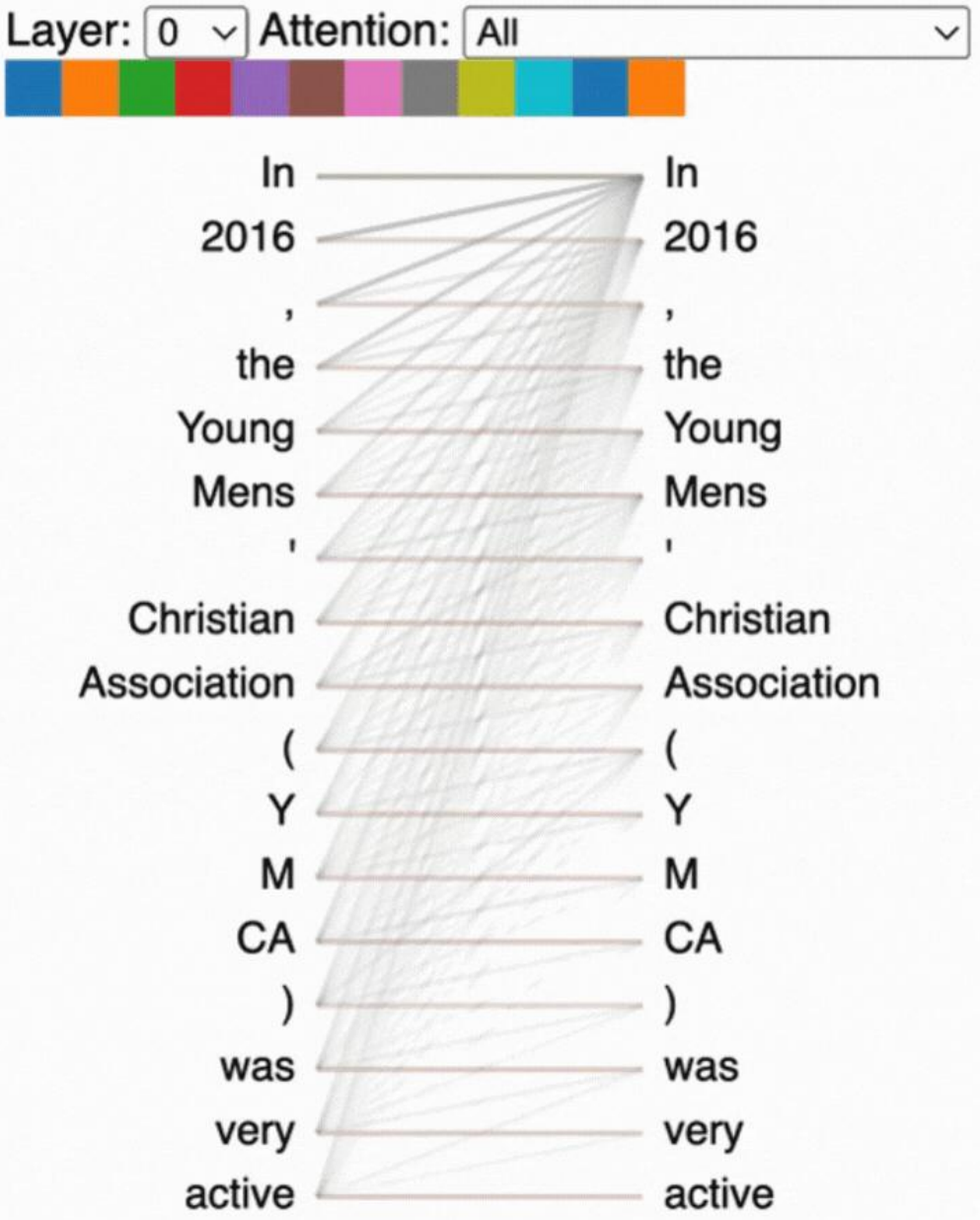
일반적으로 Scaled Dot-Product Attention을 기반으로 함.

$$MultHead(Q, K, V) = Concat(head_1, \dots, head_h) * W^O$$

$$head_i = Attention(Q * W_i^Q, K * W_i^K, V * W_i^V)$$

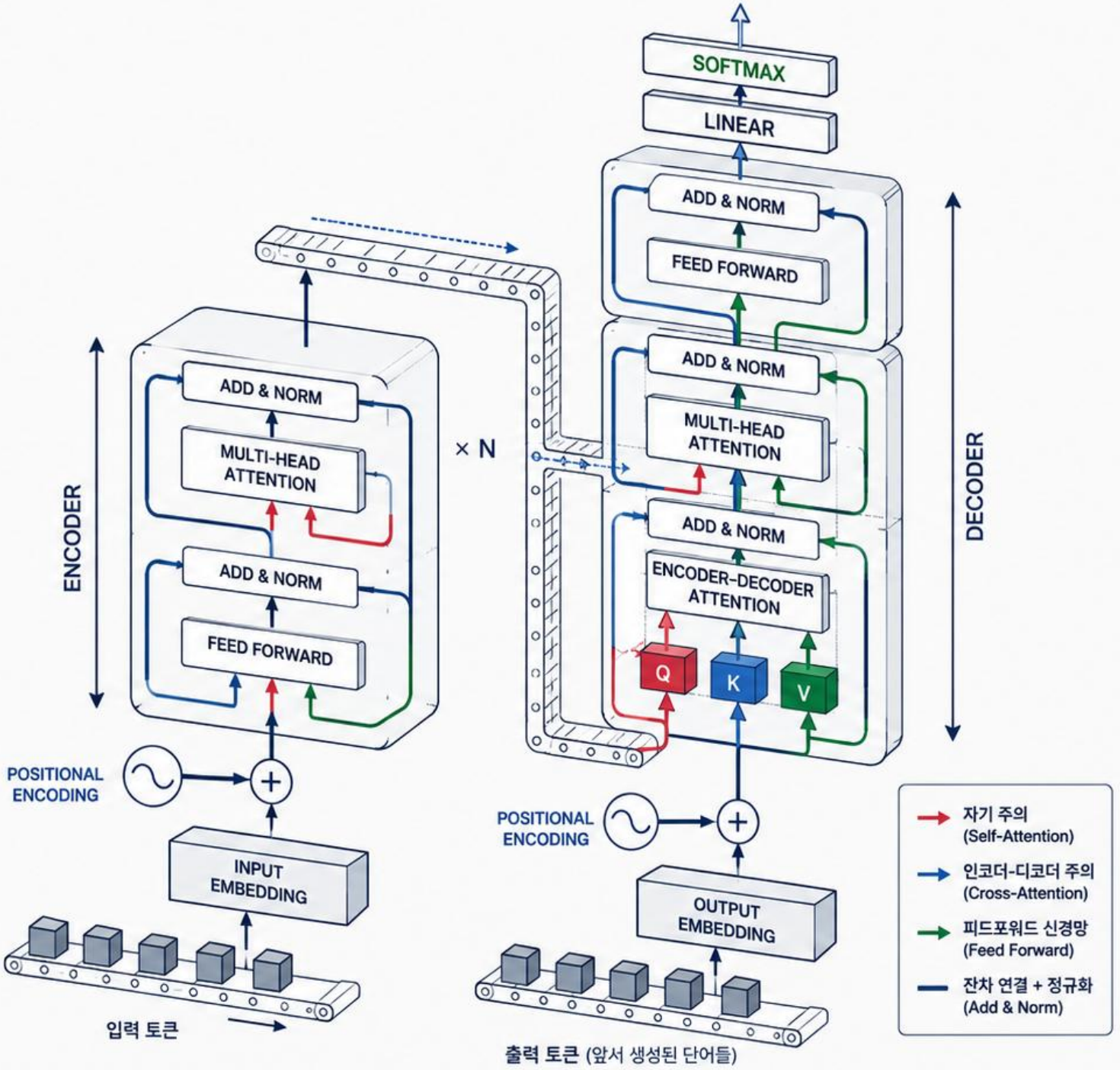
Attention_map





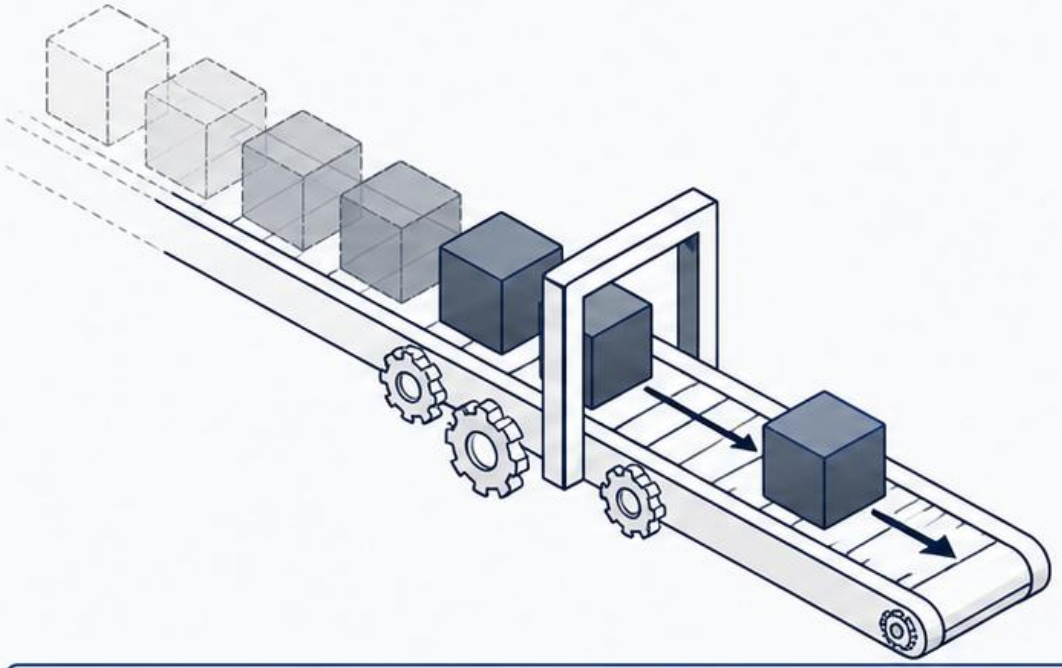
지능의 설계도: 트랜스포머 아키텍처의 해부

Attention is All You Need —
블랙박스 너머, 현대 AI를 구동하는
핵심 원리 완벽 가이드.



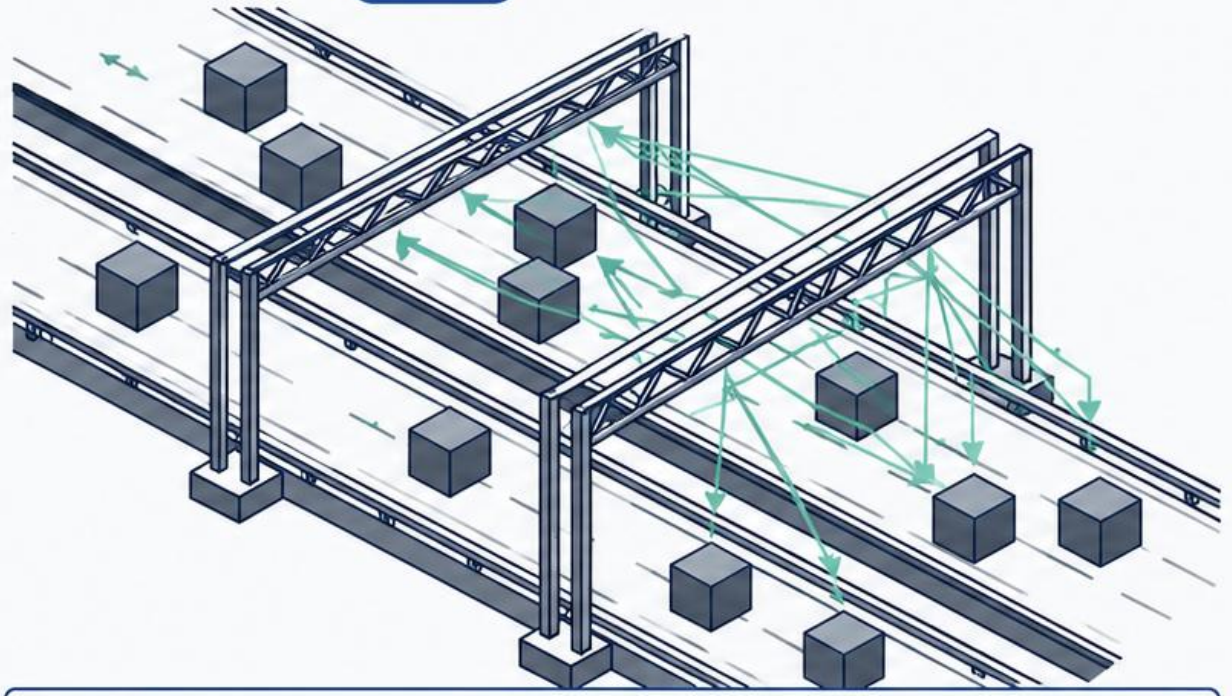
패러다임의 전환: 순차적 읽기에서 병렬적 읽기로

과거: RNN/LSTM



- **데이터 처리:** 순차적 (Sequential) - 한 번에 한 단어씩 처리.
- **장기 의존성:** 거리가 멀어지면 정보가 유실됨 (소실 경사 문제).
- **하드웨어 효율:** 병렬화 불가능, 낮은 GPU 활용도.

현재: 트랜스포머

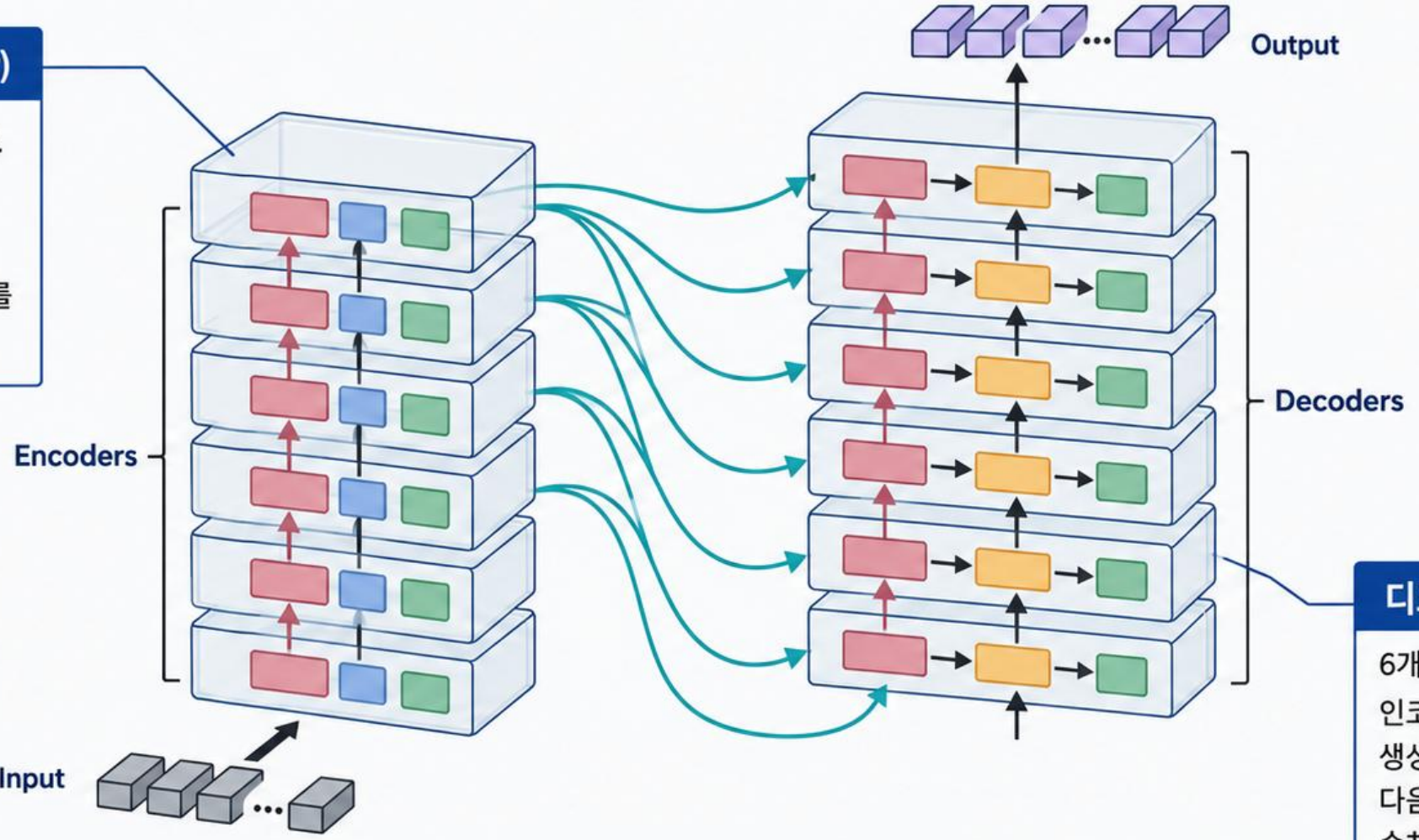


- **데이터 처리:** 병렬적 (Parallel) - 모든 단어를 동시에 처리.
- **장기 의존성:** 어텐션(Attention)을 통해 거리에 상관없이 직접 참조.
- **하드웨어 효율:** 고도의 병렬 처리로 클라우드 TPU/GPU에 최적화.

매크로 뷰: 인코더와 디코더의 상호작용

인코더 스택 (6 Layer)

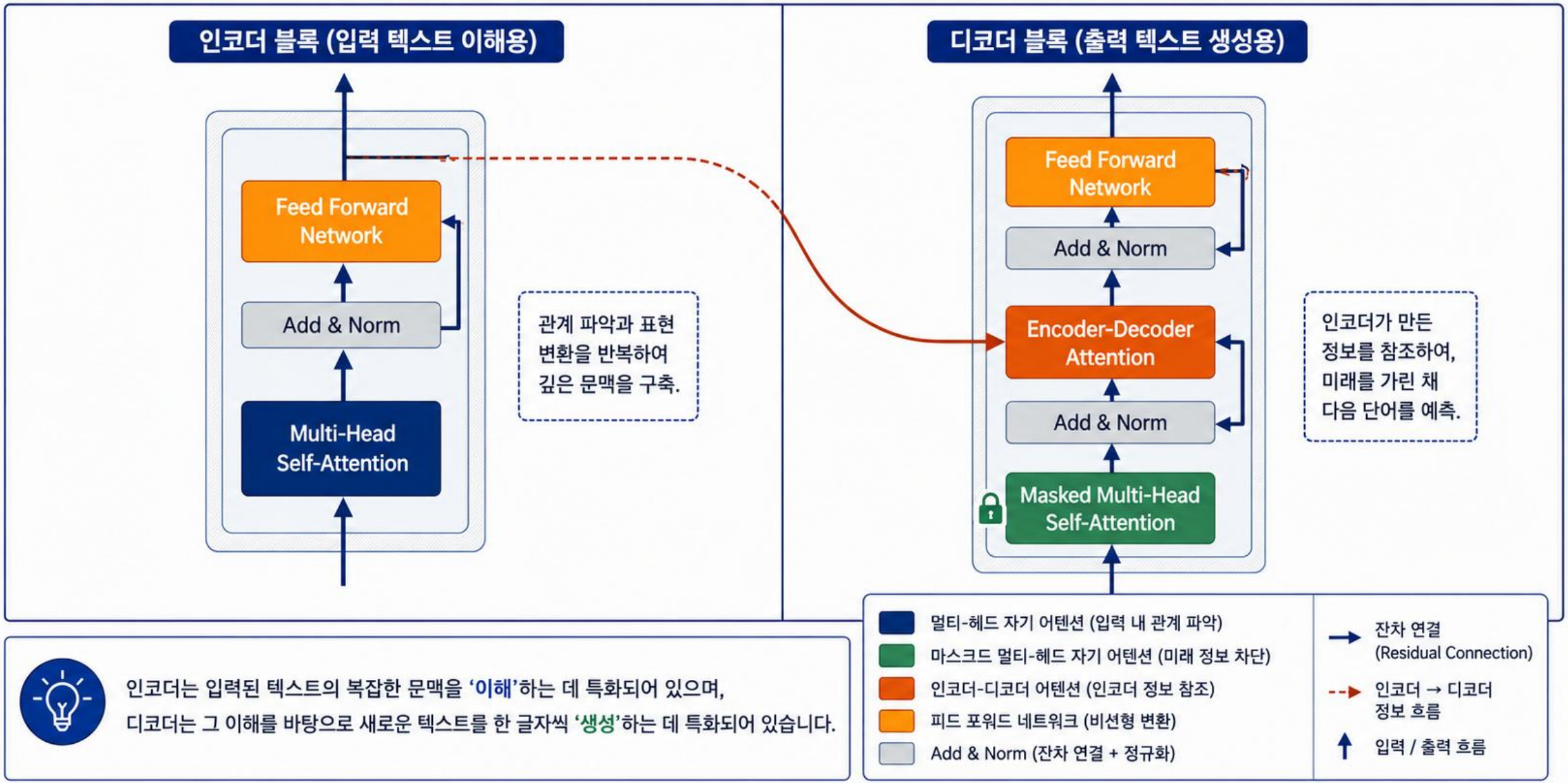
6개의 동일한 인코더 블록.
입력된 문장의 전체적인
문맥과 의미를 입력하게
파악하여 '문맥화된 백터'를
생성합니다.



디코더 스택 (6 Layer)

6개의 디코더 블록.
인코더의 출력과 이전에
생성된 단어들을 바탕으로
다음 단어를 하나씩
순차적으로 예측합니다.

아키텍처 조립: 인코더와 디코더 블록



인코더는 입력된 텍스트의 복잡한 문맥을 **'이해'**하는 데 특화되어 있으며,
디코더는 그 이해를 바탕으로 새로운 텍스트를 한 글자씩 **'생성'**하는 데 특화되어 있습니다.

The Dual-Signal Blueprint

단어의 위치가 곧 문장의 운명을 결정합니다



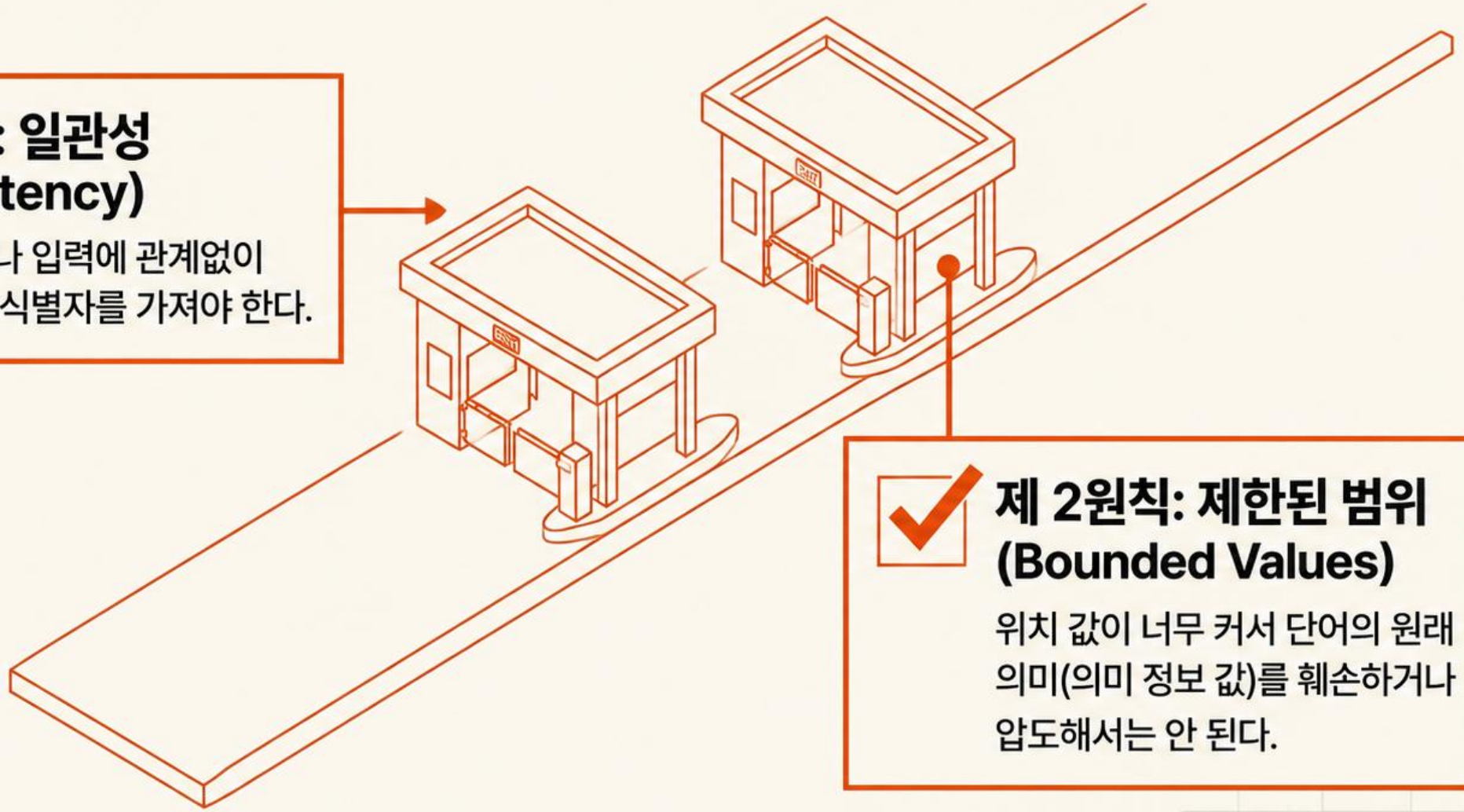
트랜스포머는 모든 단어를 한 번에 입력받기 때문에,
위치 정보가 없다면 의미는 완전히 붕괴됩니다.

The Dual-Signal Blueprint

완벽한 위치 시스템을 위한 두 가지 절대 규칙

제 1원칙: 일관성 (Consistency)

시퀀스 길이나 입력에 관계없이 동일한 위치 식별자를 가져야 한다.



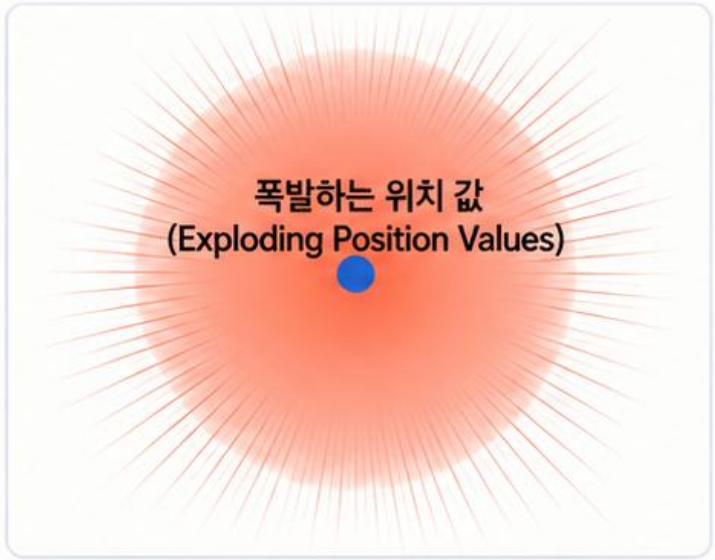
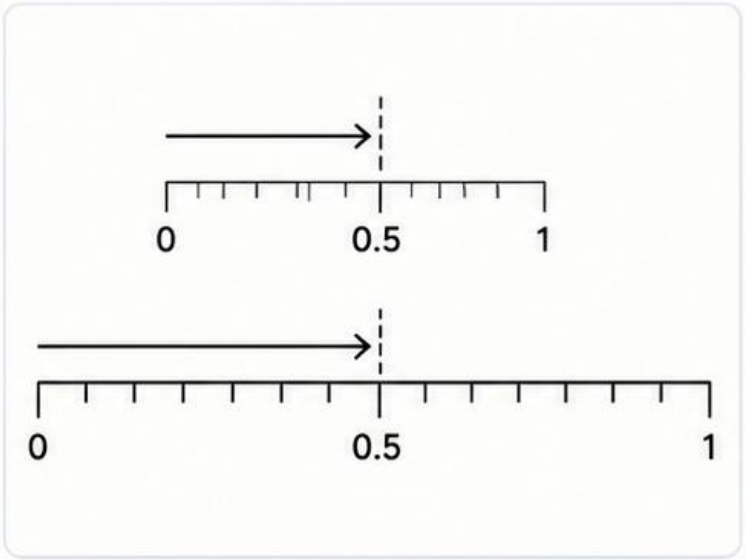
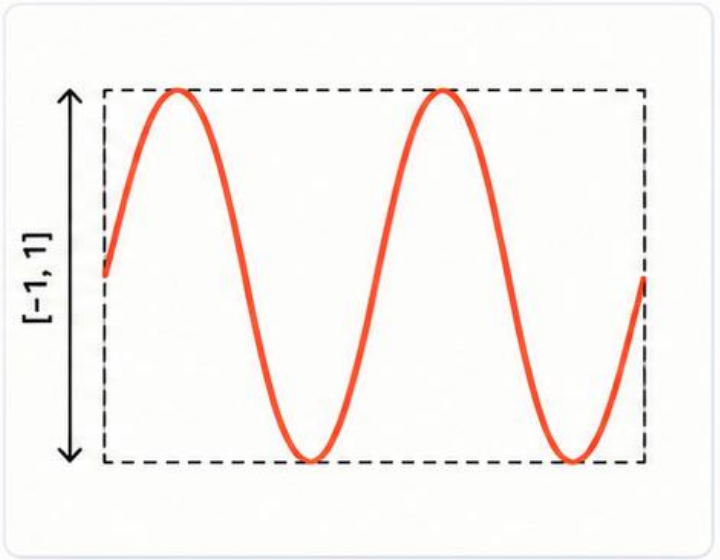
제 2원칙: 제한된 범위 (Bounded Values)

위치 값이 너무 커서 단어의 원래 의미(의미 정보 값)를 훼손하거나 압도해서는 안 된다.

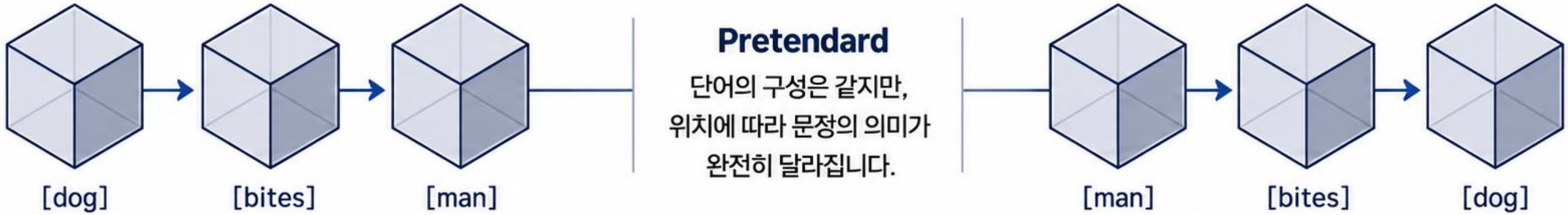
The Dual-Signal Blueprint

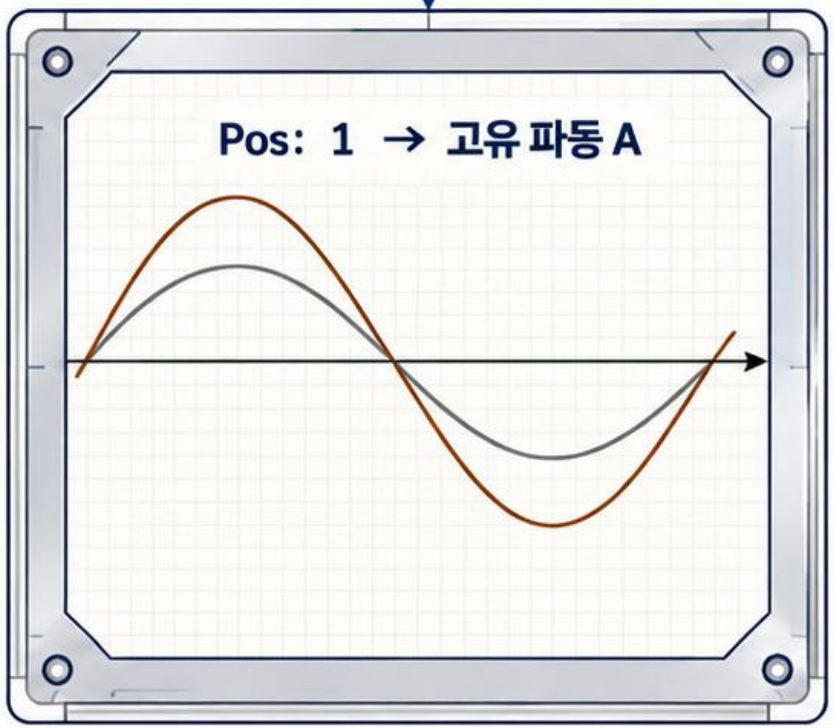
위치 벡터를 얻기 위한 실험: 왜 단순한 숫자는 실패하는가?

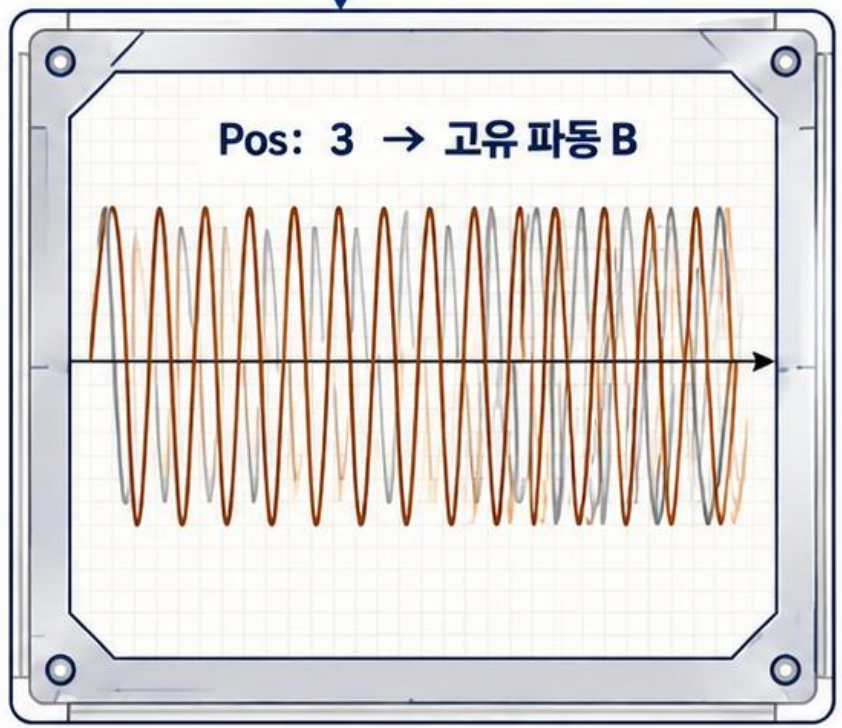
The Positional Failure Matrix

방법 1: 정수 부여 (1, 2, 3...)	방법 2: 정규화 (0.0 ~ 1.0)	방법 3: 주기 함수 (Sine/Cosine)
		
✗	✗	✓
값이 폭발하여 단어의 의미 훼손	문장 길이에 따라 위치 간격이 달라짐	일관된 간격 유지 및 크기 제한 성공

순서를 부여하는 좌표계: Positional Encoding







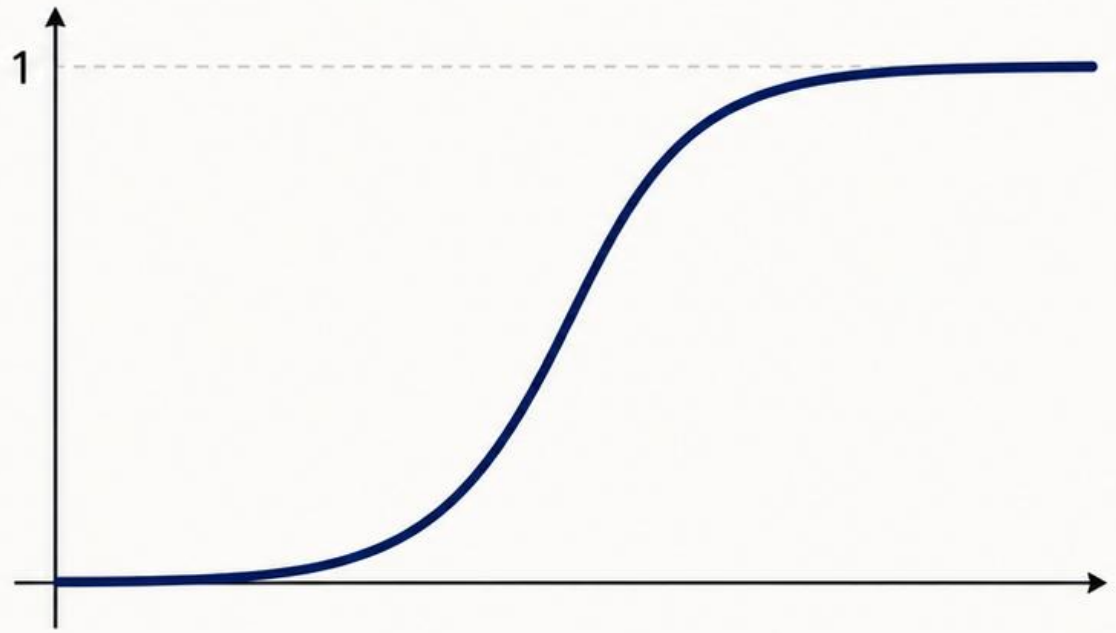
병렬 처리 모델은 단어의 순서를
자력으로 알 수 없습니다.
동일한 단어라도 문장 내 **위치(Pos)**
에 따라 사인/코사인 기반의
고유한 파동(숫자 벡터)을 더해,
기계가 단어의 **'절대적/상대적 위치'**를
공간적으로 인식하게 만듭니다.

The Dual-Signal Blueprint

The Elegance of Waves: 왜 Sigmoid가 아닌 Sine/Cosine인가?

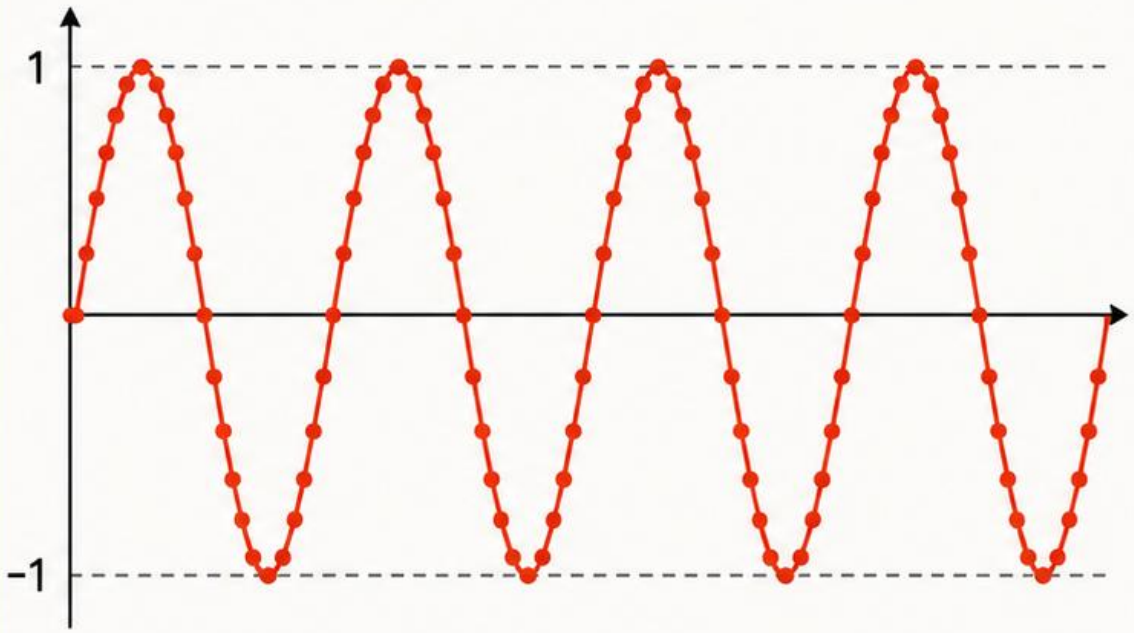
Sigmoid 함수

긴 문장에서 위치 값의 차이가 미미해짐



Sine / Cosine 함수

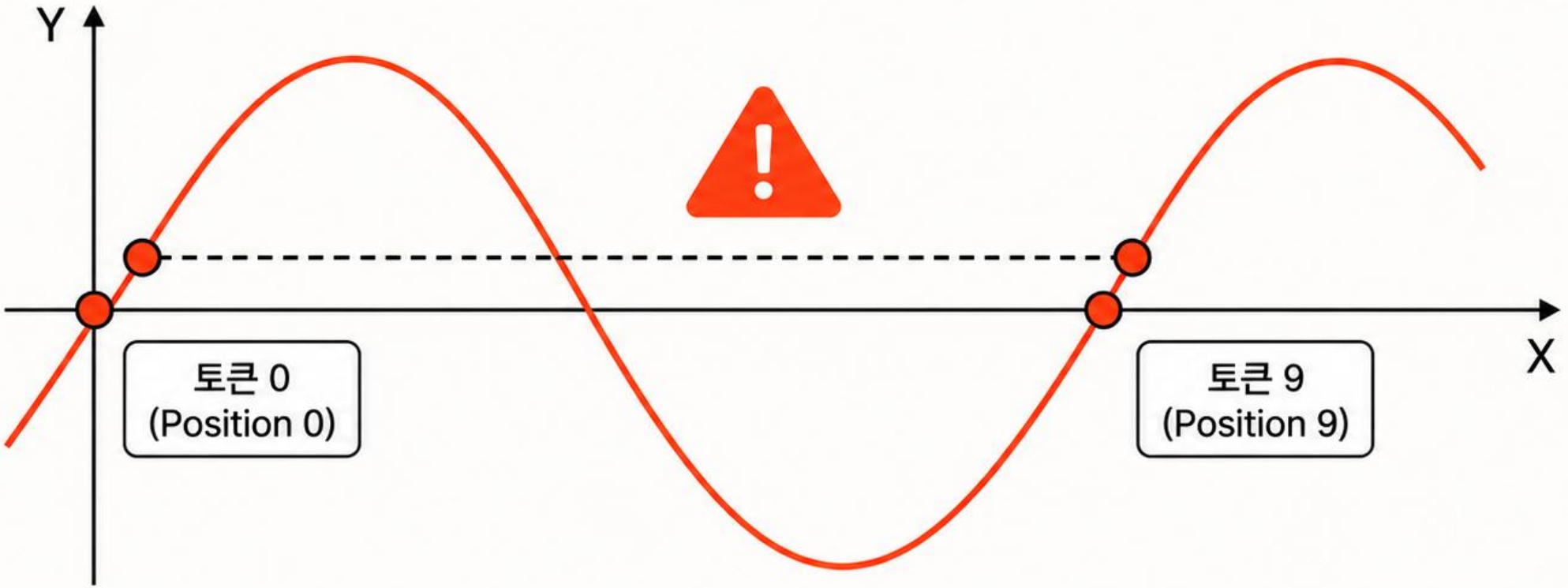
-1과 1 사이를 무한히 반복하며 뚜렷한 차이 유지



주기 파동을 통해 아무리 긴 문장이라도 **고유한 위치 값 차이**를 생성합니다.

The Dual-Signal Blueprint

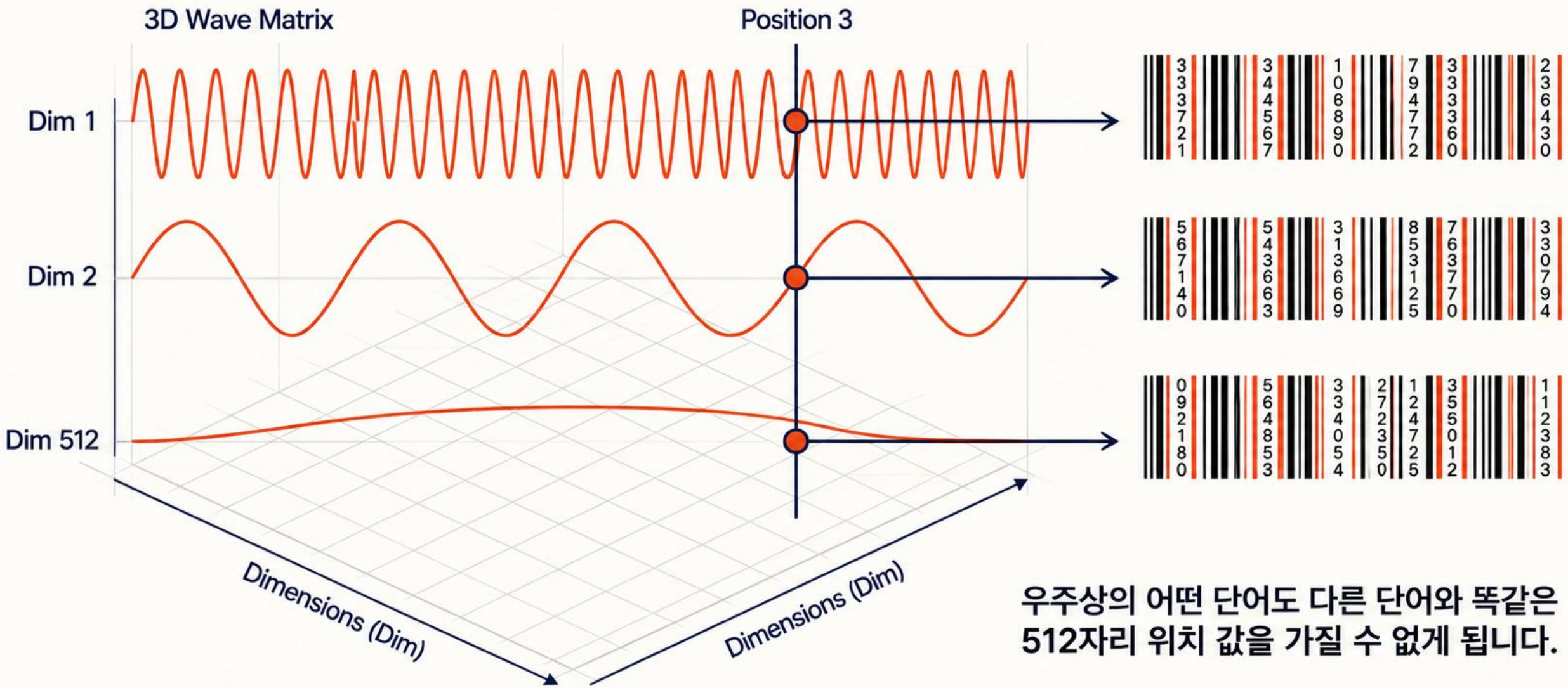
단일 주기의 한계: 동일한 위치 값의 충돌



주기 함수의 치명적 단점: -1과 1 사이를 반복하므로,
전혀 다른 위치의 토큰이 **완전히 동일한 벡터 값**을 갖는 오류가 발생합니다.

The Dual-Signal Blueprint

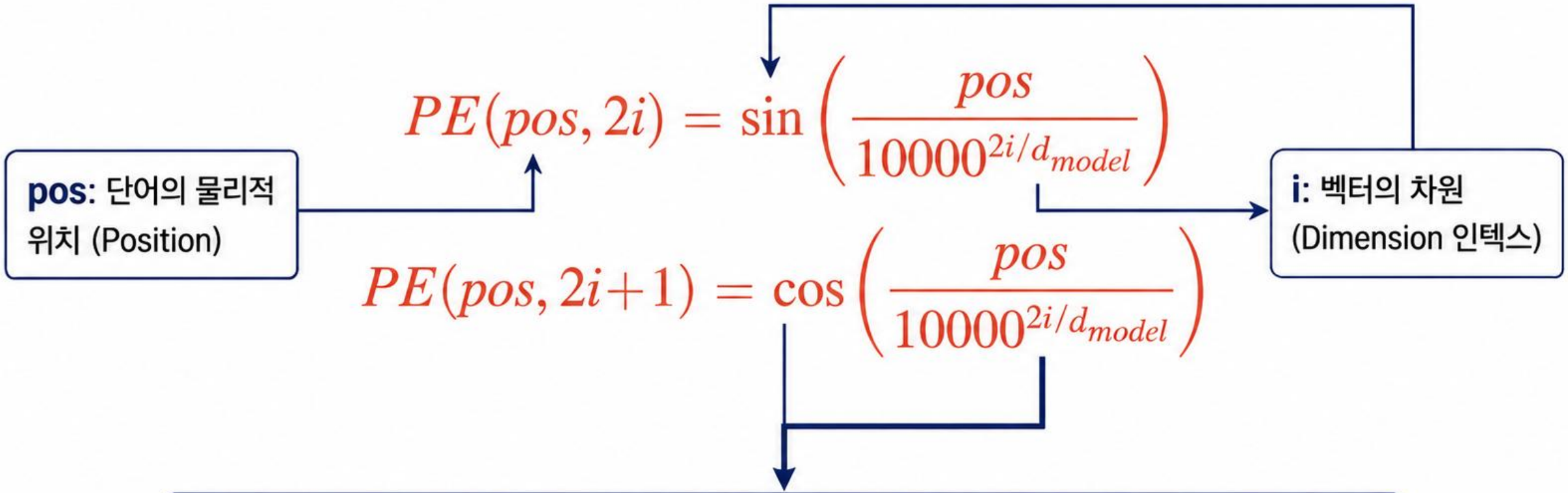
다중 주파수 중첩: 512차원의 '위치 바코드(Positional Barcode)' 생성



우주상의 어떤 단어도 다른 단어와 똑같은
512자리 위치 값을 가질 수 없게 됩니다.

The Dual-Signal Blueprint

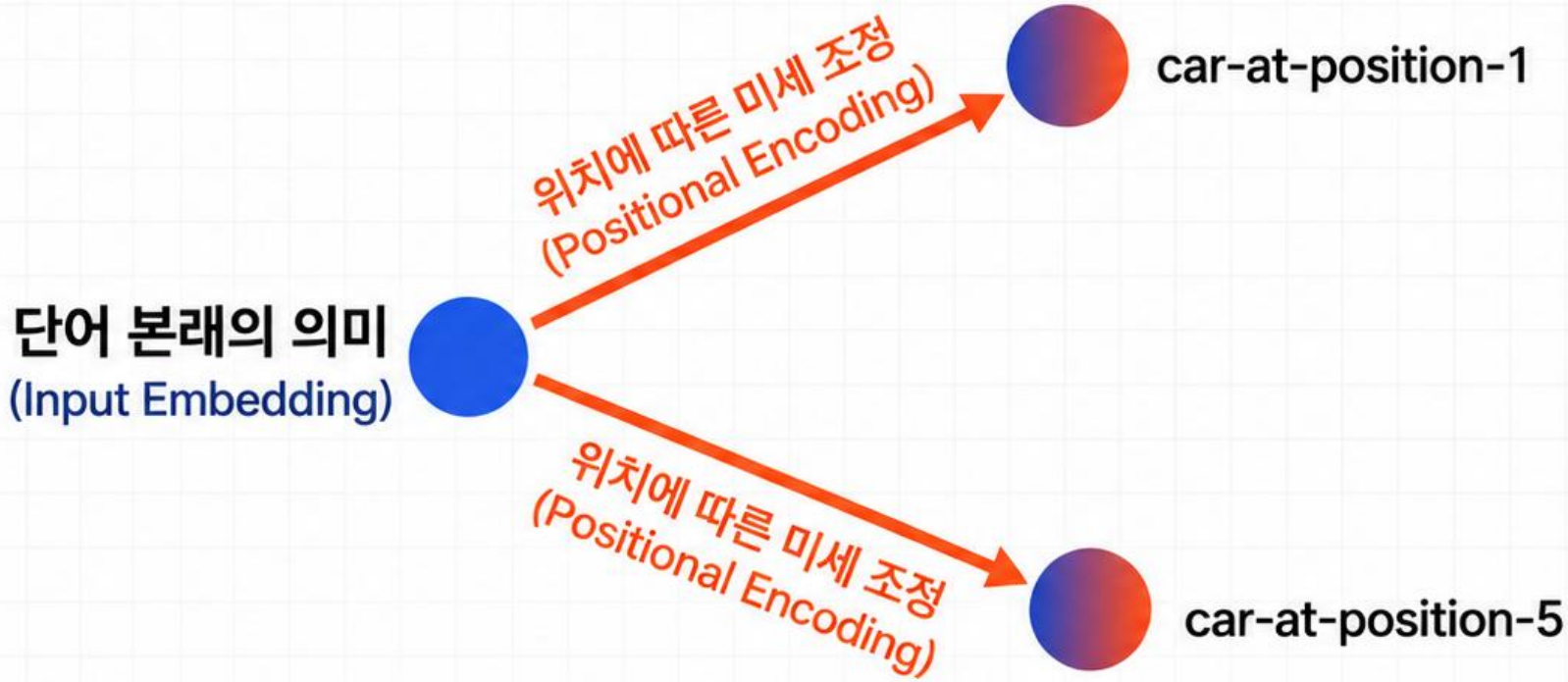
기하학적 톱니바퀴: Sine과 Cosine의 교차 배치



sin / cos 교차: 짝수 차원(2i)에는 Sine, 홀수 차원(2i+1)에는 Cosine을 번갈아 배치하여 차원 간 벡터 값 차이를 극대화.

The Dual-Signal Blueprint

벡터의 미세 조정 (The Vector Nudge): 의미와 위치의 융합



위치 벡터는 단어 벡터를 덮어쓰지 않습니다. 단어 본래 의미의 좌표를
수학적으로 정교하게 **밀어내어(Perturbation)** 문맥적 고유성을 부여합니다.

The Dual-Signal Blueprint

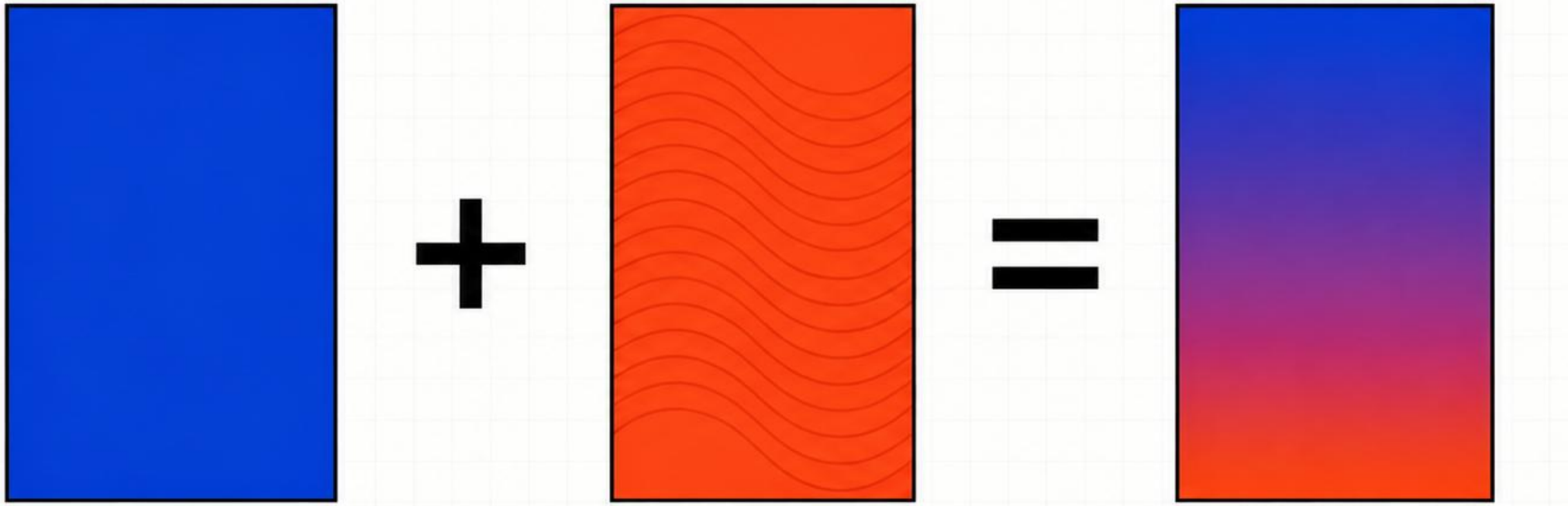
결합의 딜레마: 왜 Concatenate 대신 Summation을 선택했을까?

연산 방식	Concatenation (이어 붙이기)	Summation (더하기)
차원 수	512 + 512 = 1024차원 (**비용 폭발**)	512차원 유지 (**효율적**)
공간적 특징	의미와 위치 공간이 완벽히 분리됨 (**직교성**)	의미와 위치 정보가 섞임 (**균형 유지 필요**)

Hardware Context 2017:
논문이 발표된 2017년의 GPU 메모리 한계로 인해, 저자들은 파라미터 폭발을 막고 정보의 균형을 맞추는 **Summation(+)** 연산을 채택했습니다.

The Dual-Signal Blueprint

완성된 매트릭스: 트랜스포머의 심장으로



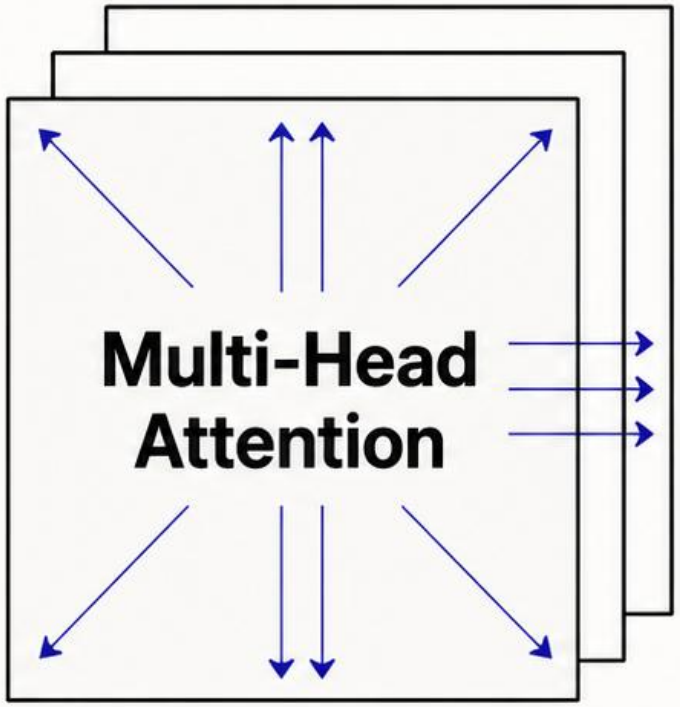
의미 정보(단어의 본질)와 위치 정보(시퀀스 내의 맥락)가 완벽한 비율로 융합되어,
비로소 병렬 처리가 가능한 **트랜스포머의 최종 입력값**으로 완성됩니다.

The Dual-Signal Blueprint

치명적 약점을 압도적 강점으로.

Final Transformer Input



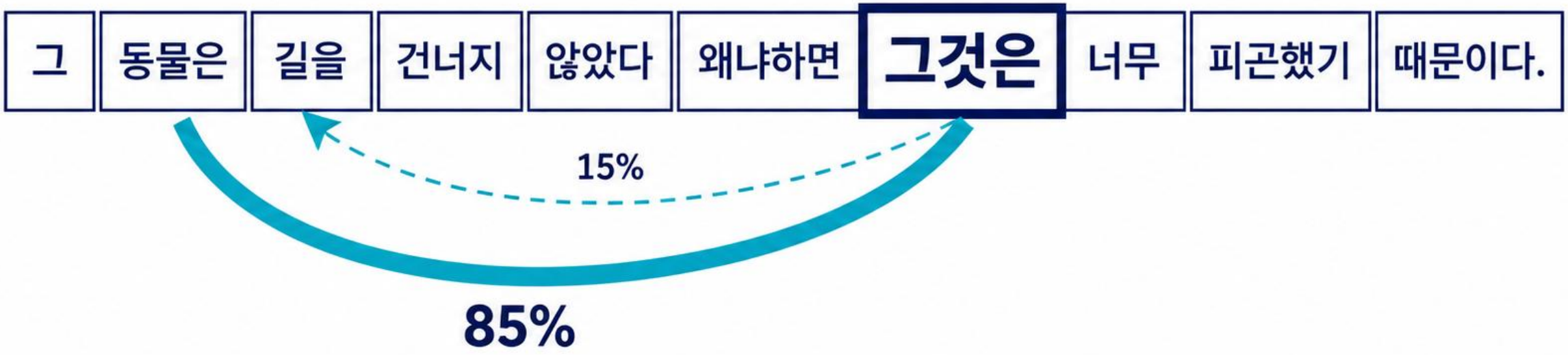


Positional Encoding이라는 기하학적 우아함 덕분에,
트랜스포머는 위치의 기억을 잃지 않고도 수만 개의 단어를
동시에 병렬 처리하는 기적을 만들어냈습니다.

이제 이 완성된 벡터들은 트랜스포머의 진정한 코어,
Multi-Head Attention으로 진입합니다.

Final Transformer Input

셀프 어텐션(Self-Attention): 문맥을 읽는 힘



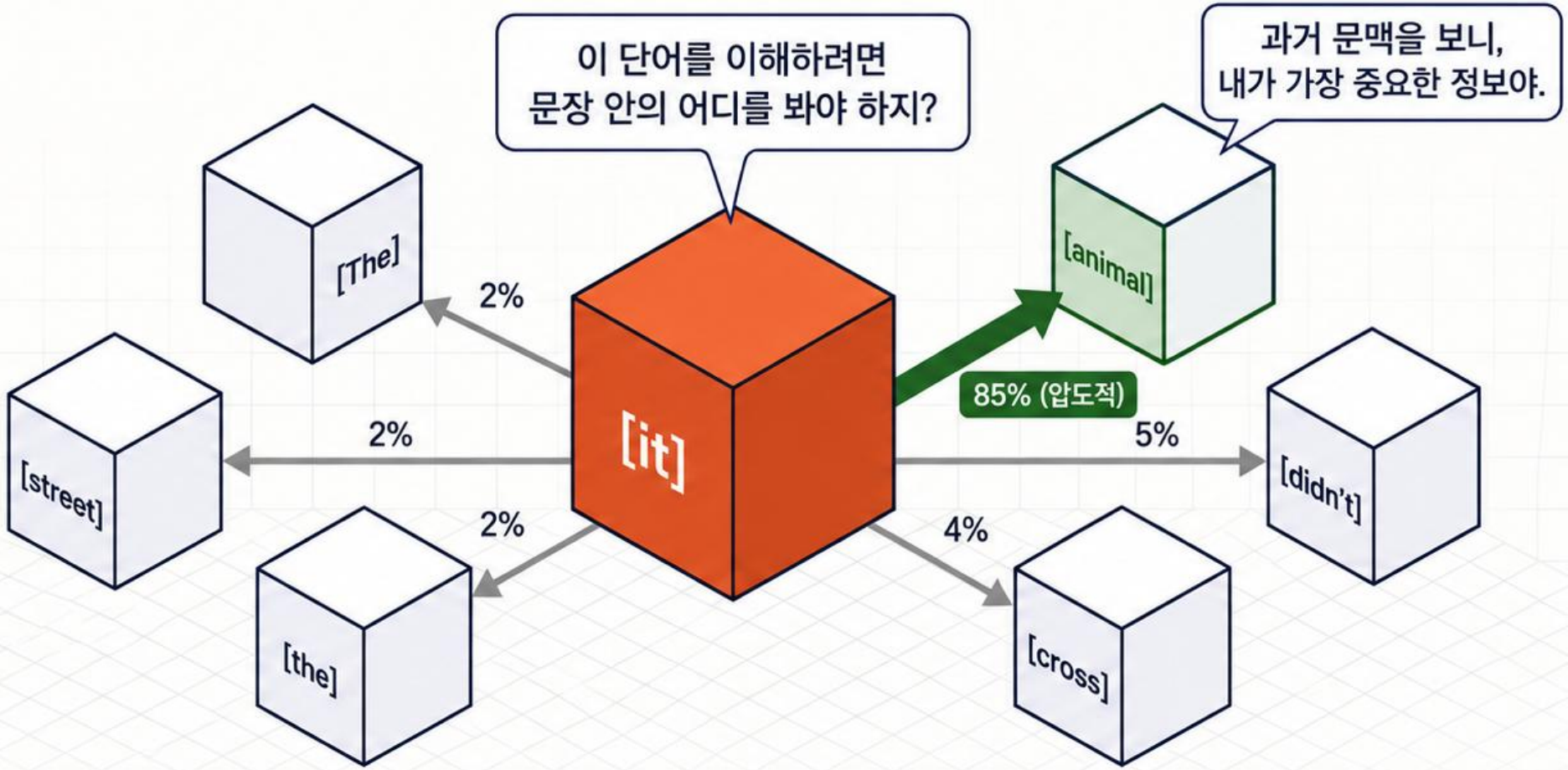
개념

'그것'이 가리키는 것은 길일까요, 동물일까요?
기계에게는 어려운 문제입니다.

메커니즘

셀프 어텐션은 한 단어를 인코딩할 때, 문장 내의
'모든 다른 단어들'을 살펴보고 연관성을 점수화합니다.
고립된 단어 임베딩이 문맥을 이해하는 지능적
임베딩으로 변환되는 순간입니다.

Self-Attention: 의미를 찾는 질문의 연속



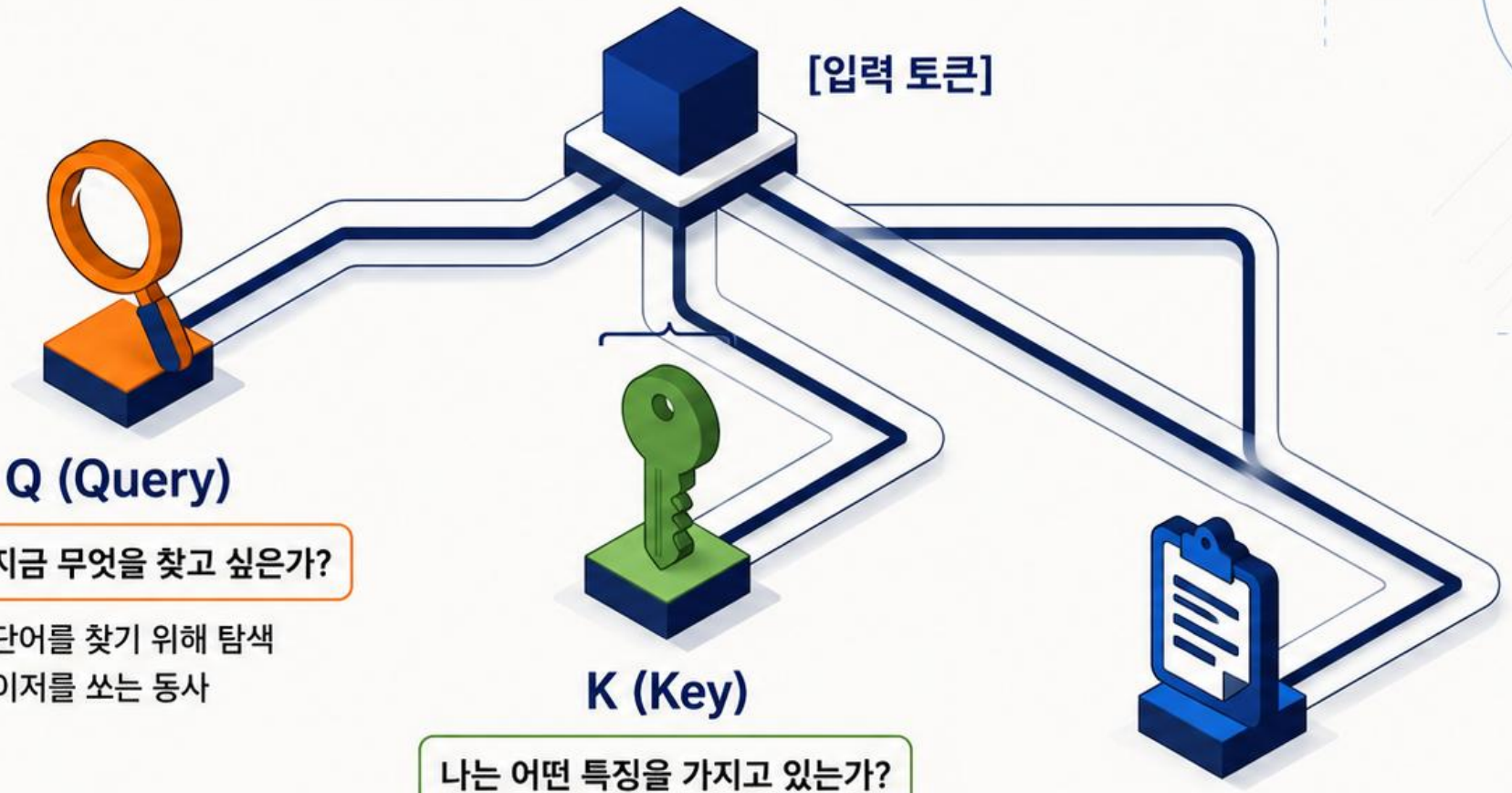
트랜스포머는 주변 단어와의 관계 가중치를 직접 계산하여,
단어가 가진 '**표현(Representation)**'을 지속적으로 갱신합니다.

진화의 완성: Sequence Models 진단 매트릭스

	RNN	LSTM	Transformer
처리 방식 (Processing)	순차적 (직렬) ⚠	순차적 (직렬) ⚠	동시적 (병렬 연산 최적화) ✓
장기 문맥 유지 (Long-range)	매우 취약 (Vanishing Gradient) ⚠	부분적 개선 (Forget Gate) ○	완벽함 (모든 거리가 O(1)) ✓
연산 복잡도 (per Layer)	O(N * d^2) ⚠	O(N * d^2) ○	O(N^2 * d) ✓

핵심 요약: Transformer는 순차적 처리를 포기한 대신, 행렬 곱셈을 통한 압도적인 병렬 처리 속도와 맥락 파악 능력을 얻어냈습니다. (단, 길이에 따른 N^2 연산량 폭발이라는 새로운 과제 탄생)

Q, K, V: 관계를 계산하는 매칭 시스템



Q (Query)

내가 지금 무엇을 찾고 싶은가?

- 예: 단어를 찾기 위해 탐색 레이저를 쏘는 동사

K (Key)

나는 어떤 특징을 가지고 있는가?

- 예: '나 자신이 명사이며 단수형임'을 알리는 해시태그

V (Value)

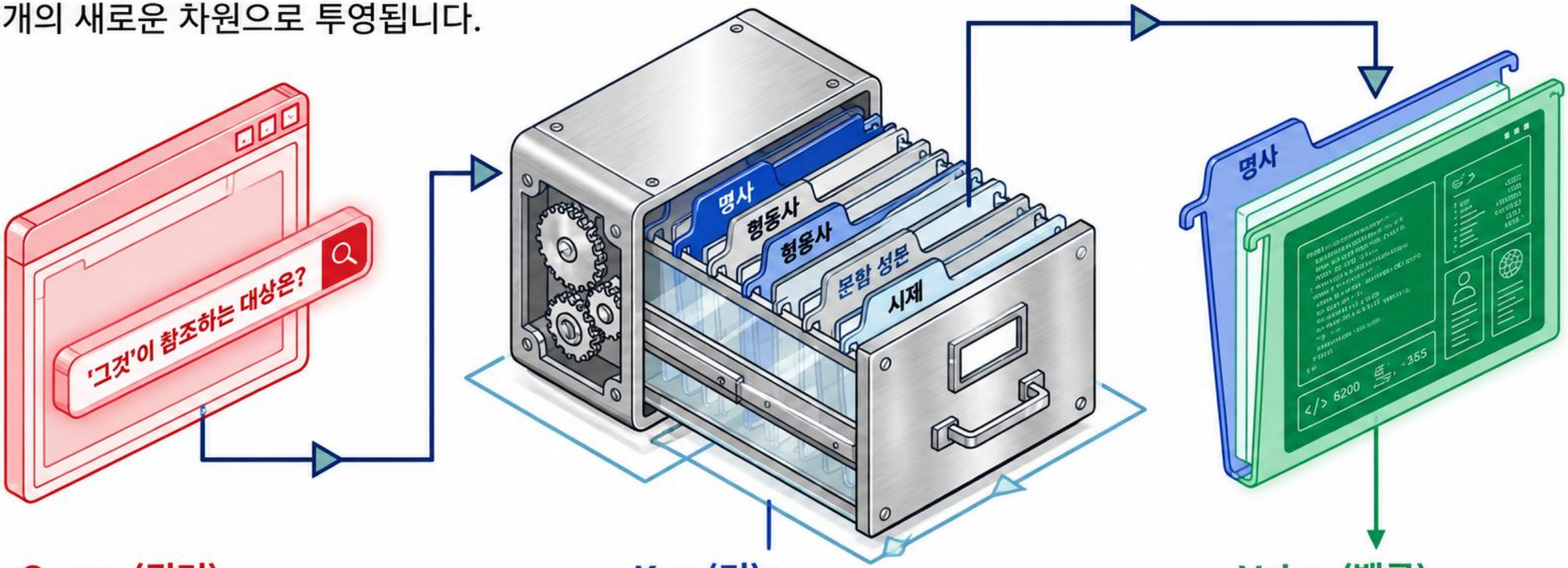
실제로 전달할 핵심 정보는 무엇인가?

- 예: 다음 계산 단계로 넘겨줄 단어의 실제 의미 벡터

하나의 토큰은 선형 변환을 통해 Q, K, V라는
세 가지 독립적인 역할로 분화됩니다.
한 토큰의 Query가 다른 모든 토큰들의 Key를 스캔하여
'누구를 얼마나 참고할지'를 결정하는 것이 핵심입니다.

Q, K, V: 지식 검색 메커니즘

임베딩 벡터는 학습 가능한 가중치 행렬과 곱해져
3개의 새로운 차원으로 투영됩니다.

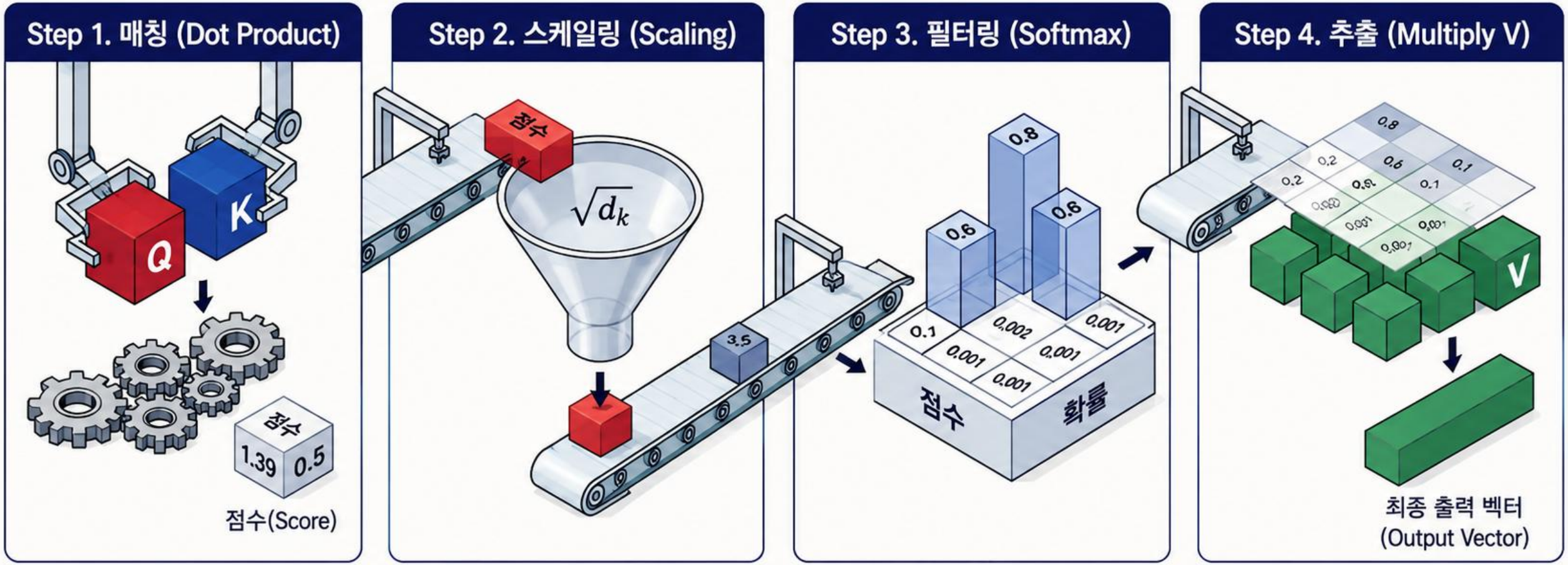


Query (쿼리):
현재 단어가 다른 단어들에게
던지는 질문.

Key (키):
다른 단어들이 가지고 있는
라벨 또는 태그.

Value (밸류):
그 단어가 실제로 담고 있는
의미 값.

셀프 어텐션 조립 라인 (행렬 계산의 시각화)



Step 1. 매칭 (Dot Product)

쿼리(Q)와 키(K)를 내적하여 연관성 점수를 계산합니다.

Step 2. 스케일링 (Scaling)

점수를 키 벡터 차원의 제곱근으로 나누어 기울기를 안정화합니다.

Step 3. 필터링 (Softmax)

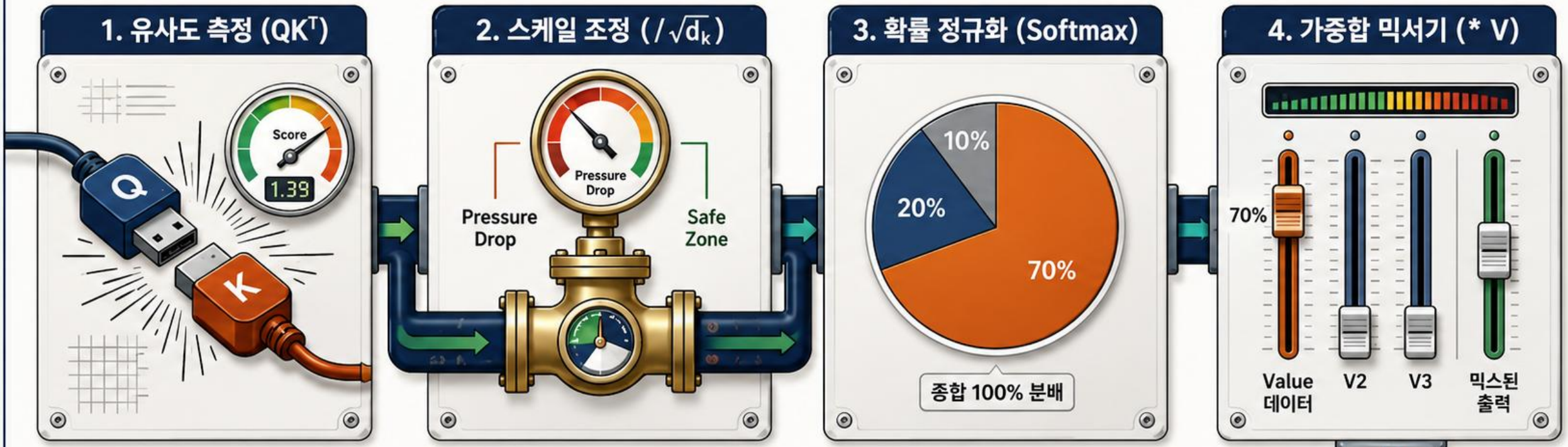
점수를 0~1 사이의 확률값으로 변환합니다 (총합 1), 관련 없는 단어는 0.001처럼 작아집니다.

Step 4. 추출 (Multiply V)

Softmax 점수와 벨류(V)를 곱하여 가중합(Weighted Sum)을 구합니다. 이것이 최종 출력 벡터입니다.

어텐션 대시보드: 수식의 시각화

$$\text{Attention}(Q,K,V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

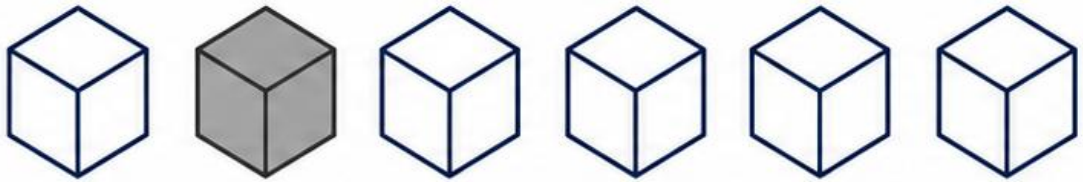


수식의 본질은 결국 '어떤 토큰들을 얼마나 참고할지 확률로 정하고(Softmax),
그 비율만큼 정보를 믹스하여(V) 새로운 문맥 표현을 만든다'는 것입니다.

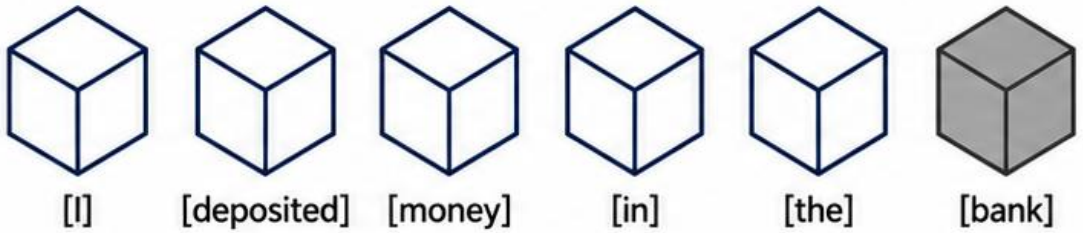
카멜레온 단어: 문맥 임베딩의 마법

과거: 정적 임베딩 시대

The bank is near the river.

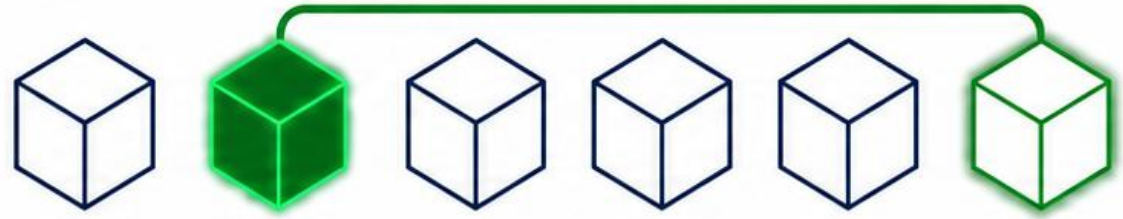


I deposited money in the bank.



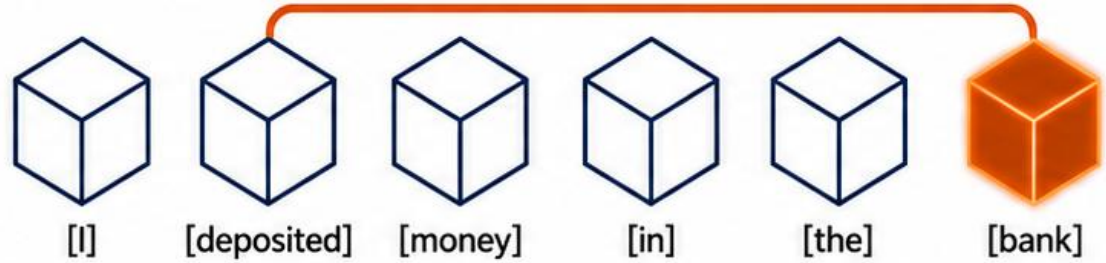
현재: 트랜스포머의 문맥 임베딩

The bank is near the river.



강(河) 벡터로 변환

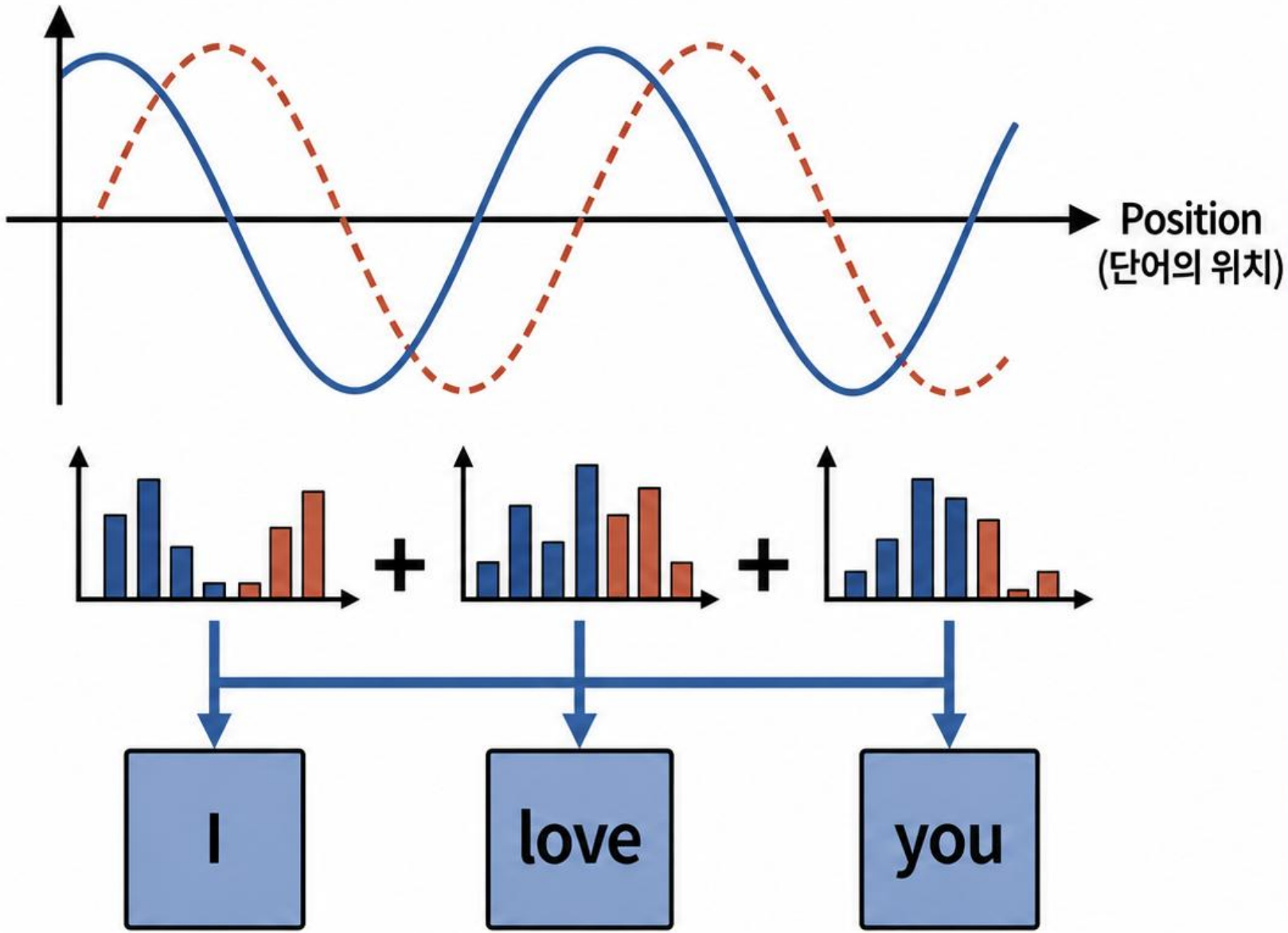
I deposited money in the bank.



금융 기관 벡터로 변환

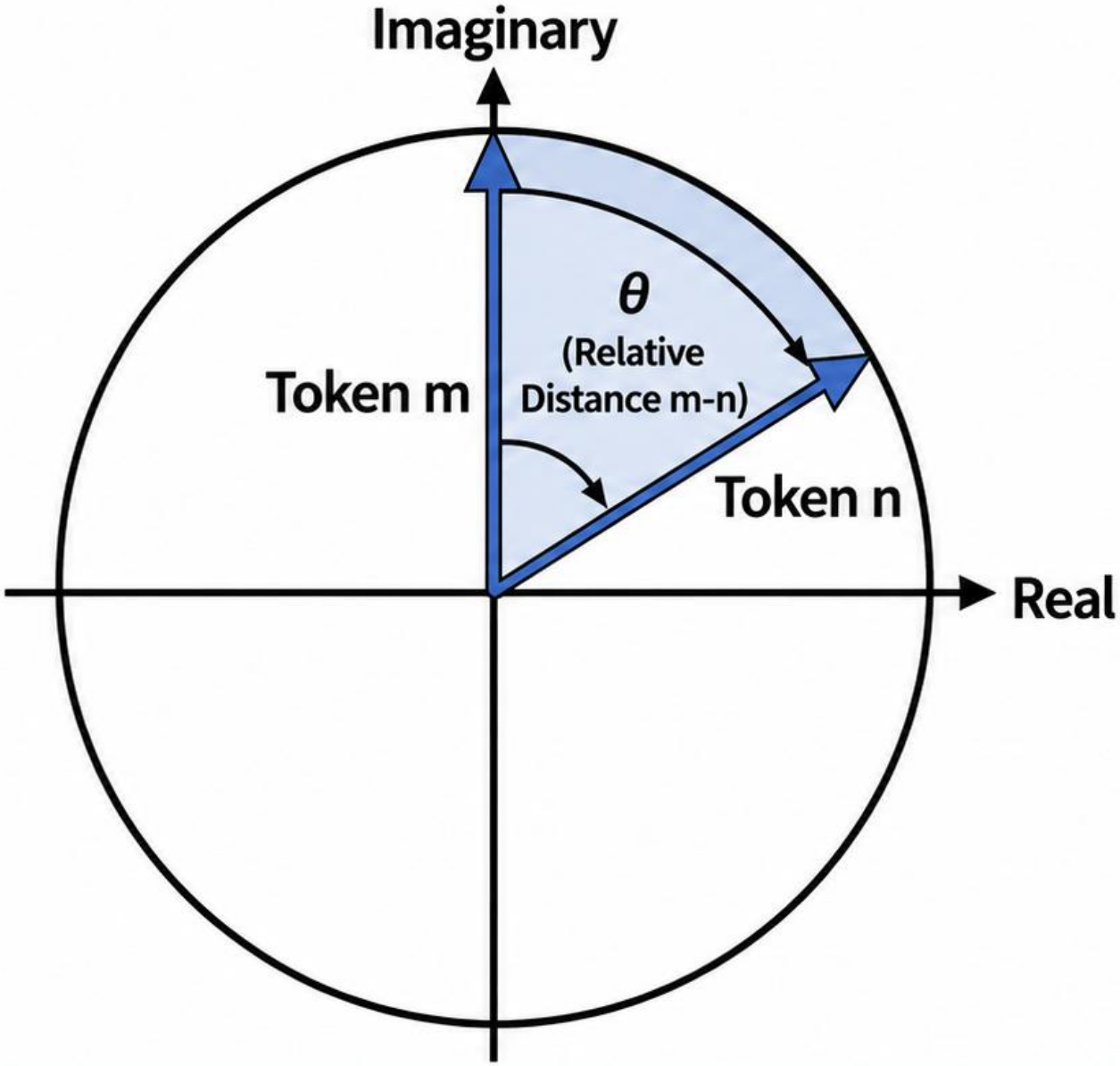
과거에는 단어가 항상 고정된 하나의 벡터를 가졌습니다. 트랜스포머는 **Self-attention**을 통해 주변 단어들의 정보를 흡수하여, 동일한 단어라도 완전히 다른 문맥 의존적 상태로 진화시킵니다.

잃어버린 순서의 복원: Absolute Positional Encoding



- 문제: 트랜스포머는 단어를 동시에 쏟아 붓기 때문에 어순을 스스로 파악하지 못함.
- 해결: 짝수 차원은 사인파, 홀수 차원은 코사인파를 사용하여 각 토큰에 고유한 절대적 위치 좌표를 임베딩 벡터에 직접 더함(Addition).

차세대 위치 인코딩: RoPE (Rotary Positional Encoding)



- 절대 위치를 더하는(Add) 대신, 토큰의 벡터 자체를 위치에 비례하는 각도만큼 회전(Rotation)시킴.
- 오일러 공식을 활용한 수학적 우아함: 내적(Dot-product) 연산 시 절대적 위치 값은 상쇄되고 두 토큰 간의 상대적 거리(m-n)만이 각도의 차이로 남음.
- 최신 LLM(Llama 등)의 표준: 컨텍스트 윈도우가 길어져도 단어 간의 상대적 거리를 정확히 보존.

디코더와 인과적 마스킹 (Causal Masking)

Fog of War Matrix

	I	am	a	student	<eos>
I		-∞	-∞	-∞	-∞	-∞
am			-∞	-∞	-∞	-∞
a				-∞	-∞	-∞
student						-∞
<eos>						

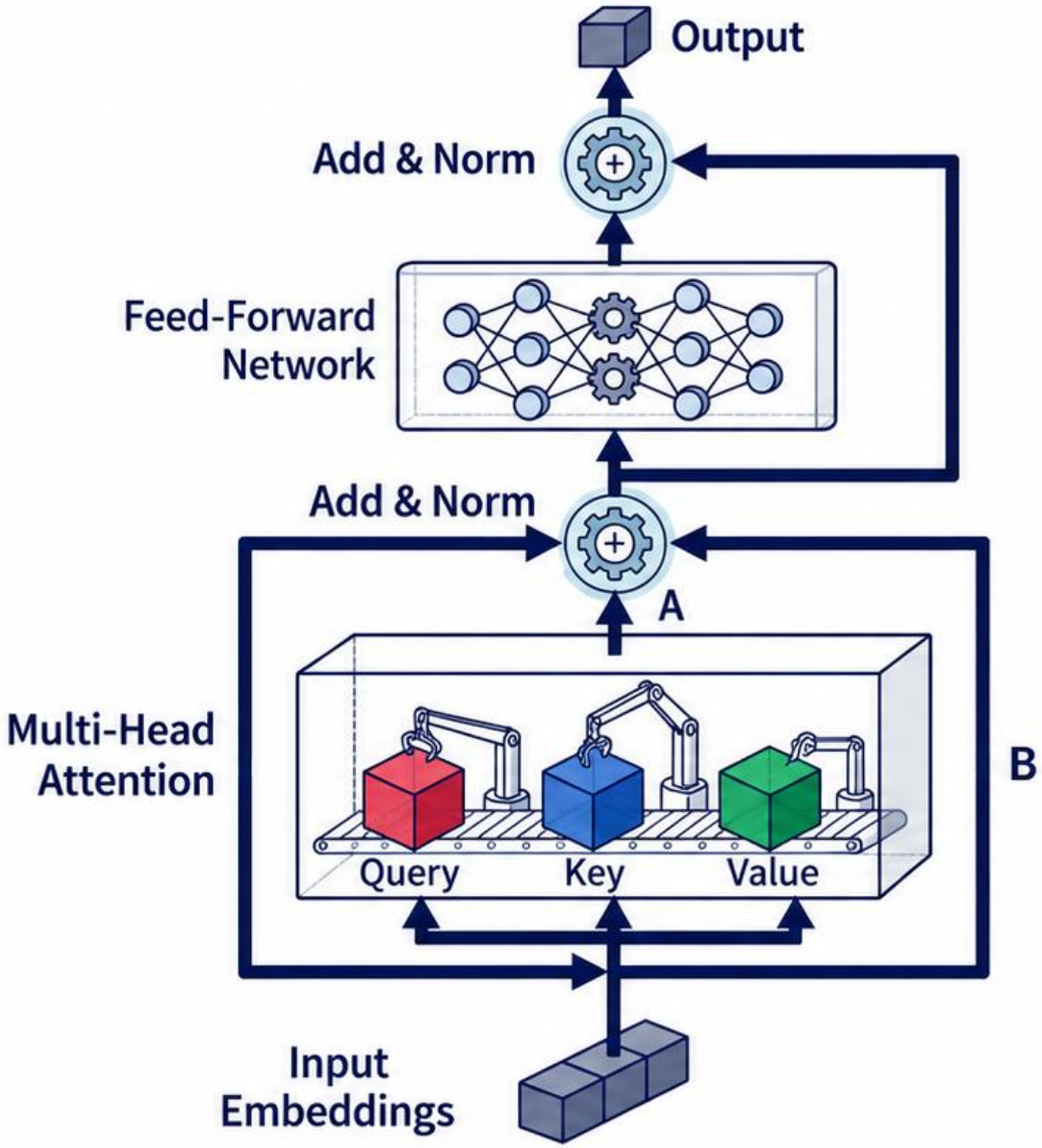
문제 및 해결 (Causal Masking)

학습 시, 모델이 생성해야 할 미래의 정답을 흠쳐보면 안 됩니다. 셀프 어텐션 계산 시 미래의 위치에 마스킹(-∞)을 적용하여 이전 위치의 단어들에만 집중하도록 강제합니다.

인코더-디코더 어텐션

디코더의 두 번째 레이어입니다. 쿼리(Q)는 디코더에서, 키(K)와 밸류(V)는 인코더의 최종 출력에서 가져와 원본 입력 문장의 적절한 위치에 집중합니다.

인코더 블록 완성: 안정적인 깊은 학습을 위하여



잔차 연결 (Residual Connection)

입력 데이터(X)를 서브 레이어의 결과(F(X))에 그대로 더합니다. 이는 정보의 유실을 막고 소실 경사(Vanishing Gradient) 문제를 해결합니다.

레이어 정규화 (Layer Normalization)

활성화 값을 정규화하여 학습 과정을 크게 안정화하고 수렴 속도를 높입니다.

Feed-Forward Network (FFN)

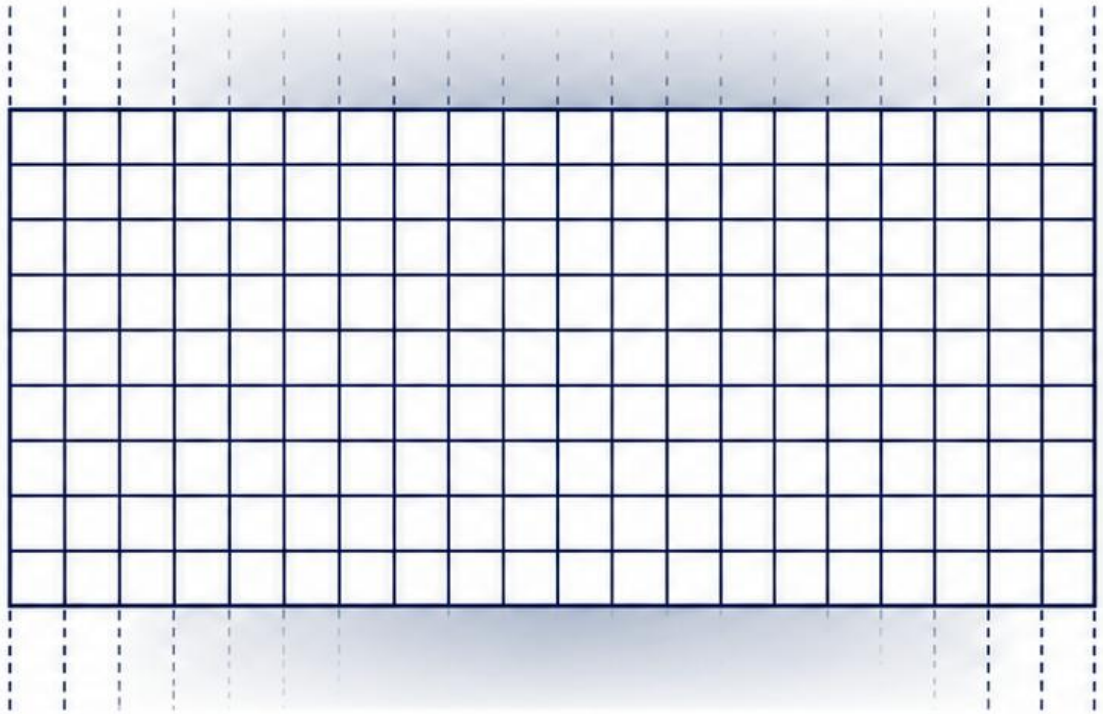
어텐션으로 모인 정보를 각 단어별로 독립적으로 처리하는 2계층 다층 퍼셉트론입니다.

출력 생성: 벡터에서 단어로

디코더
출력 벡터

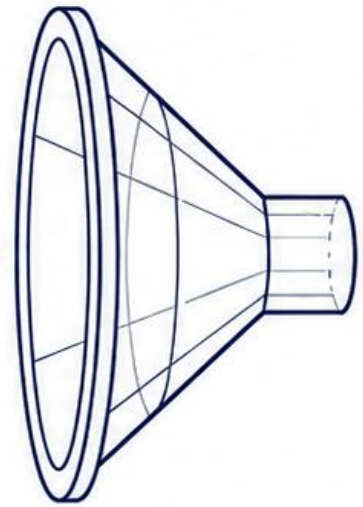


Linear Layer (Vocabulary Size: 10,000+)



소수점 벡터를 모델이 아는 모든 단어의 수만큼 투영(Projection)합니다.

Softmax Layer



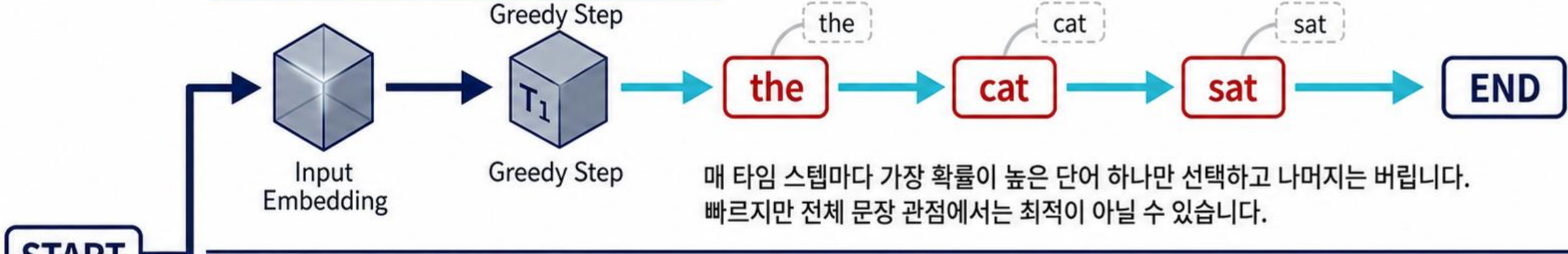
Logits 점수들을 총합 1.00이 되는 확률 분포로 변환합니다.



가장 높은 확률값을 가진 단어가 최종 출력됩니다.

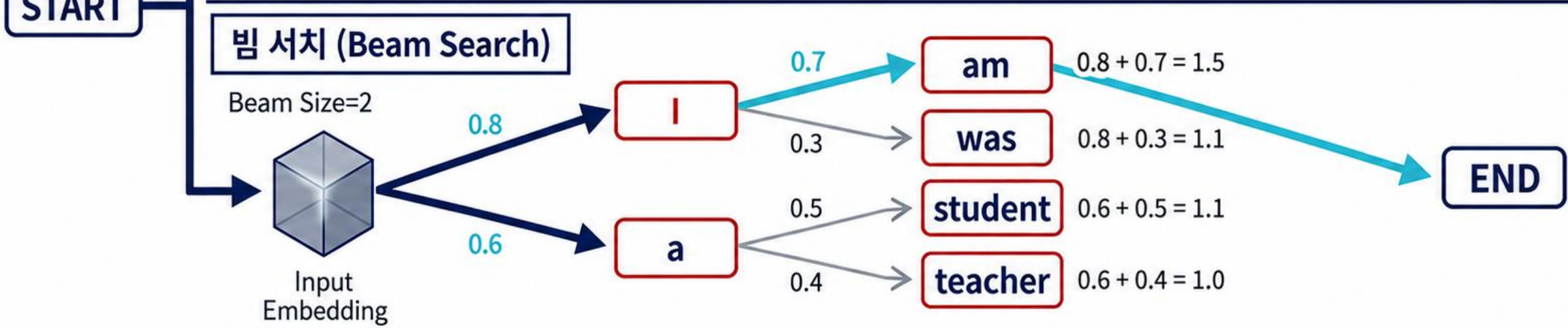
단어 선택 전략: 최적의 경로 찾기

탐욕적 디코딩 (Greedy Decoding)



매 타임 스텝마다 가장 확률이 높은 단어 하나만 선택하고 나머지는 버립니다.
빠르지만 전체 문장 관점에서는 최적이 아닐 수 있습니다.

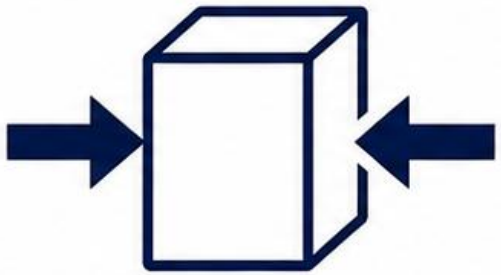
빔 서치 (Beam Search)



확률이 높은 N개의 단어를 유지하며 여러 갈래의 시나리오를 동시에 계산합니다.
누적 에러가 가장 적은 최적의 문장 경로를 최종적으로 선택합니다.

트랜스포머 진화 계통도: 하나의 구조, 무한한 확장

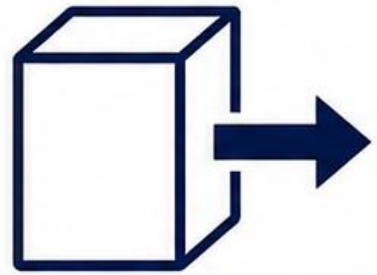
인코더 전용 (Encoder-Only)



- 양방향(Bidirectional) 문맥 파악.
- 마스킹된 빈칸 채우기로 학습.
- 문장 분류, 의미 분석에 탁월.

[대표 모델: BERT, RoBERTa]

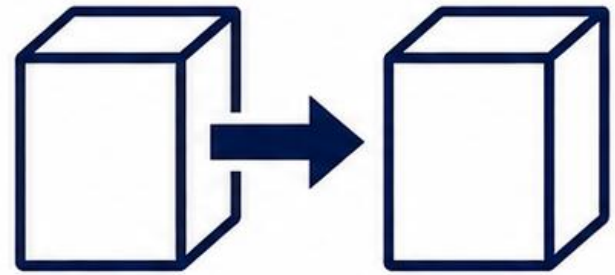
디코더 전용 (Decoder-Only)



- 단방향(Causal) 자기회귀.
- 이전 단어들로 다음 단어를 예측하며 학습.
- 텍스트 생성, 프롬프트 지시 수행에 압도적.

[대표 모델: GPT 시리즈, LLaMA, Claude]

인코더-디코더 (Encoder-Decoder)



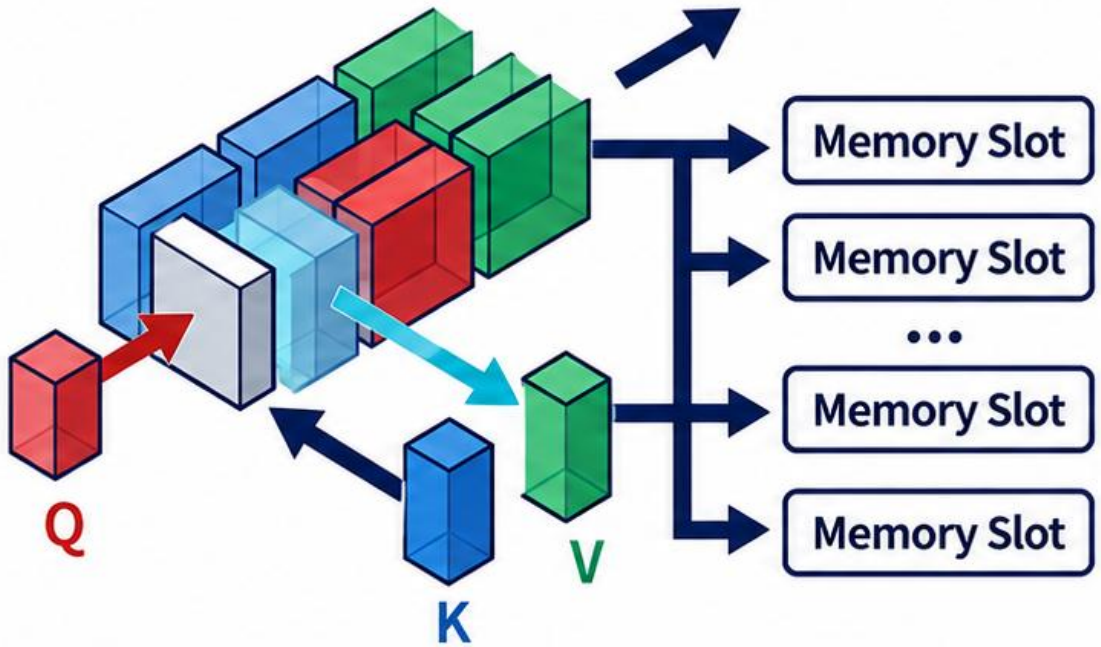
- Sequence-to-Sequence 구조.
- 입력 시퀀스를 완전히 새로운 시퀀스로 변환.
- 번역, 문서 요약에 최적화.

[대표 모델: T5, BART]

아키텍처의 한계 극복과 현대화

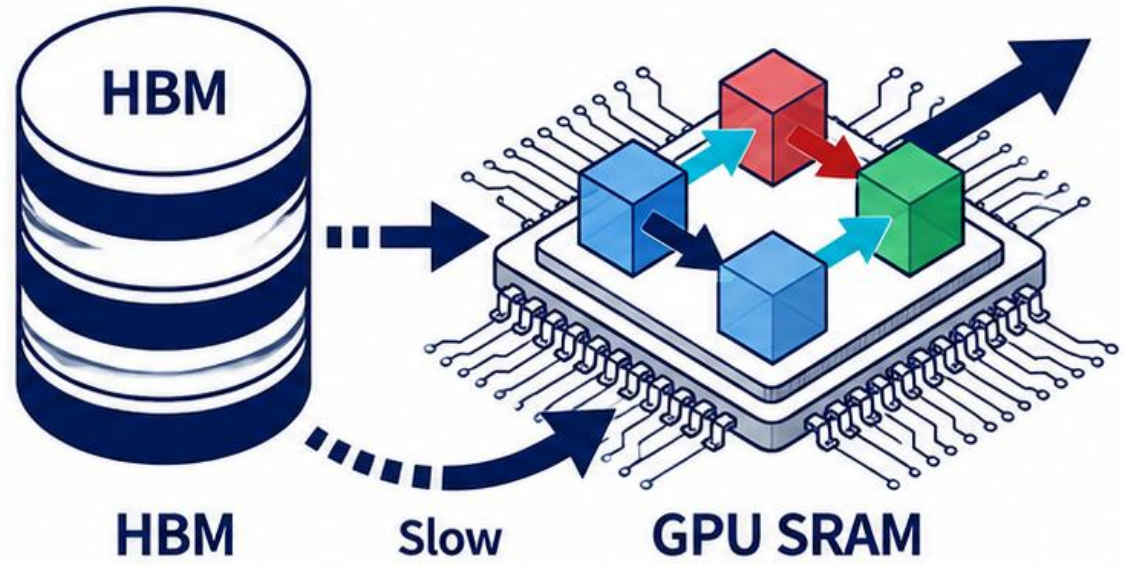
어텐션 연산은 $O(N^2)$으로 계산량과 메모리가 폭증합니다. 현대 LLM은 이를 하드웨어 최적화로 해결합니다.

KV Caching



자기회귀 생성 시, 이전 타임스텝의 Key와 Value를
캐시에 저장하여 불필요한 중복 계산을 방지.
PagedAttention 메모리 페이징 기술 적용.

FlashAttention



GPU 캐시(SRAM) 간의 데이터 이동(IO)을
최소화하는 하드웨어 인식 알고리즘.
어텐션 계산 속도를 획기적으로 가속.

Attention Is All You Need: 모든 모달리티를 정복하다



Text (언어 모델)
텍스트 이해 및 생성 (LLMs)



Audio (음성)
스펙트로그램 토큰화
(Whisper, Conformer)

Multi-Head Attention

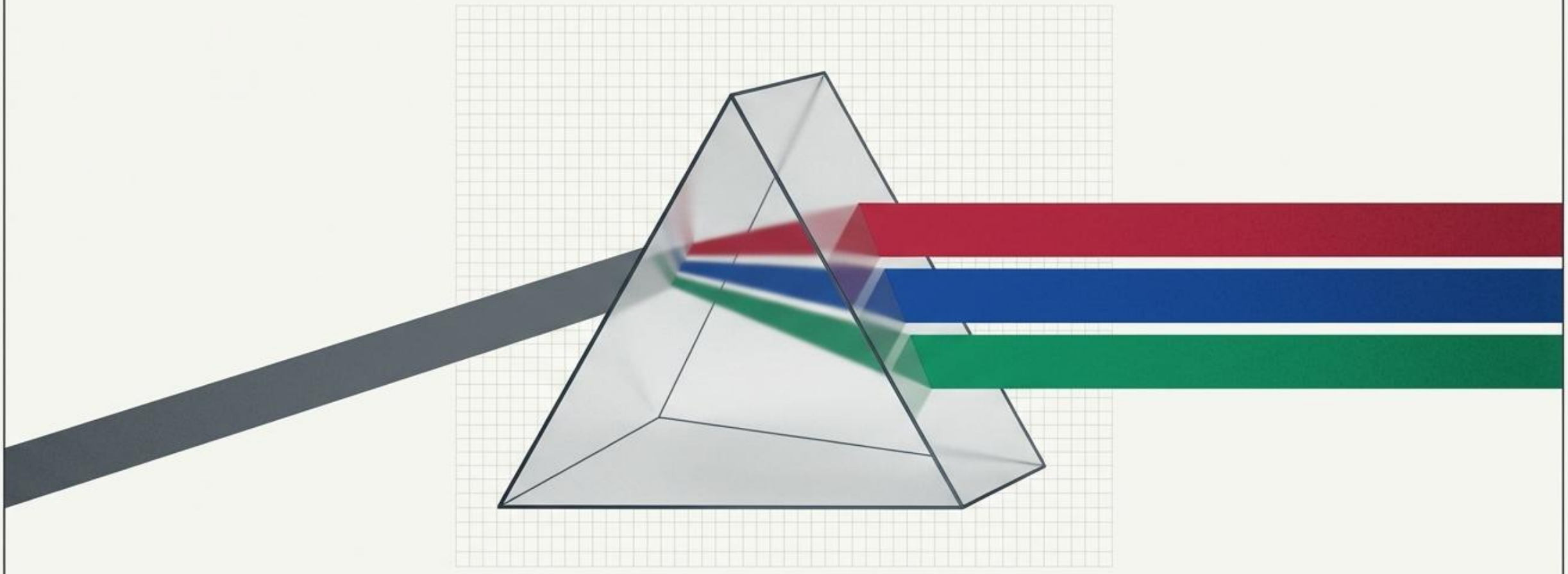


Image (비전)
패치 토큰화를 통한 이미지
인식/생성 (ViT, Diffusion)



Actions (로봇 공학)
의사결정 및 로봇 제어
(Decision Transformers)

2017년 기계 번역을 위해 고안된 이 '블랙박스'는 데이터 간의 '관계(Attention)'를 스스로 학습하는 우아한 설계를 바탕으로,
현대 인공지능의 표준 설계도(Blueprint)로 자리 잡았습니다.



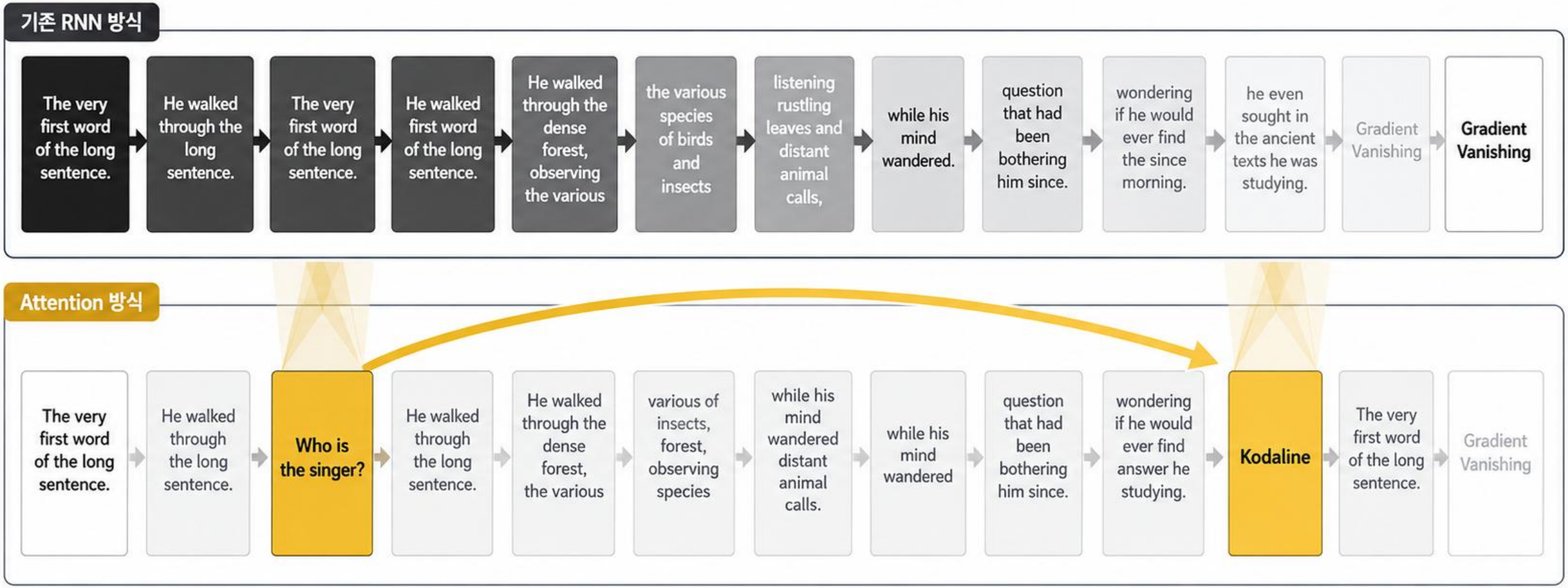
트랜스포머 파헤치기: Multi-Head Attention

문맥을 이해하는 인공지능의 핵심 메커니즘 시각화 가이드

NotebookLM

텍스트가 길어질수록 희미해지는 기억

기존 모델은 문장이 길어질수록 초기 정보가 소실되는 문제가 발생합니다.
Attention 메커니즘은 문장 전체를 보지 않고, 정답을 찾는 데 가장 중요한 단어에만 소프트라이트(가중치)를 비추어 이 문제를 해결합니다.

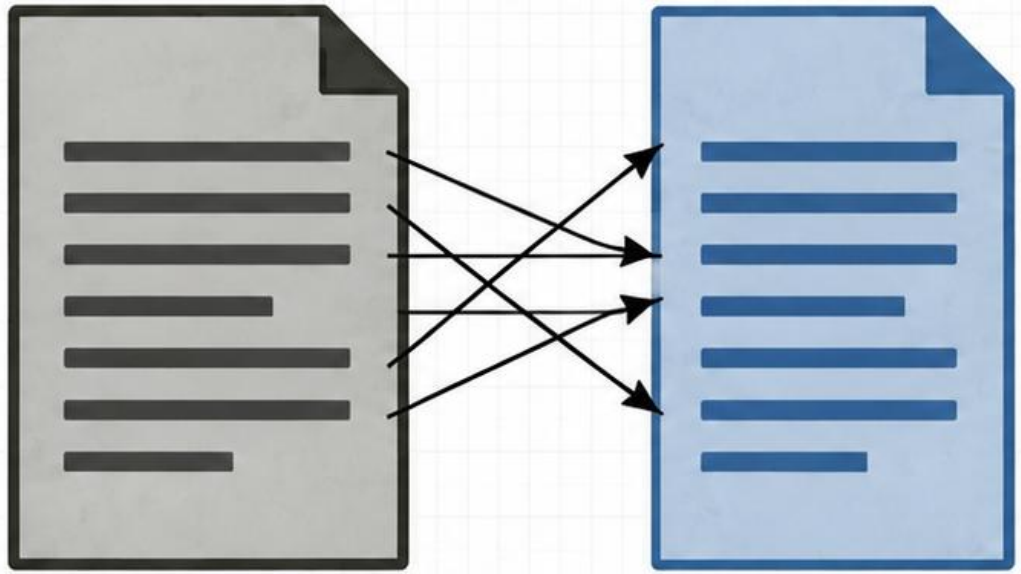


- 어두울수록 초기 정보가 소실됨 (Gradient Vanishing)
- 빈칸은 정답 찾기에 덜 중요한 단어 (낮은 가중치)
- 강조된 단어는 정답과 관련된 핵심 단어 (높은 가중치)
- Attention이 핵심 단어들 간에 직접 연결되어 정보를 유지

Attention vs. Self-Attention

Standard Attention

Source-to-Target 매핑

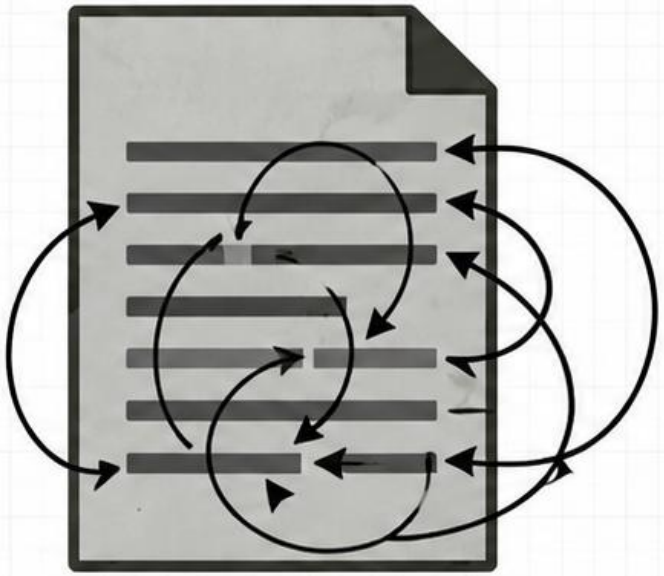


기계 번역 (영어 입력 -> 프랑스어 출력)

입력 시퀀스의 단어들과 출력 시퀀스의
단어들 사이의 연관성을 계산.

Self-Attention

Intra-sentence 매핑 (Encoder-to-Encoder)



문장 내 문맥 파악

동일한 문장 내 단어들 간의 관계와
유사도를 고려하여 어텐션을 계산.

문맥이 의미를 결정한다

단어의 의미는 고정되어 있지 않습니다.

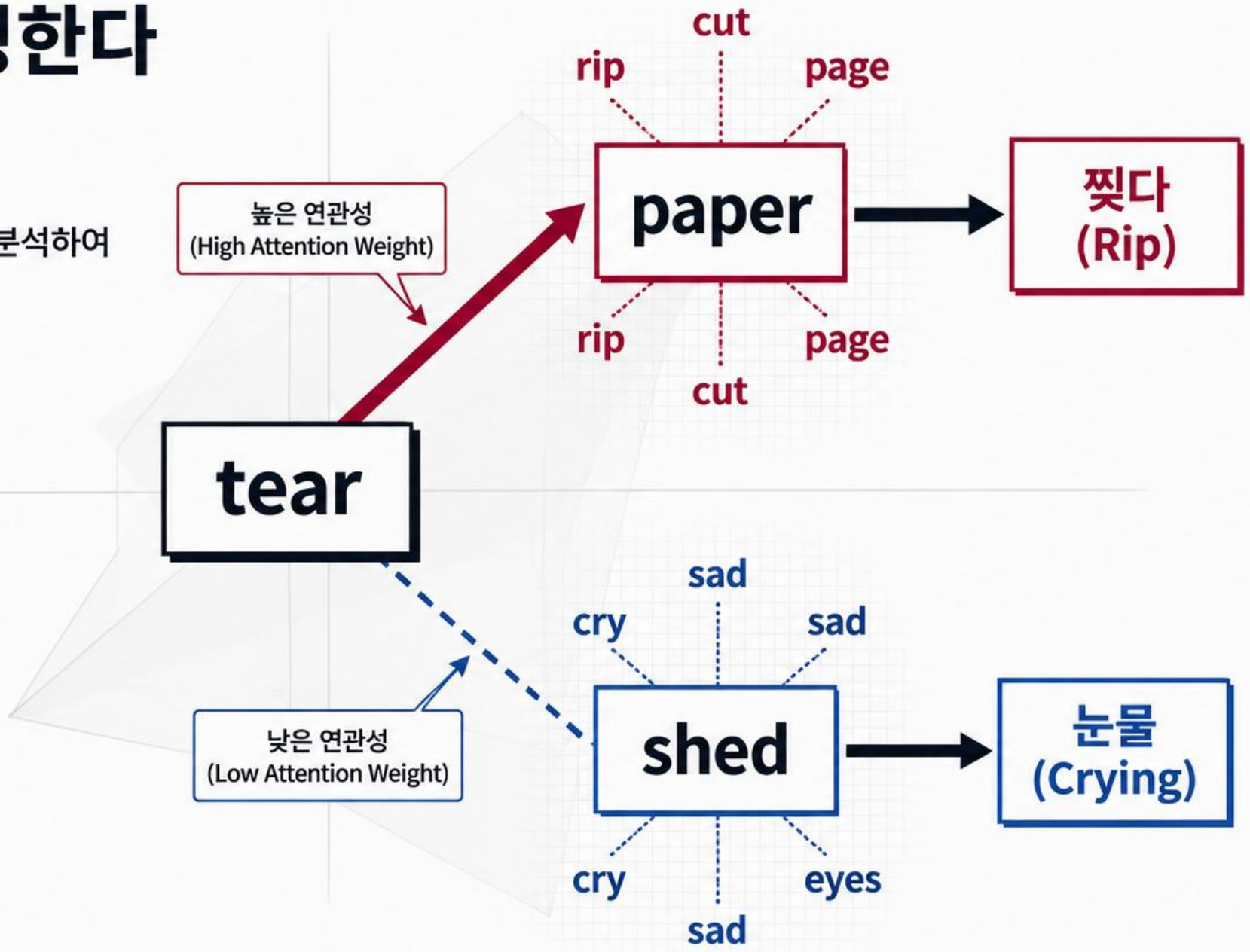
Self-Attention은 주변 토큰들과의 관계를 분석하여

현재 단어의 **'진짜 의미'**를 추론합니다.

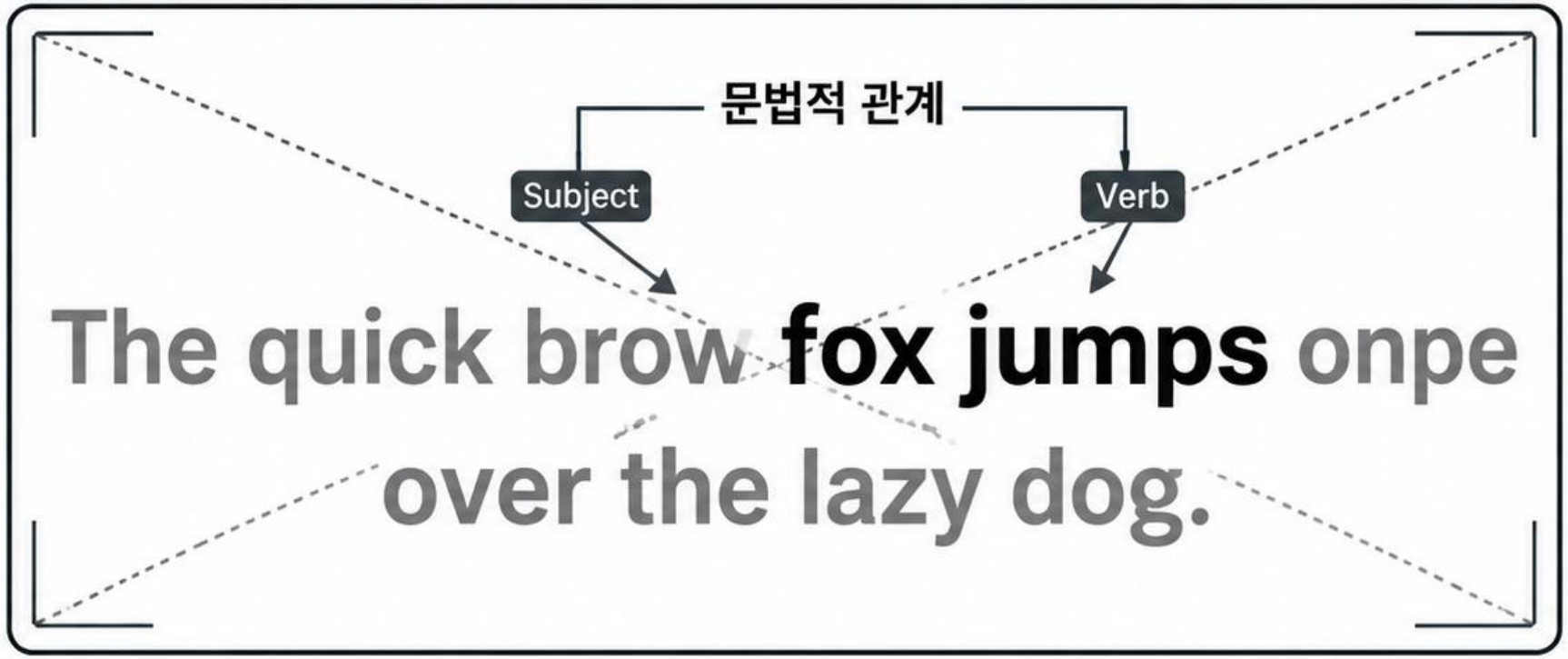
주변에 어떤 단어가 오느냐에 따라

연관성 가중치가 달라집니다.

단어의 되는 처줍한다.



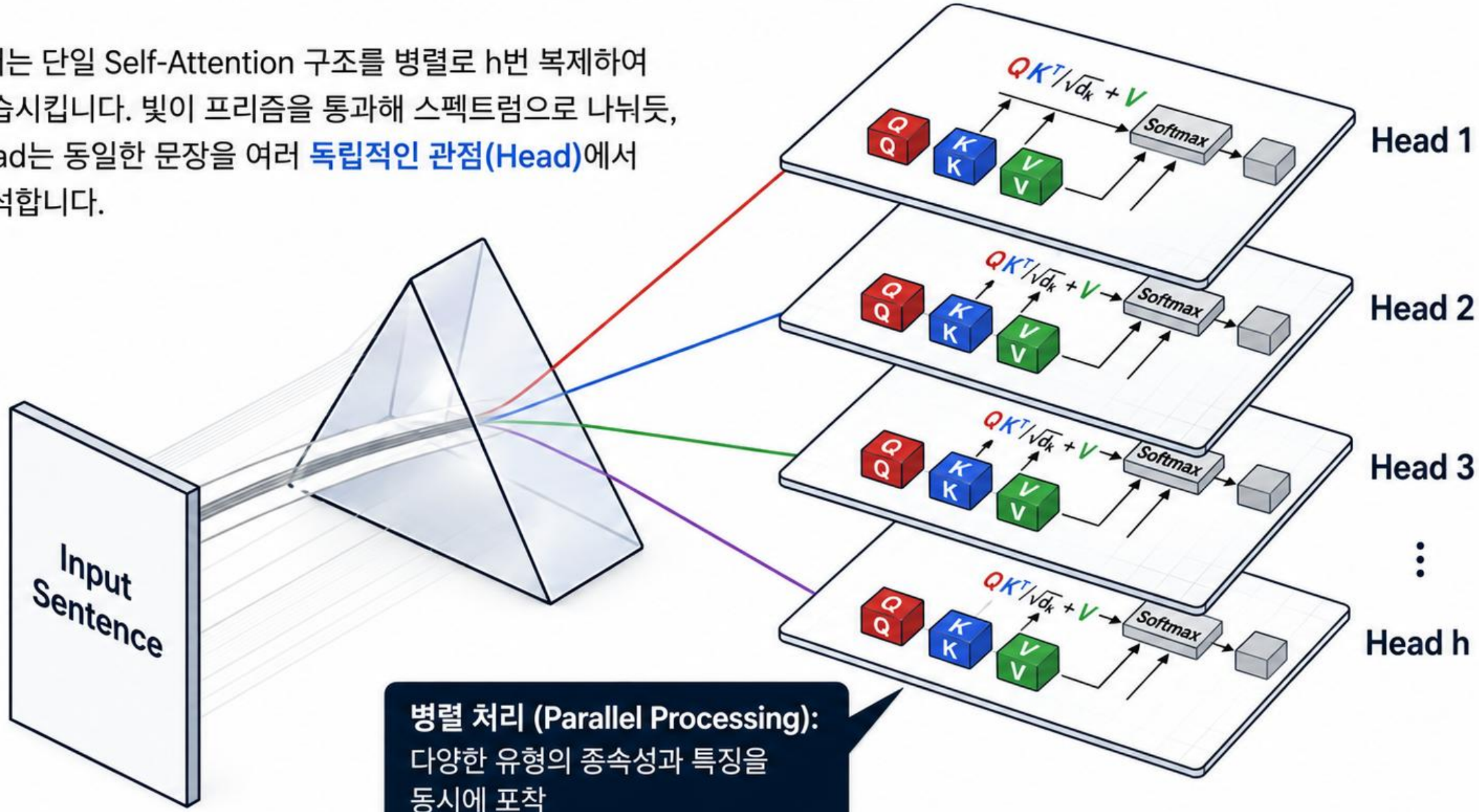
단일 Attention의 한계: 왜 멀티헤드인가?



지금까지 살펴본 단일 Self-Attention은 한 번의 연산으로
문장 내의 특정한 **'한 가지 관계성'**에만 편향될 위험이 있습니다.
문맥, 감정, 뒤앙스 등 언어의 다층적인 복잡성을
단 하나의 **관점(Head)**으로 모두 포착하는 것은 수학적으로 불가능합니다.

Multi-Head Attention: 인지적 프리즘

트랜스포머는 단일 Self-Attention 구조를 병렬로 h번 복제하여 동시에 학습시킵니다. 빛이 프리즘을 통과해 스펙트럼으로 나눠듯, Multi-Head는 동일한 문장을 여러 **독립적인 관점(Head)**에서 동시에 분석합니다.



병렬 처리 (Parallel Processing):
다양한 유형의 종속성과 특징을 동시에 포착

다각도에서 바라보는 문맥의 층위

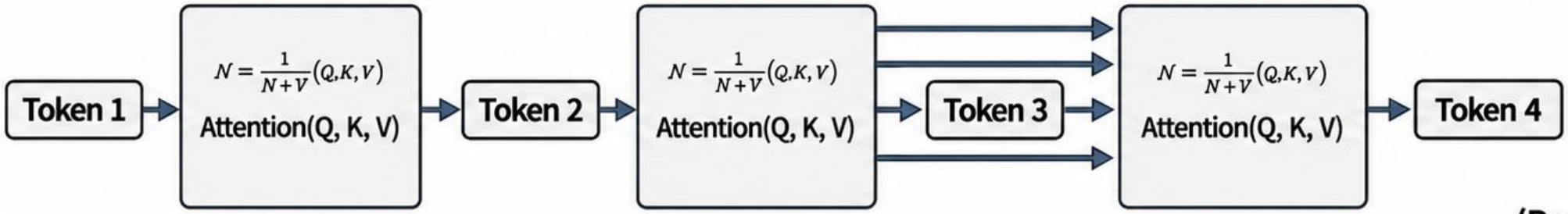
각 Head는 입력 시퀀스의 서로 다른 부분에 집중하여 복잡하고 미묘한 뉘앙스를 포착합니다.

Head 1: **문장 타입 분석**	**Which do you like better, coffee or tea?**
Head 2: **명사 개체 분석**	**Which do you like better, coffee or tea?**
Head 3: **관계성 분석**	**Which do you like better, coffee or tea?**

문장의 구조/타입을 파악 주요 명사(개체)에 집중 명사 간 관계와 선호 비교에 집중

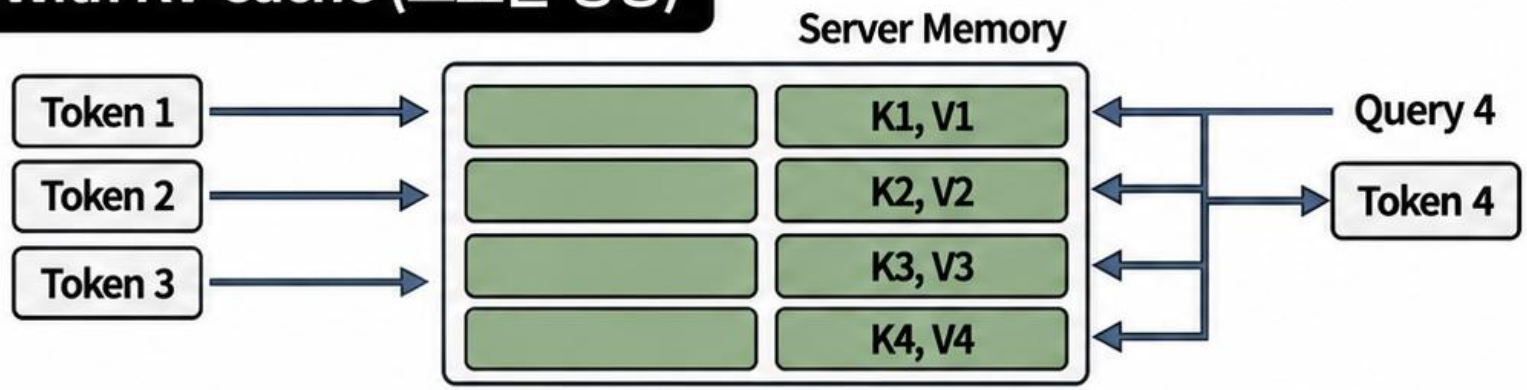
연산 병목의 해결 (1): KV Caching 전략

Without Cache (비효율적 생성)



반복적인 재계산
(Redundant Recalculation)

With KV Cache (고효율 생성)

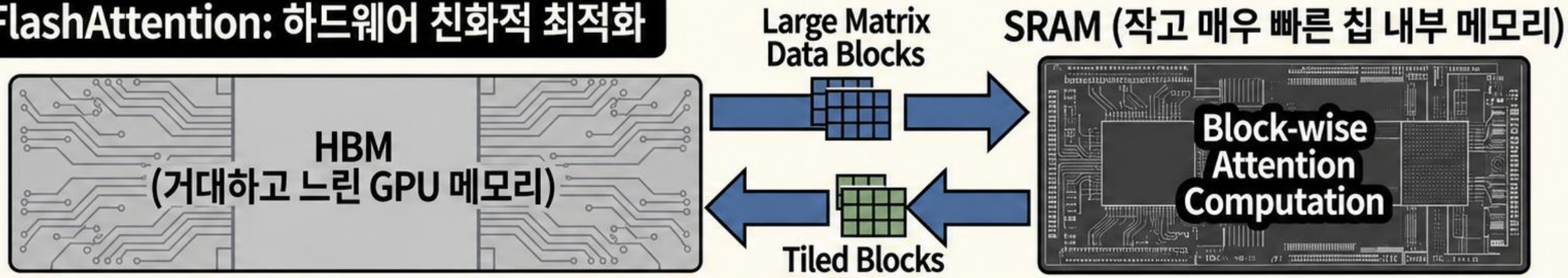


캐시 활용으로 재계산 생략
(Skipped Recalculation with Cache)

PagedAttention: 최신 추론 서버(vLLM 등)는 OS의 가상 메모리 페이징 기법을 도입하여 KV 캐시의 메모리 단편화를 해결하고 동시 처리량을 극대화함.

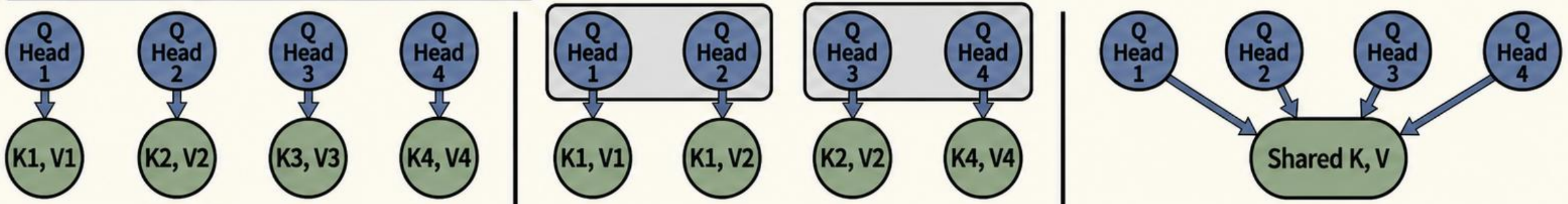
연산 병목의 해결 (2): FlashAttention & MQA

FlashAttention: 하드웨어 친화적 최적화



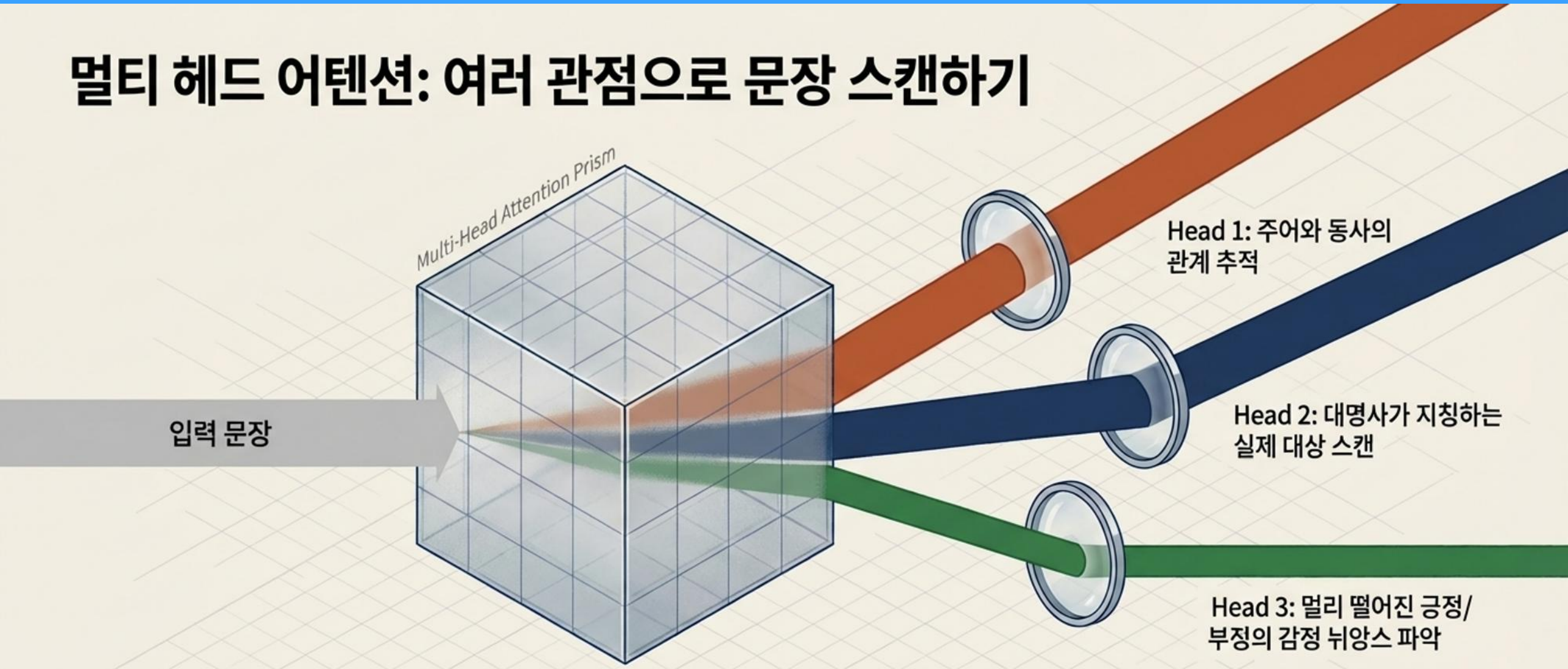
어텐션 행렬을 잘게 쪼개어(Tiling), 느린 HBM 접근을 최소화하고 빠른 SRAM 내에서 모든 연산을 끝냄.

MQA & GQA (KV 파라미터 공유)



수많은 Query Head가 Key와 Value 가중치를 공유하여, 추론 시 메모리 용량과 대역폭 병목을 획기적으로 줄임.

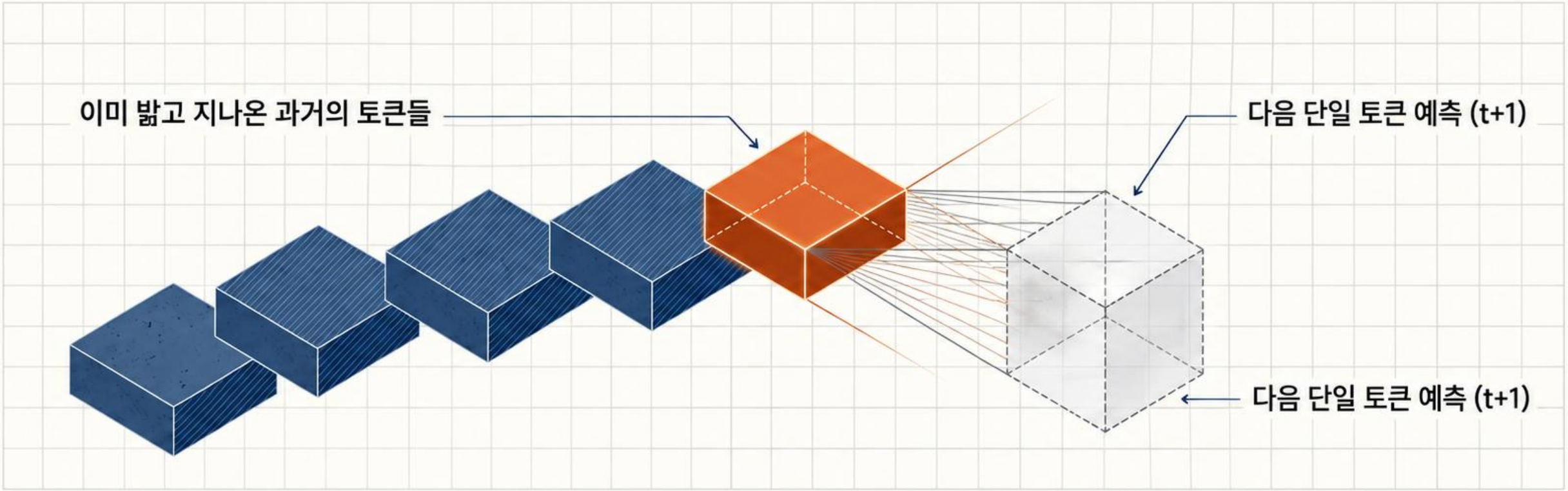
멀티 헤드 어텐션: 여러 관점으로 문장 스캔하기



하나의 Attention만 사용하면 한 종류의 관계에만 편향될 수 있습니다.
트랜스포머는 여러 개의 Head를 병렬로 배치하여, 하나의 문장을
문법, 지시어, 감정 등 다양한 관점에서 동시에 다각도로 관찰합니다.

GPT의 본질: 한 걸음씩 미래를 짓는 자기회귀

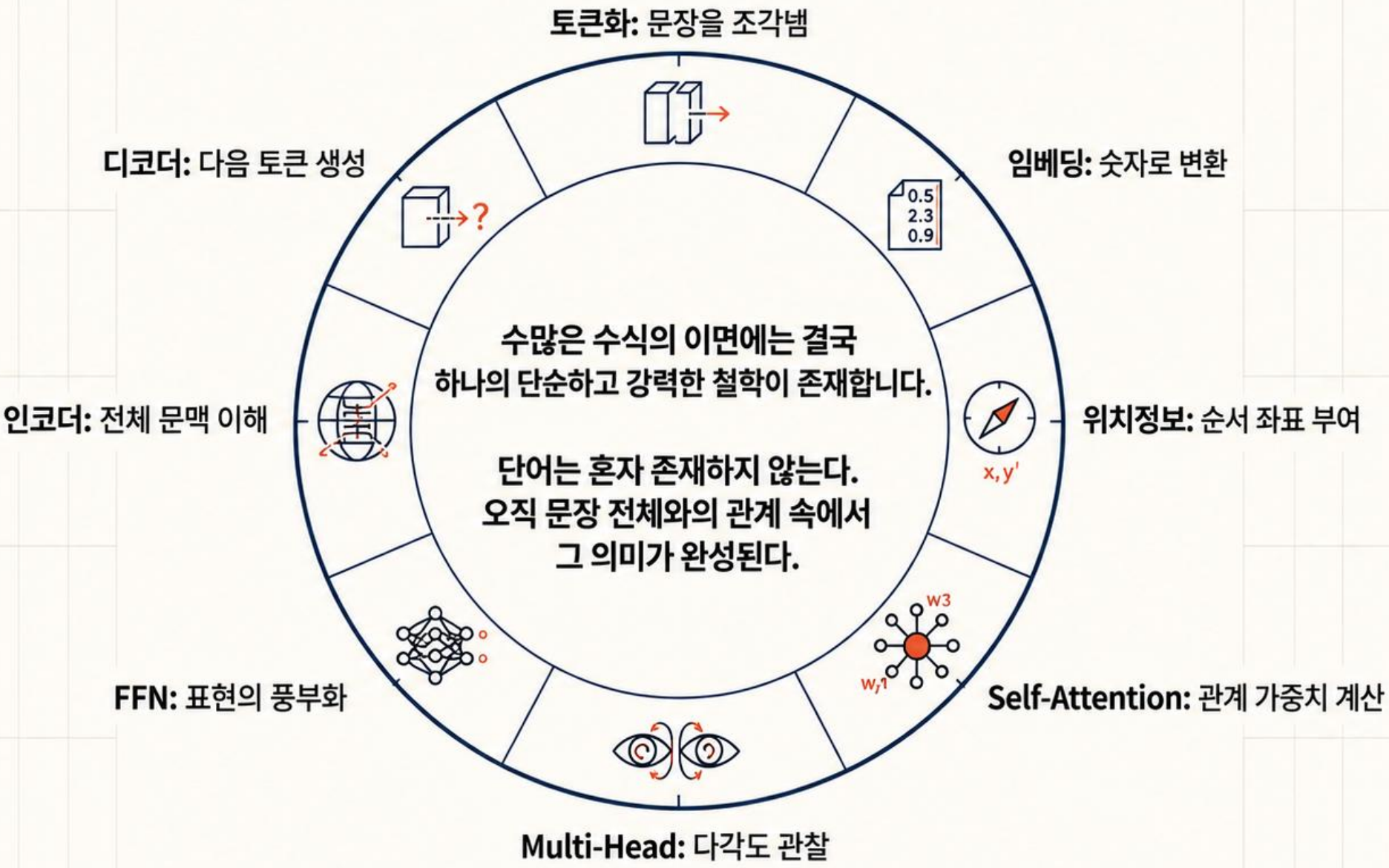
$$P(x_t \mid x_{<t})$$



- **GPT(Autoregressive Model)**는 오직 앞서 동장한 과거의 토큰들만으로 다음 '단 하나의 토큰'을 예측하는 훈련에 모든 구조적 자원을 쏟아붓습니다.

- 이 단순해 보이는 '다음 단어 맞추기'가 아주 깊고 거대한 신경망 안에서 누적될 때, 기계는 인간의 추론 및 논리 전개와 놀랍도록 유사한 창발성을 띠게 됩니다.

요약: 트랜스포머라는 패러다임

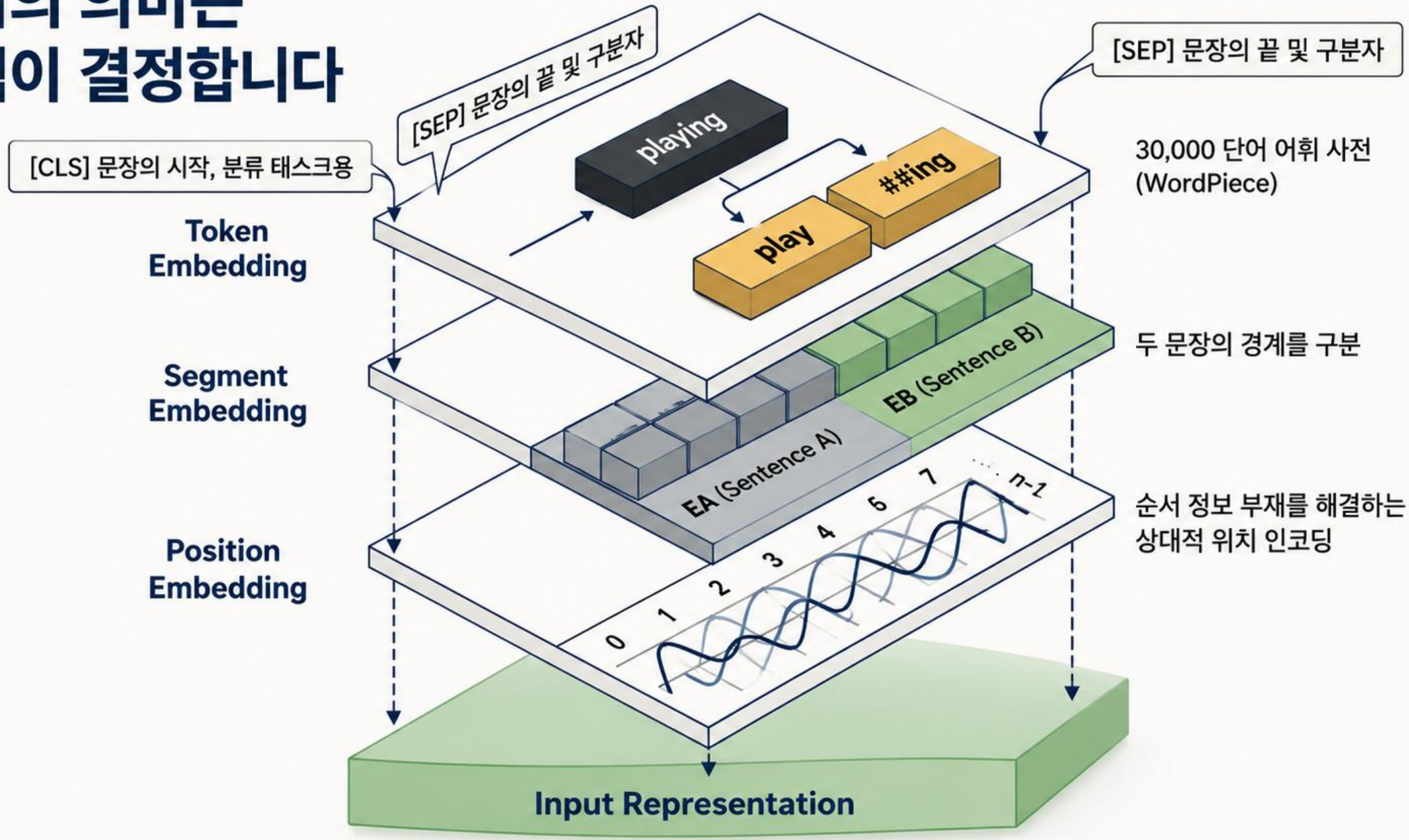


단어의 의미는 문맥이 결정합니다



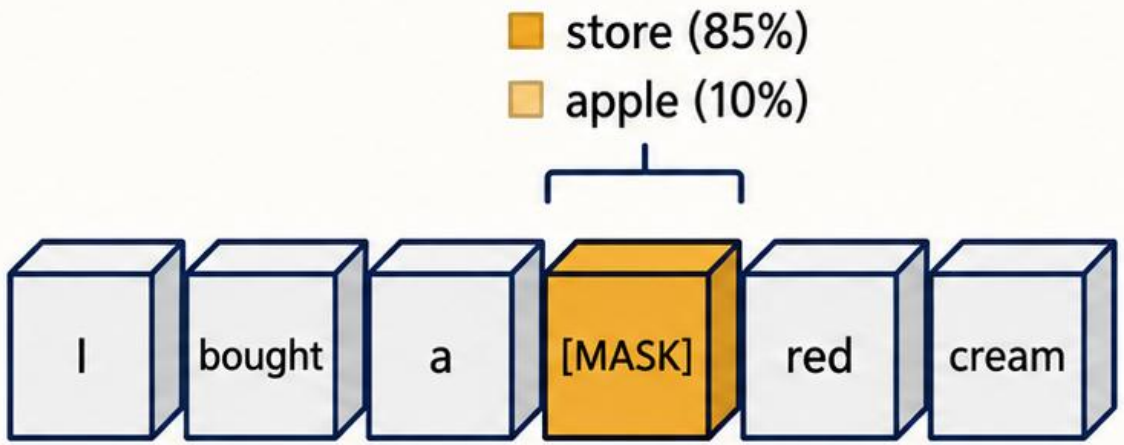
- Self-Attention 메커니즘: 단독으로는 '글'인지 '문자 전송'인지 알 수 없는 "Text"라는 단어가, 주변 단어들과의 양방향 정보 교환을 통해 **'문자 메시지'**라는 정확한 의미로 잠금(**Lock-in**)됩니다.
- 이것이 **BERT**가 일방향 모델을 압도한 결정적 이유입니다.

단어의 의미는 문맥이 결정합니다



자기 지도 학습 (Self-Supervised Learning) 메커니즘

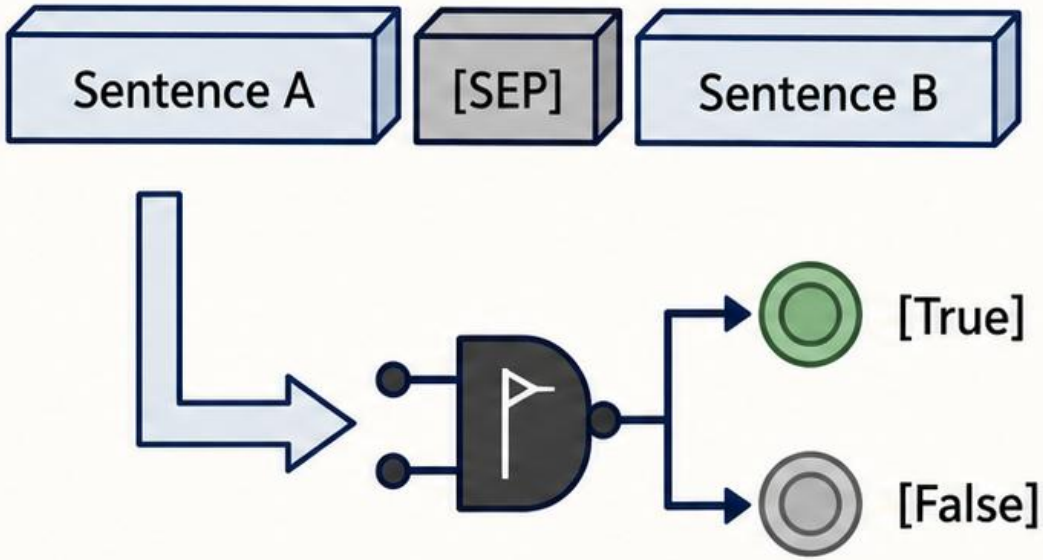
MLM (Masked Language Model)



- 80%: [MASK] 토큰으로 대체
- 10%: 무작위 단어로 치환
- 10%: 원본 단어 유지

학습과 실제 추론 간의 괴리 방지

NSP (Next Sentence Prediction)

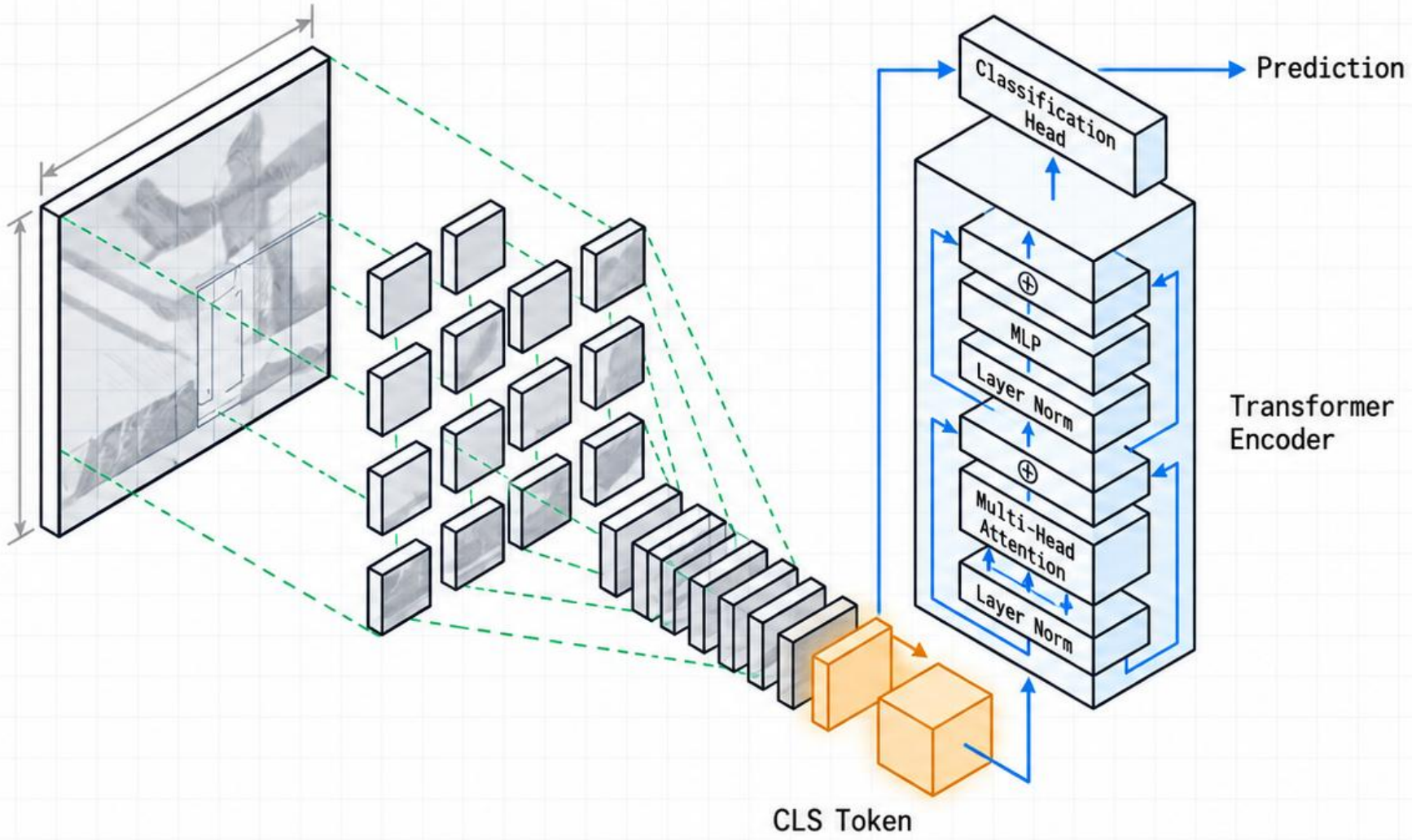


문장 간의 관계성 학습

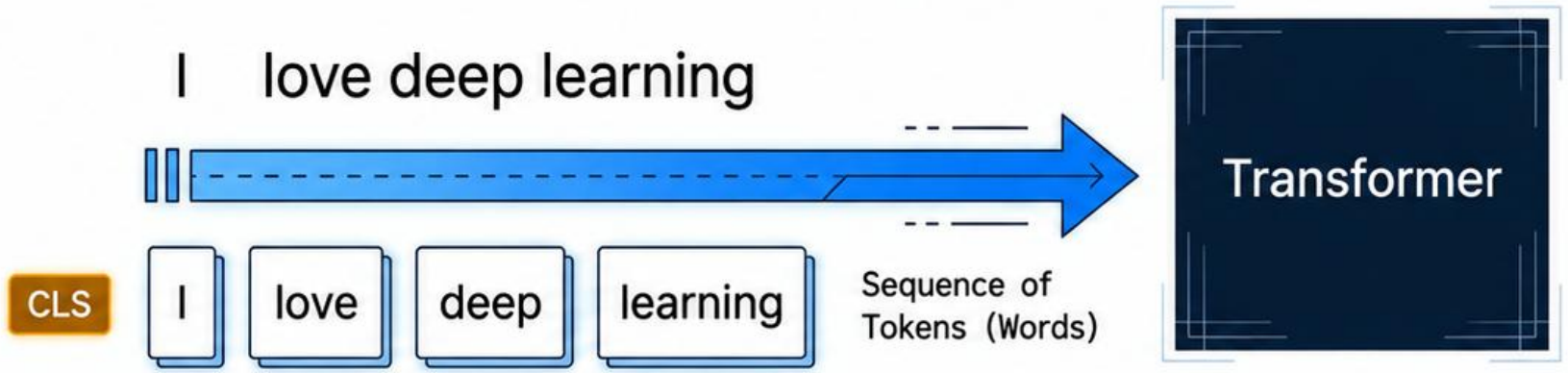
(QA 및 자연어 추론 능력 향상)

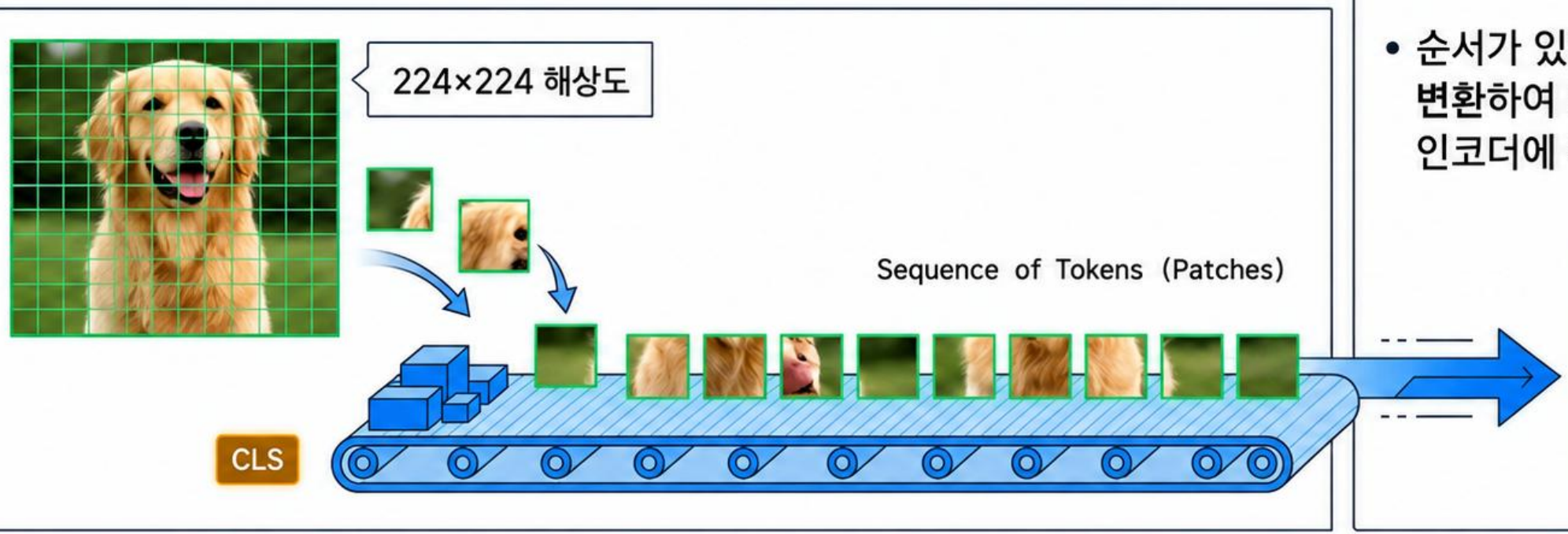
Vision Transformer (ViT) 해부 및 구현

An Image is Worth 16×16 Words — NLP의 거인이 이미지를 정복하는 방법 (ICLR 2021)



이미지를 단어들의 시퀀스(Sequence)로 취급하다

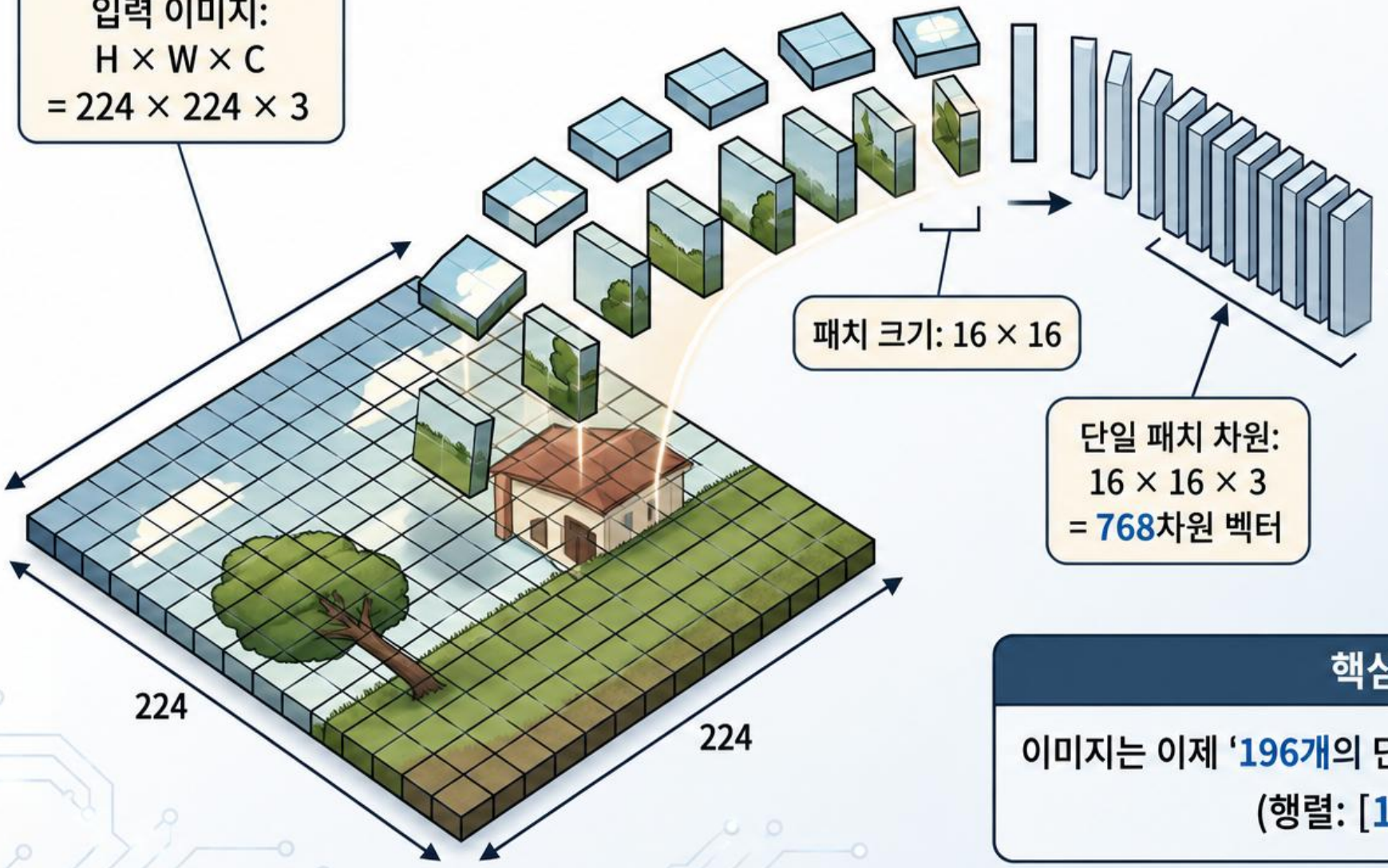




- CNN의 픽셀 단위 연산을 버리고, 이미지를 고정 크기의 패치(Patch)로 분할.
- 텍스트의 단어(Word) = 이미지의 패치(Patch)
- 순서가 있는 토큰 시퀀스로 변환하여 트랜스포머 인코더에 직접 입력.

이미지를 단어의 시퀀스로 해체하다

입력 이미지:
H × W × C
= 224 × 224 × 3



패치 크기: 16 × 16

단일 패치 차원:
16 × 16 × 3
= 768차원 벡터

토큰 개수 (N):
(224/16) × (224/16) = 196개



핵심 통찰:

이미지는 이제 '196개의 단어로 이루어진 문장'이 됩니다.
(행렬: [196, 768])

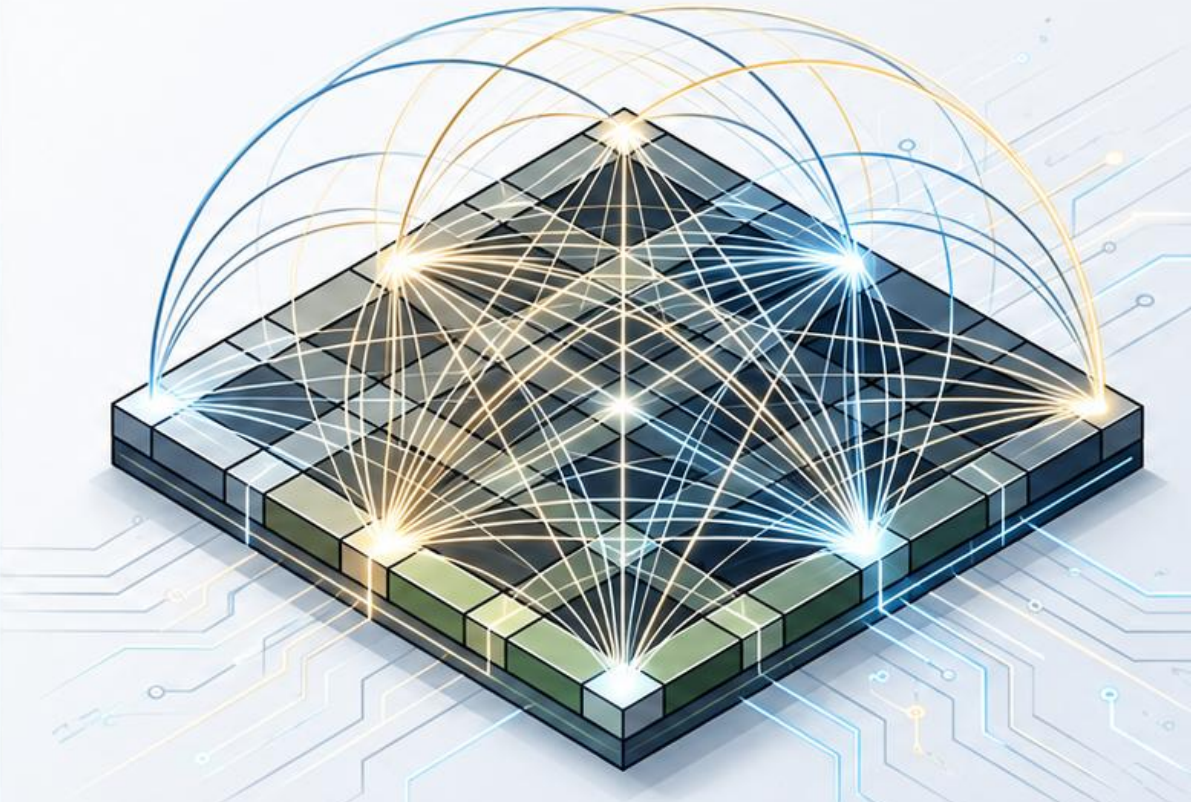
귀납적 편향(Inductive Bias)을 넘어서

CNN: 과거의 표준



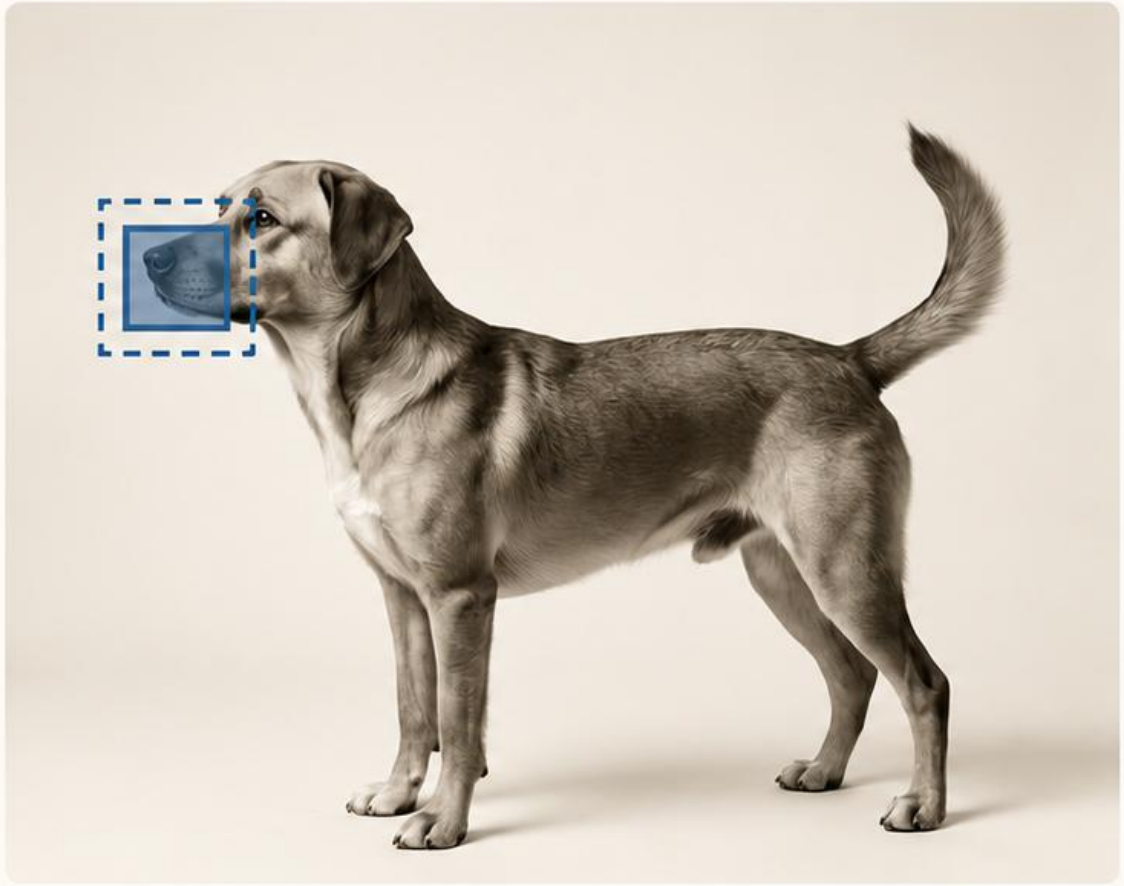
- **특징:** 지역적 특징 추출 (Local features), 귀납적 편향 및 이동 변변성(Translation equivariance) 의존.
- **한계:** 전역적(Global) 문맥을 한 번에 파악하기 어려움.

Vision Transformer: 새로운 패러다임

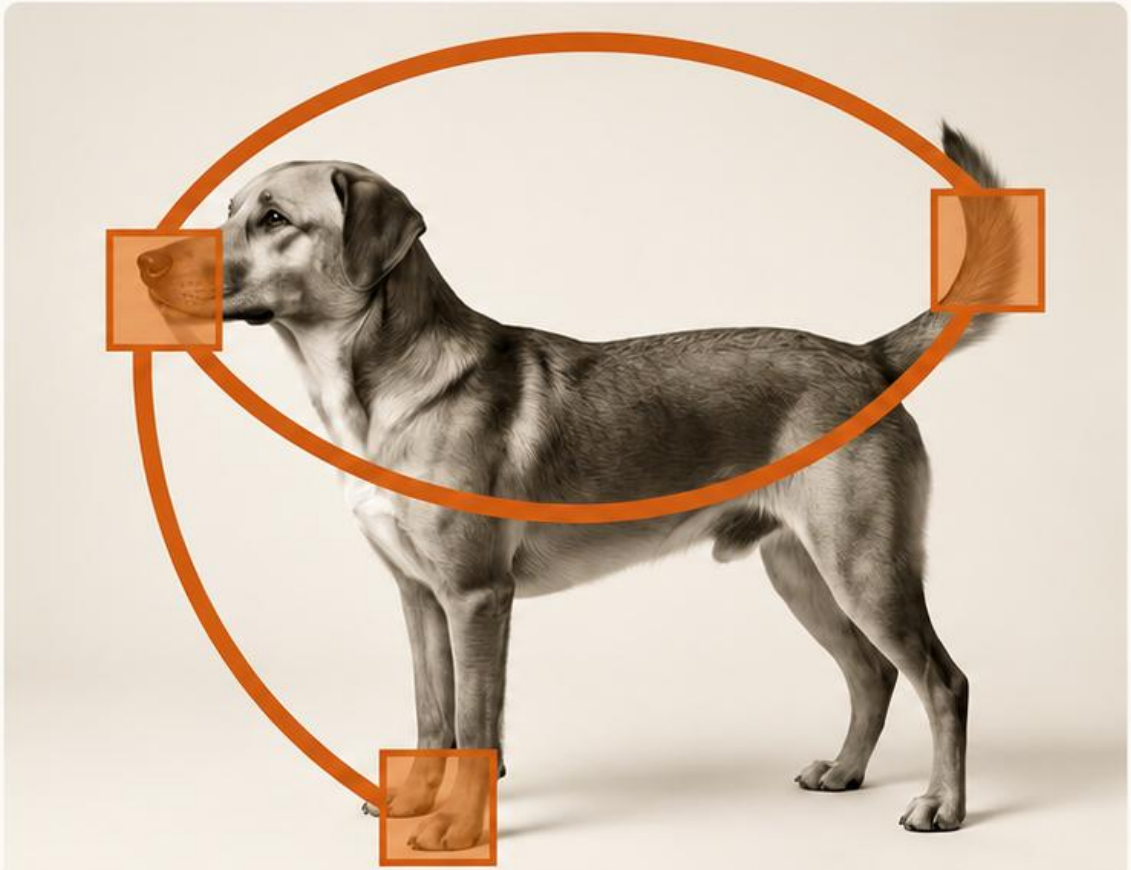


- **특징:** 전역적 문맥(Global context) 학습, Self-Attention을 통한 데이터 기반 관계 추론.
- **장점:** 수작업으로 설계된 기하학적 편향을 제거하고 End-to-End 학습 가능.

수용 영역(Receptive Field): 무엇을 볼 수 있는가?



CNN (국소적 관찰): 초기 레이어에서는 오직 인접한 픽셀(코의 질감)만 볼 수 있습니다. 전체를 보려면 깊은 레이어를 통과해야 합니다.



ViT (전역적 연결): 첫 번째 레이어부터 코와 꼬리, 발 사이의 관계(Self-Attention)를 즉각적으로 계산합니다.

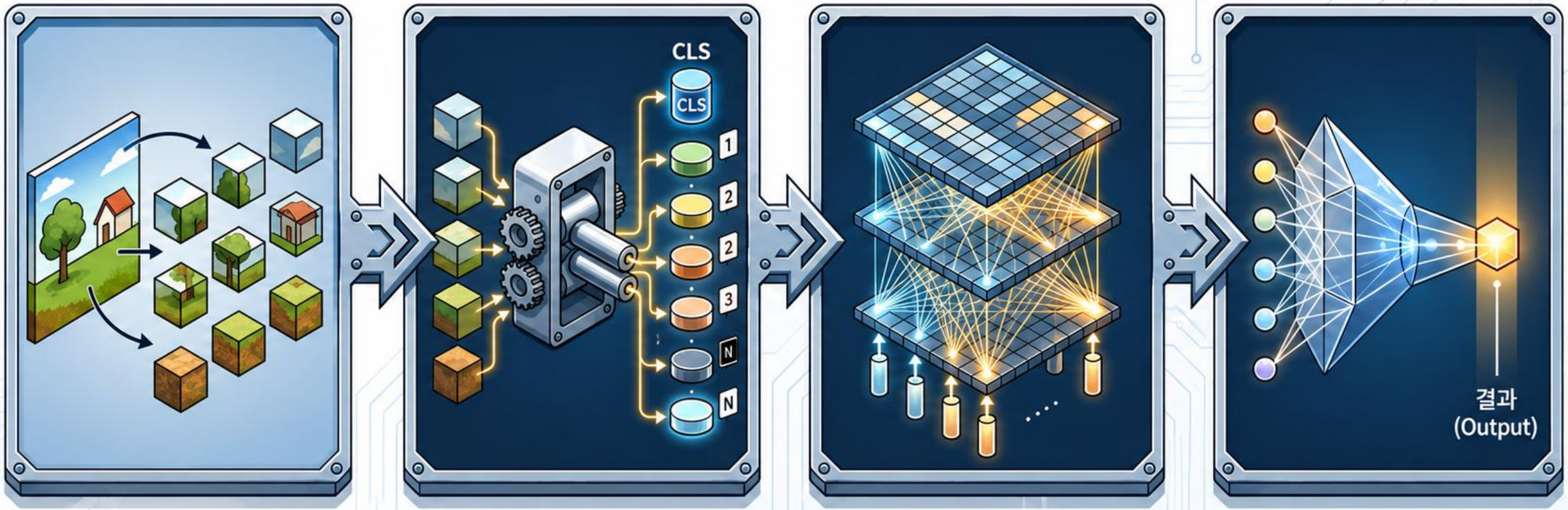
ViT 아키텍처: 데이터 조립 공정의 청사진

Patchify
(패치 분할)

Embed & Pos/CLS
(임베딩 및 문맥 부여)

Multi-Head Attention
(전역적 관계 학습)

MLP / Classification
(비선형 변환 및 분류)



Patchify
(패치 분할)

Embed & Pos/CLS
(임베딩 및 문맥 부여)

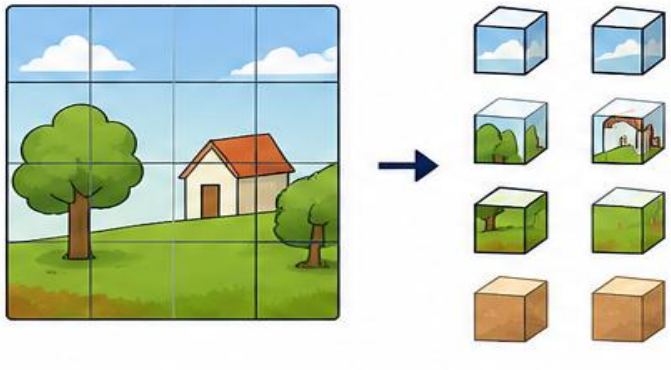
Multi-Head Attention
(전역적 관계 학습)

MLP / Classification
(비선형 변환 및 분류)

Embed & Pos/CLS의 의미

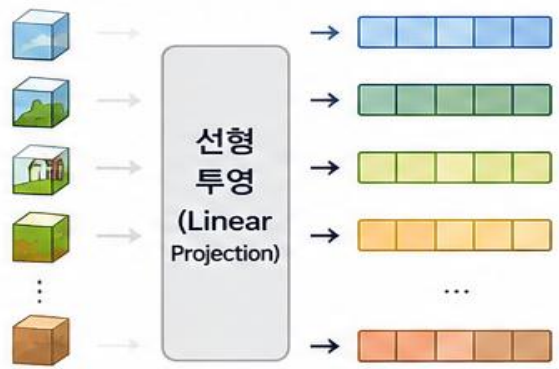
이미지의 각 패치를 숫자 벡터(임베딩)로 바꾸고, 순서 정보(위치 임베딩)와 전체 요약 토큰(CLS)을 더하는 과정

1. 이미지 → 패치 분할



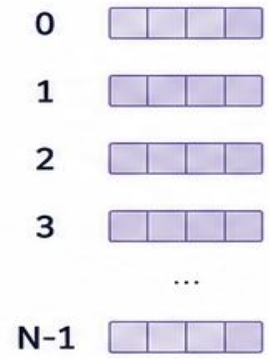
이미지를 고정 크기 패치
(예: 16×16)로 나눕니다.

2. 임베딩 (Embedding)



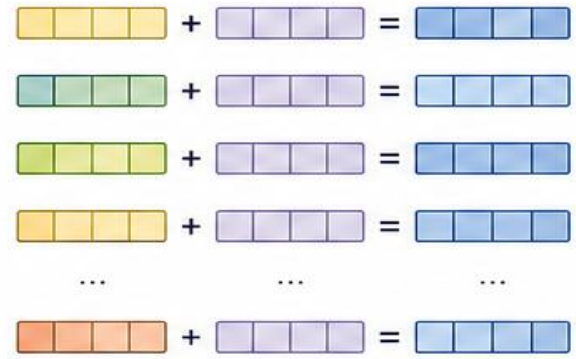
각 패치(또는 토큰)를 D차원의 벡터로 변환합니다.
(예: D = 768)

3. 위치 임베딩 (Position Embedding)



패치의 순서(위치) 정보를
D차원의 벡터로 만듭니다.

4. 결합 (합산)

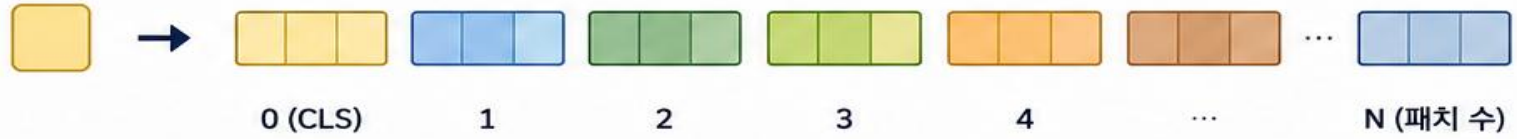


임베딩 벡터와 위치 임베딩을
같은 차원에서 더합니다.

CLS (Classification Token) : 전체를 대표하는 토큰

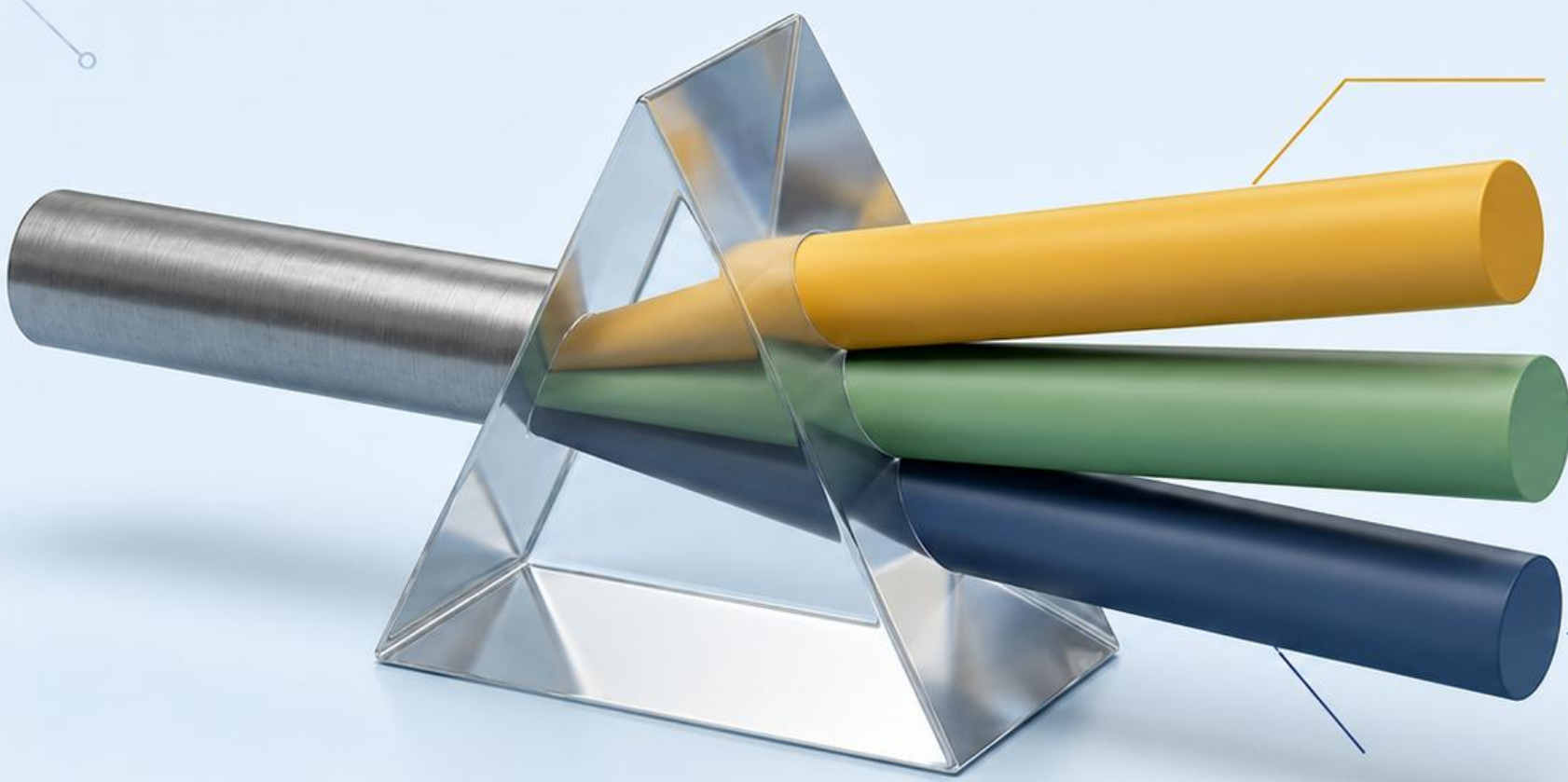
특별한 [CLS] 토큰을 맨 앞에 추가하여, 모든 패치의 정보를 요약·집약하는 역할을 합니다. 보통 분류, 점색, 전체 표현 추출 등의 작업에 사용됩니다.

학습 가능한
CLS 토큰



핵심 포인트 임베딩은 '의미'를, 위치 임베딩은 '순서'를, CLS 토큰은 '전체 요약'을 담당합니다.
이 세 가지가 합쳐져 Transformer가 이미지를 이해할 수 있는 입력이 됩니다.

토큰의 3분할: 질문, 특징, 그리고 정보



Q Q (Query / 질의):
내가 무엇을 찾고 싶은지
(Search intent)

K K (Key / 키):
내가 어떤 특징을
가지고 있는지 (Identity)

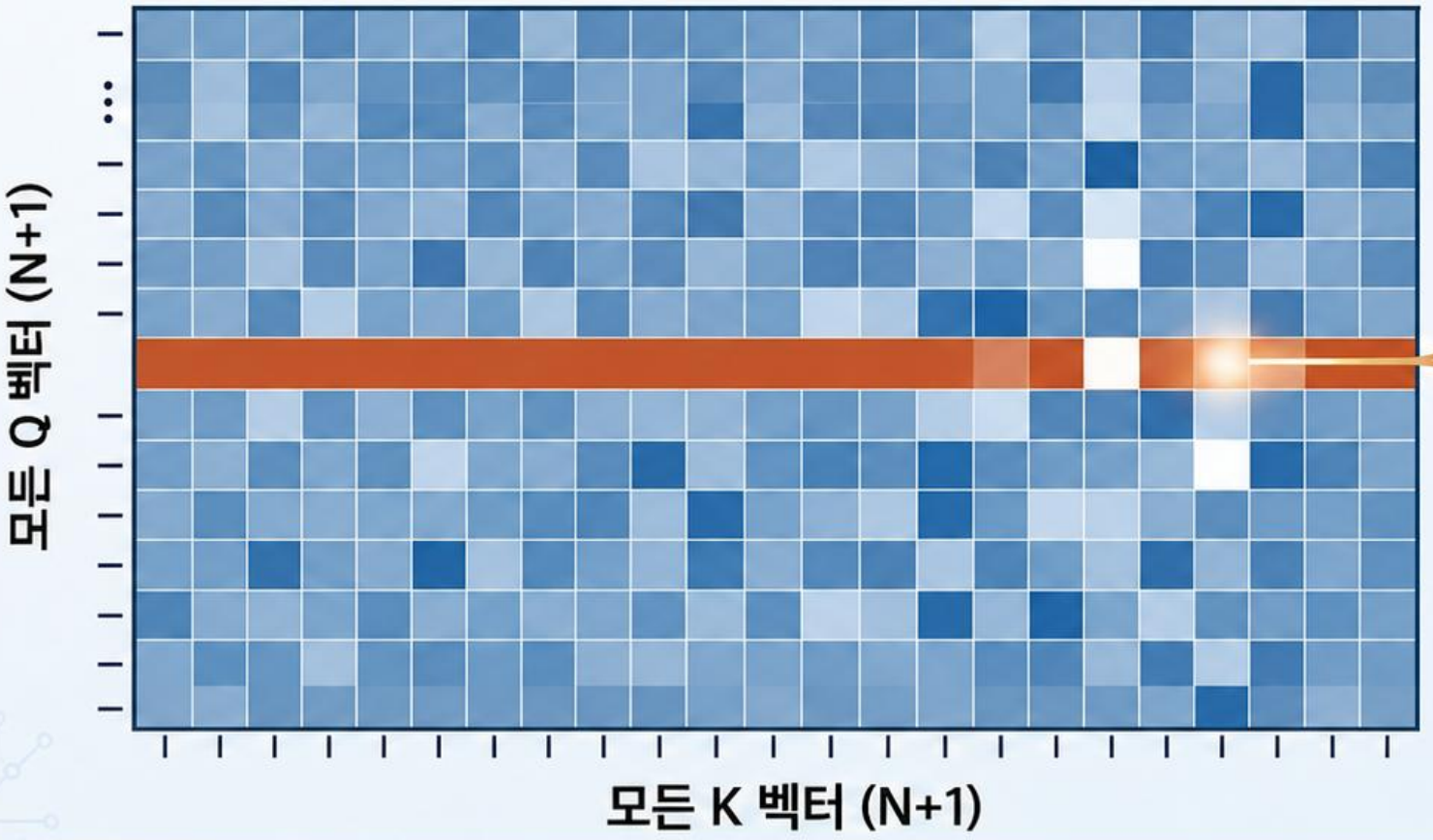
V V (Value / 값):
내가 실제로 가진 정보
(Content)

모든 N+1개의 토큰은 각자의 Q, K, V 벡터를 생성합니다.
이를 통해 패치들은 서로 대화할 준비를 마칩니다.

토큰1 토큰2 토큰3 ... 토큰N 토큰N+1

전역적 관계의 매핑: 누가 누구에게 집중하는가?

$Q \cdot K \rightarrow$ Softmax $\rightarrow$ Attention Score



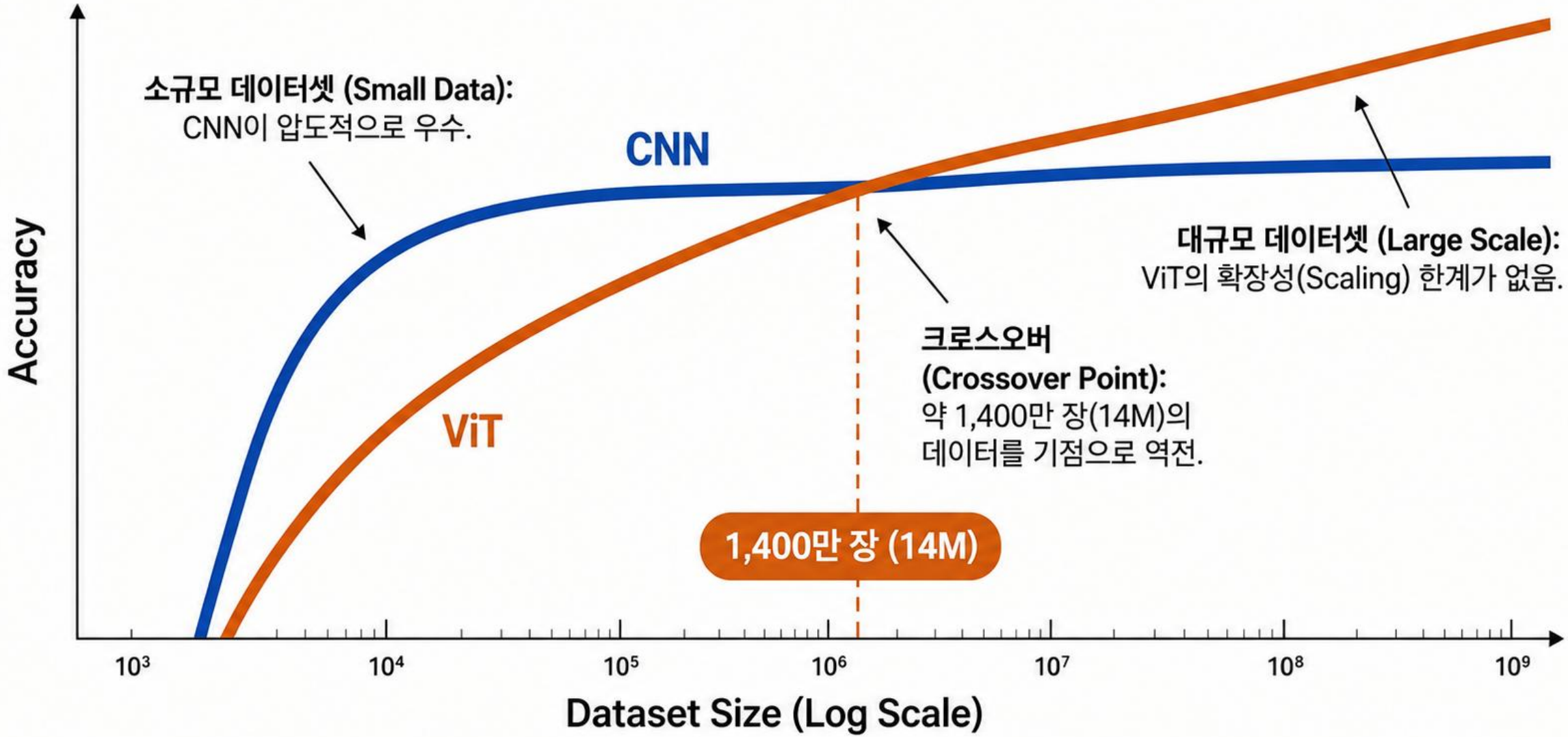


이 Score 행렬이 V(Value) 벡터의 가중치가 됩니다.
CNN과 달리, ViT는 첫 번째 레이어부터 196개 패치 전체의 관계(Global Context)를 한 번에 계산합니다.

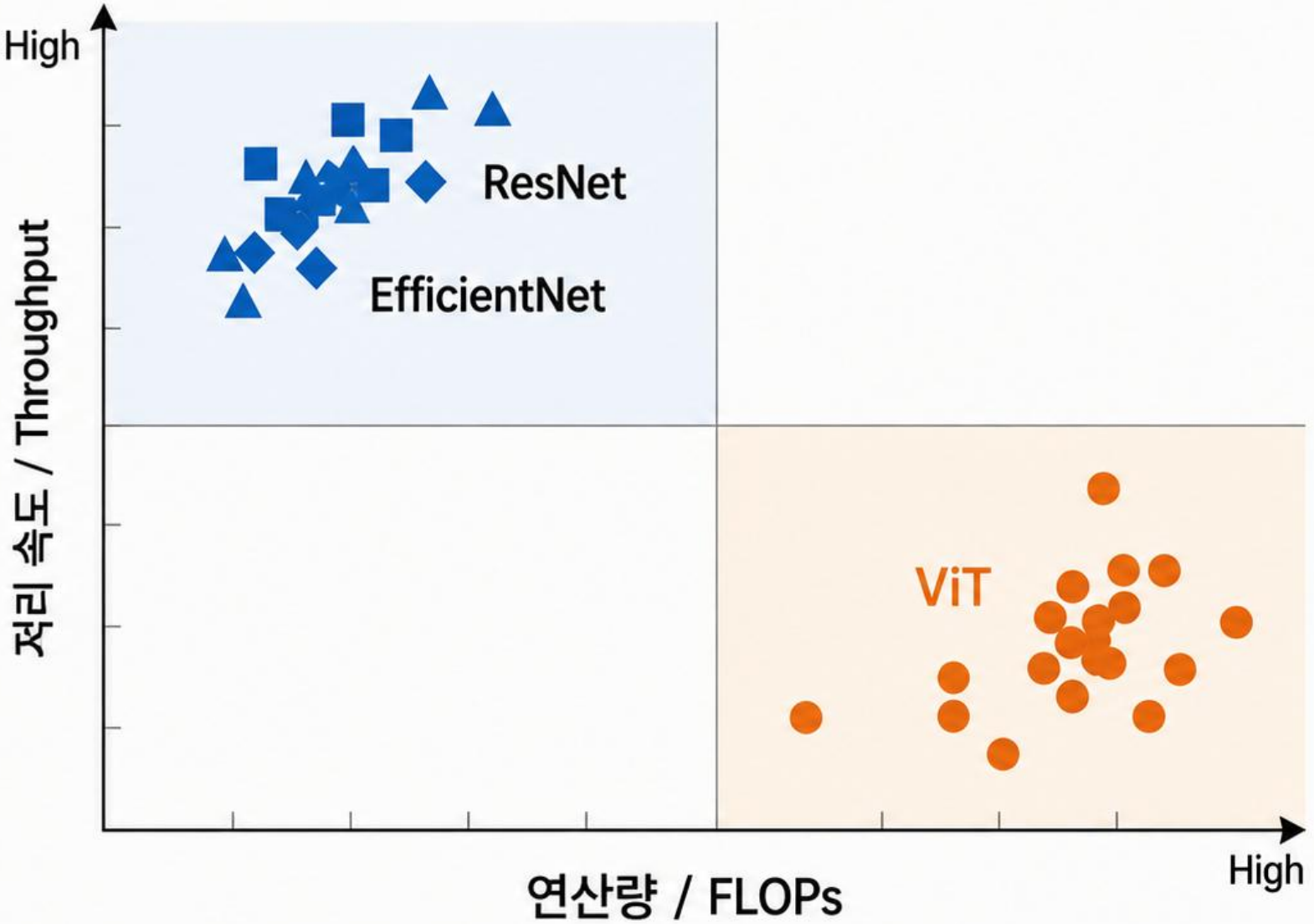
아키텍처 진단 매트릭스

	CNN	ViT
귀납적 편향 (Inductive Bias)	 높음 (High) - 가정 기반	 낮음 (Low) - 데이터 의존
데이터 요구량 (Data Hunger)	 중간 (Moderate) - 소규모 데이터에 강함	 극상 (Extreme) - 초거대 데이터 필요
공간적 맥락 (Spatial Context)	 국소적 (Local)	 전역적 (Global)
하드웨어 효율성 (Hardware Efficiency)	 고도로 최적화됨 (Highly Optimized)	 연산량 과다 (Computationally Heavy)
왜곡에 대한 강건성 (Robustness to Distortions)	 낮음 (Lower)	 높음 (Higher)

데이터와 성능의 상관관계: 1,400만 장의 전환점



실제 운영 환경에서의 제약: 연산 비용과 속도



CNN의 이점

엣지 디바이스(모바일, IoT) 및 제한된 GPU 환경에 고도로 최적화되어 있습니다.

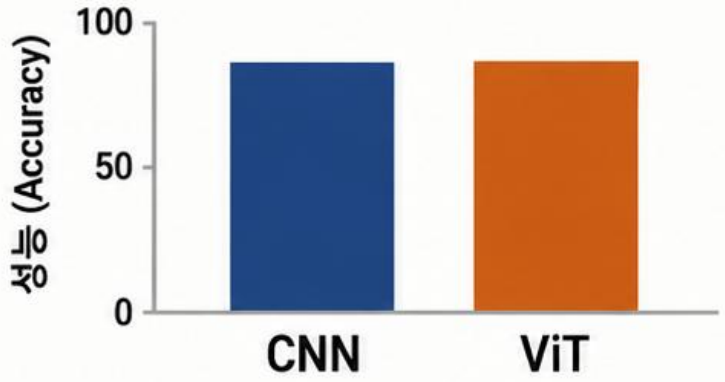
ViT의 한계

Self-Attention의 연산 복잡도($O(N^2)$)로 인해 막대한 메모리와 고성능 TPU/GPU 클러스터가 요구됩니다.

실세계 노이즈 및 왜곡에 대한 강건성 (Robustness)

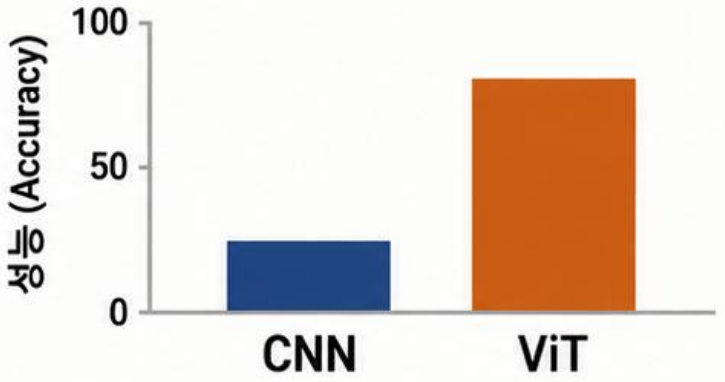
노이즈 없음 (깨끗한 이미지)





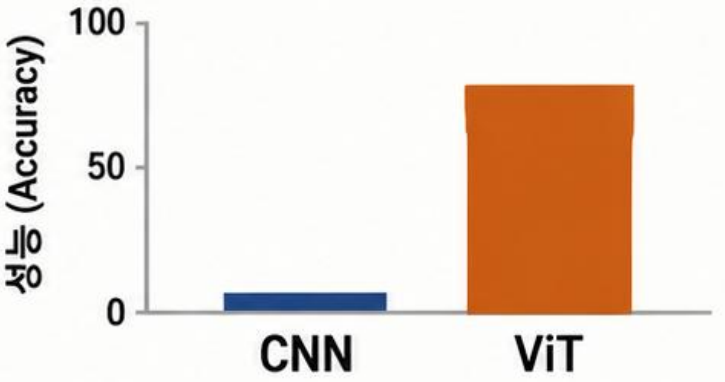
강한 노이즈 (Gaussian Noise)





부분 가림 (Occlusion)

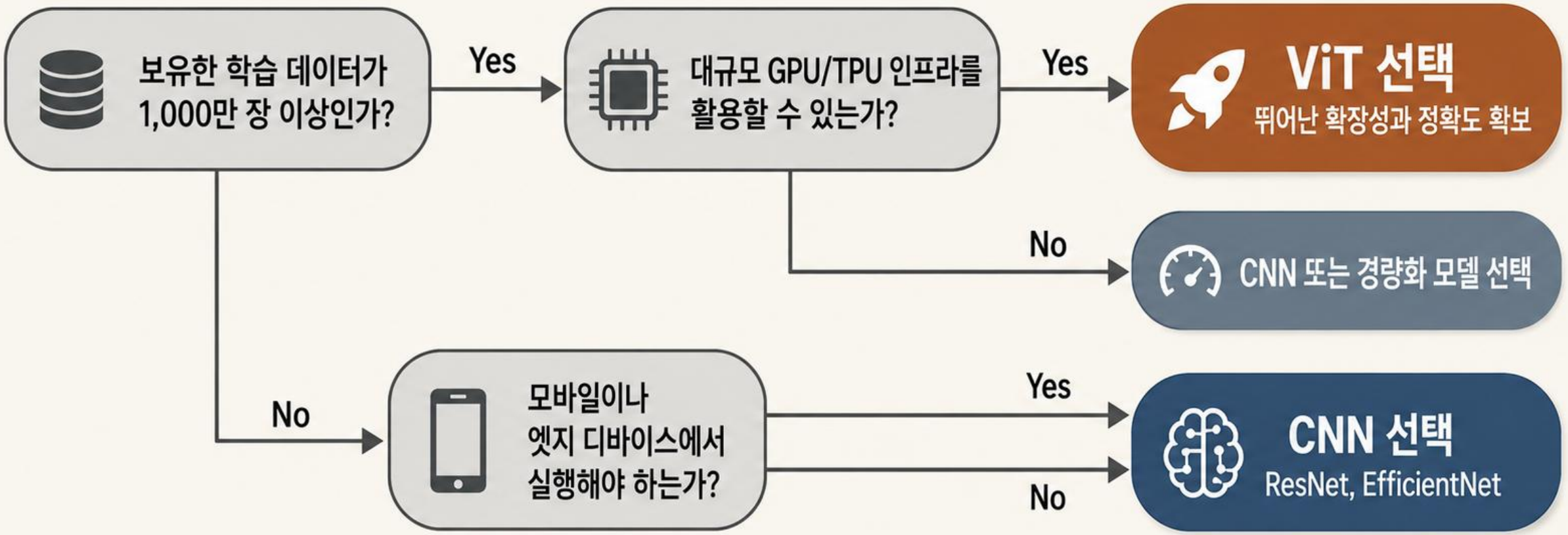




CNN: 텍스처(Texture)와 국소적 패턴에 크게 의존하여, 노이즈나 부분적인 가려짐(Occlusion)에 취약합니다.

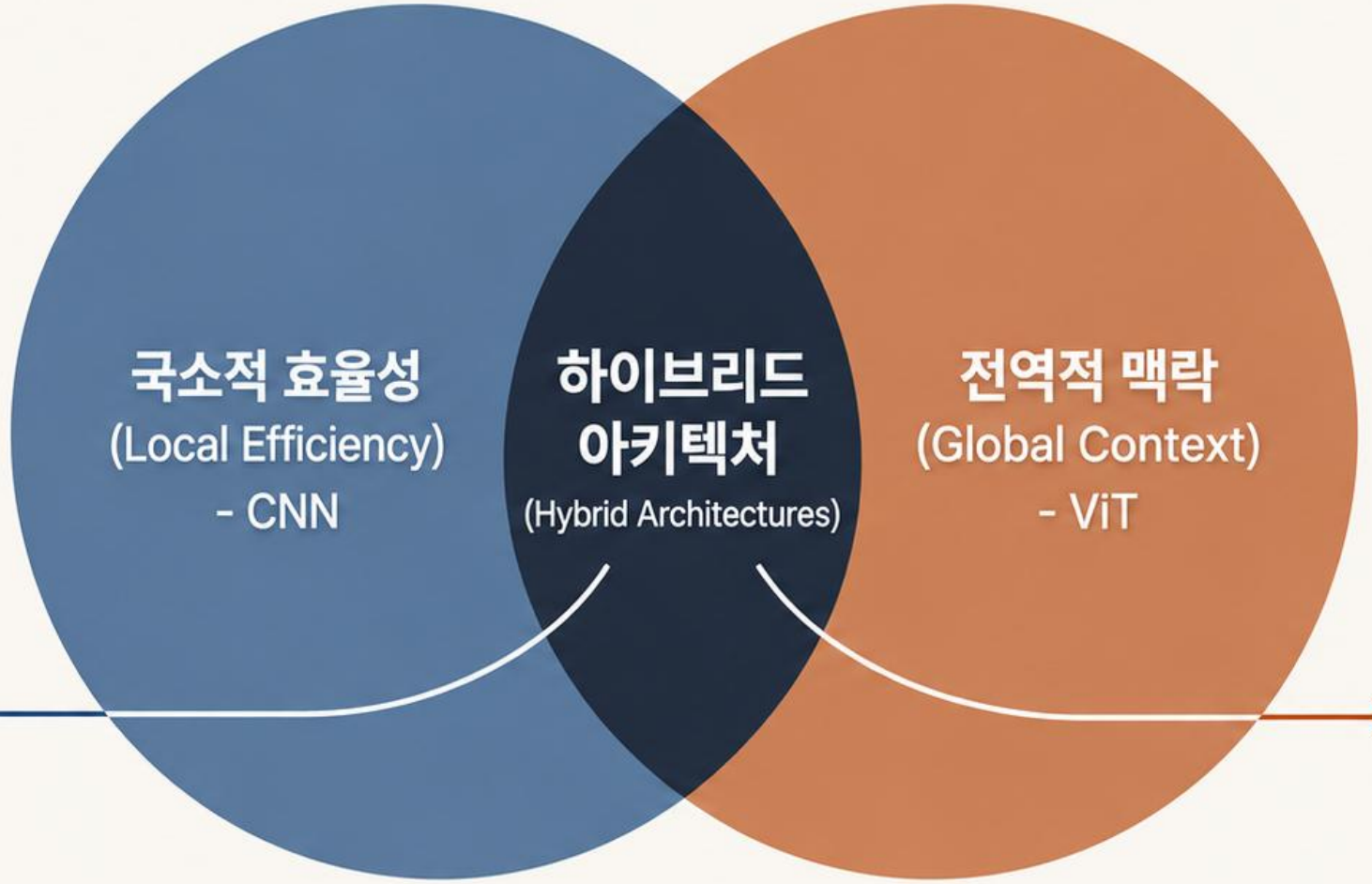
ViT: 형태(Shape)와 전역적 구조를 파악하므로, 데이터가 손상되거나 방해물이 있어도 정확도를 유지합니다.

프로젝트를 위한 아키텍처 선택 가이드



데이터 자산과 컴퓨팅 인프라의 규모가 최종 결정의 핵심 기준입니다.

진화의 방향: 하이브리드 아키텍처의 부상



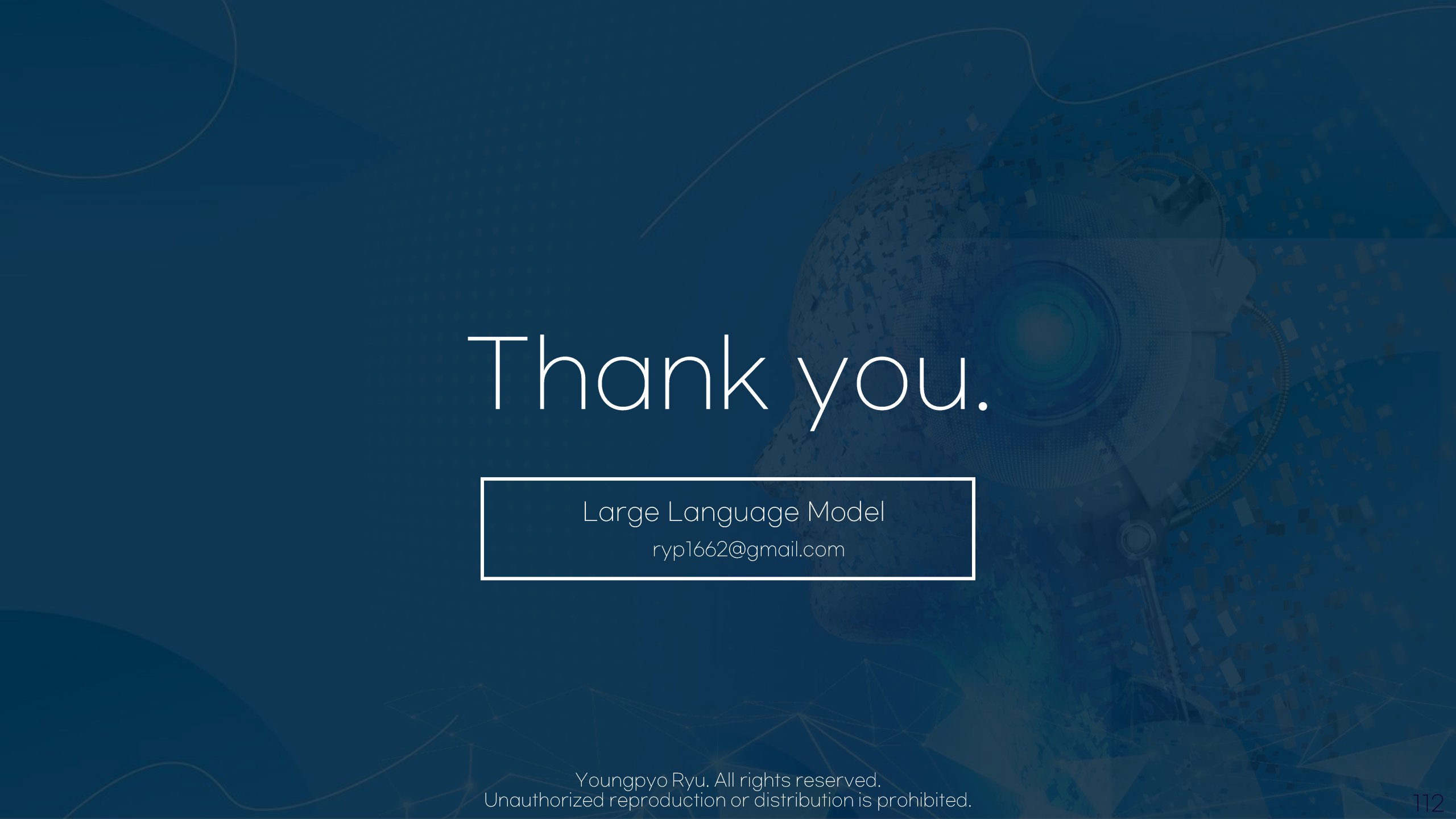
ConvNeXt:

CNN의 구조를 유지하면서 ViT의 설계 기법을 도입하여, 트랜스포머급 성능을 달성한 현대적 CNN.

CvT / MaxViT:

트랜스포머 내부에 합성곱(Convolution) 연산을 결합하여 데이터 효율성과 전역적 어텐션을 동시에 확보.

두 패러다임은 서로를 대체하는 것이 아니라,
상호 보완하며 새로운 컴퓨터 비전의 표준을 만들고 있습니다.



Youngpyo Ryu. All rights reserved.
Unauthorized reproduction or distribution is prohibited.